NUMERICAL COMPUTING IN C

Polyphase Float

User Guide
and Reference

Arbitrary precision. Explicit rounding.
Checked numerical workflows.

Komorebi P.

A practical reader's manual with worked C programs
and a complete core function reference.

Contents

Reading this manual 1

1 Finding your way 3

2 Building, installing, and linking 8

3 Numbers, contexts, and ownership 14

4 Rounding, flags, and exceptional results 20

5 First complete programs 27

6 Arithmetic, rounding, and exponent ranges 33

7 Constants and certified intervals 40

8 Elementary functions without avoidable information loss 46

9 Special functions and difficult domains 51

10 Numerical recipes and reproducible results 57

11 Text, formatting, and binary interchange 62

12 Memory ownership and in-place operations 68

13 Threads, runtime policy, and cache lifetime 73

14 Application integration and performance measurement 78

15 Troubleshooting and diagnostic workflow 83

16 Numerical limits and admission policies 90

17 Core API reference 94

18 Alphabetical function index **276**

19 Types and named constants **287**

20 Complete example programs **295**

21 Glossary **311**

Reading this manual

Polyphase Float is a C library for computations whose precision, rounding, and resource use need to be explicit. It stores binary floating-point values at a precision chosen by the application and provides arithmetic, elementary and selected special functions, constants, conversion, text formatting, and binary interchange. Its core uses GMP for exact integer machinery and does not call MPFR. An optional adapter moves values between the two libraries.

The first part of this book develops a working application from initialization through checked results and cleanup. The middle chapters explain numerical workflows, storage, threads, and integration. The final part contains the numerical resource policies, every core function declaration, an alphabetical symbol index, and a glossary. You can read straight through or use the function reference beside an existing program.

Begin with the first-program chapter if the library is already installed. Read the rounding and error chapter before interpreting numerical return values. Read the memory and overlap chapter before replacing inputs in place. If a calculation returns an error, use the troubleshooting chapter and the numerical limits rather than assuming that more precision is always the remedy.

The source package includes the complete programs in `examples/manual/`. Short excerpts in the prose are teaching aids. Build and run the complete file when you want its full setup, checks, and cleanup. Each example has a named source file and an expected result that you can compare with your own run.

0.1 Scope and conventions

The qualified baseline is Linux x86-64 ELF with the LP64 data model and 64-bit GMP limbs. Linux under WSL follows that same target contract. Native Windows, macOS, ARM64, big-endian, and 32-bit targets require separate qualification. OpenMP and the MPFR adapter are optional. Portable-looking source code alone does not establish support for every platform on which it can be compiled.

The manual describes the governed core API and ABI epoch 1. The source package's version macros and release record identify the particular library build. Precision is measured in bits unless decimal digits are stated explicitly. An equation describes an exact mathe-

matical value; a stored `ppf_t` describes its finite binary representation. These are different objects, and much of reliable numerical programming consists of keeping the distinction visible.

The library has finite domain and resource policies. A successful certified path and a bounded numerical rejection are different outcomes. A complete API inventory is useful for navigation, but is not a whole-library formal proof. The numerical-limits appendix separates documented behavior from remaining review and boundary-evidence obligations.

0.2 License and attribution

The library and accompanying source are distributed under LGPL-3.0-or-later. See `LICENSE.md`, `LICENSE`, `COPYING`, and `THIRD_PARTY_NOTICES.md` in the source package for the applicable terms and third-party notices. The release author uses the name Komorebi P.

Chapter 1

Finding your way

Polyphase Float is a C library for binary floating-point arithmetic with a precision you choose for each number. It provides arithmetic, roots, mathematical constants and functions, conversions, and input/output through the `ppf_t` number type. A calculation can use more significand bits than the machine's ordinary `double`, and it can choose how each result is rounded.

This book takes you from a working installation to calculations whose precision, ownership, and error handling you can explain. The first five chapters form a practical introduction. Later chapters develop the arithmetic families, numerical limits, interchange formats, and advanced interfaces. The reference entries are there when you need the exact declaration or contract of an individual function.

You do not need to understand the implementation's multiplication algorithms to use the library. You do need to distinguish a number's precision from its printed length, initialize and clear owned objects, and interpret return values correctly. Those three habits account for much of the introductory material.

1.1 Choose a reading route

Your starting point	Suggested route
New to C numerical libraries	Build and run the supplied example, read objects and contexts, then work through all four introductory programs.
Comfortable with C, new to arbitrary precision	Read the precision discussion below and the rounding chapter before adapting the first program.
Moving from another multiprecision library	Read ownership, explicit versus default contexts, return-value conventions, and the function reference for each migrated call.
Integrating an existing application	Start with the installed CMake consumer, then review thread ownership, error handling, and the exact contracts of your operation families.
Investigating an unexpected answer	Check the original input, destination precision, intermediate rounding, flags, and result category in that order.

The examples deliberately use small, inspectable calculations. Their simplicity makes API mistakes visible. Once those patterns are familiar, use the same initialization, return checking, and cleanup structure around a larger algorithm.

If you are reading a PDF, the source links identify complete files in the accompanying source tree. Commands run from the source directory unless a section explicitly changes directory. Shell prompts such as `$` are omitted so that commands can be copied directly.

1.2 The four ideas behind a calculation

A typical operation combines four things:

1. **Operands** contain the values already stored in the input objects.
2. **A destination** supplies the precision of the result.
3. **A rounding mode** chooses a representable answer when the exact result does not fit.
4. **A context** supplies the exponent range, default policy, allocation callbacks, and diagnostic flags.

For example, `ppf_mul(&result, &a, &b, PPF_RNDN, &ctx)` multiplies the stored values of `a` and `b`, rounds into the existing precision of `result`, and uses `ctx` for the calculation. The `&` operator passes the address of an object. The first argument is the object to be written; the following number arguments are inputs.

An operand's high precision does not automatically make the destination high precision. A 256-bit input multiplied into a 32-bit destination produces a 32-bit result. The context's default precision does not resize an existing destination either. Choosing the destination is part of choosing the numerical calculation.

1.3 Bits, decimal digits, and magnitude

Precision is measured in **binary significand bits**. It describes the amount of significant binary information a regular value can hold, not the number of digits after the decimal point. The exponent controls magnitude separately. A very large value and a very small value can therefore use the same precision.

A useful conversion is

$$\text{decimal-digit equivalent} \approx p \log_{10}(2),$$

where p is the number of significand bits.

Precision	Approximate decimal-digit equivalent
24 bits	7.22
53 bits	15.95
80 bits	24.08
128 bits	38.53
256 bits	77.06

These are measures of storage resolution. They are not guarantees that an arbitrary calculation has that many correct decimal digits. Inaccurate input, cancellation, and an ill-conditioned problem can reduce the useful accuracy of a result.

To budget space for roughly d decimal significant digits, start with $\lceil d \log_2(10) \rceil$ bits, then allow additional working precision according to the algorithm. For example, 50 decimal digits correspond to about 167 bits before any allowance for intermediate error. There is no universal number of extra bits that makes every algorithm reliable.

Some values are exactly representable in a finite binary significand. One half is exact because it is a power of two. One tenth and one third have nonterminating binary expansions, so storing them at a finite precision generally involves rounding. Printing more decimal digits can reveal the stored approximation; it cannot create missing information.

The default precision is 53 bits. That familiar size does not make every aspect of the object model identical to a C `double`. Polyphase Float has its own exponent range, explicit rounding controls, flags, and special-value rules.

1.4 What the core depends on

The production core uses GMP for integer storage and arithmetic support. MPFR is optional: it supports an interoperability adapter and differential tests. You can build and use the core without an MPFR runtime dependency. The build chapter starts with that smaller configuration.

The supported baseline is Linux x86-64 with the LP64 data model and 64-bit GMP limbs. LP64 means that pointers and C `long` integers are 64 bits. Windows readers can follow the Linux workflow inside an x86-64 Ubuntu environment under WSL; native Windows toolchains are outside this baseline. The installation chapter makes the shell boundary explicit.

The public API includes finite-precision arithmetic and bounded algorithms. Some mathematical-function implementations can reject requests outside their practical limits or after a finite refinement budget. The relevant reference entries describe those limits and failure behavior. A wide representable exponent range does not imply that every mathematical function can successfully evaluate every representable argument.

1.5 Read results at three levels

When an operation finishes, ask three separate questions:

- **Did the operation report an unrecovered failure?** Ternary-returning operations use `PPF_TERNARY_ERROR` for this purpose.
- **What kind of result was produced?** A result may be regular, zero, infinity, or NaN.
- **What numerical diagnostics were raised?** Context flags record conditions such as inexact rounding, overflow, or division by zero.

A nonzero ternary result is usually a rounding direction, not a failure. Conversely, a special value may be produced through the documented arithmetic rules without returning `PPF_TERNARY_ERROR`. The rounding chapter develops both distinctions with runnable examples.

The word **exact** also needs a reference point. If an input string has already been rounded into an eight-bit object, copying that stored value to a 128-bit object may be exact. The copy still contains the original eight-bit approximation. Exactness of one operation does not recover information lost earlier.

1.6 A small vocabulary

A **significand** carries the significant digits of a number; an **exponent** scales that significand by a power of the radix. The **radix** is the numeral-system base, such as two or ten. A **destination** is the initialized object an operation writes. A **rounding mode** chooses among nearby representable results. A **ternary result** describes whether the stored result is below, equal to, or above the exact result of the operation. A **sticky flag** remains set until explicitly cleared or restored. An **owned object** is responsible for storage that must eventually be released.

These terms recur because they describe different responsibilities. Precision belongs to a number; flags belong to a context; storage belongs to an owner; and mathematical accuracy belongs to the complete calculation.

1.7 Before moving on

You are ready to begin when you can identify the source directory, choose a Linux build environment, and set aside a separate build directory. In the next chapter you will compile the library, run an example, and link a small application against an installed package. You can return to the numerical discussion after seeing that first program run.

Check your understanding. Does requesting 40 decimal digits of printed output make a 53-bit number more precise?

Answer. No. It changes the text representation. To retain more information, initialize the destination at a higher binary precision and compute or parse the value at that precision. Widening an existing approximation alone does not recover its discarded information.

Chapter 2

Building, installing, and linking

The goal of this chapter is a working C program linked to Polyphase Float. Start with the serial core, verify it, and add optional facilities when your application needs them. Keep the source, build output, and installation prefix separate: that makes it easier to repeat a build or diagnose which library an application is using.

A **source directory** contains `CMakeLists.txt`, `include/`, and `src/`. A **build directory** contains generated files and compiled binaries. An **installation prefix** is the destination tree from which another project consumes the headers, libraries, and CMake package. Building does not automatically install the library.

2.1 Prepare the system

The supported baseline is Linux x86-64 ELF, using LP64 and 64-bit GMP limbs. You need:

Component	Purpose
C11 compiler, such as GCC or Clang	Compile the library and application
CMake 3.23 or newer	Configure, build, test, and install
Make or Ninja	Execute the generated build rules
Official GMP development headers and library	Build the core arithmetic library
POSIX threads and the platform math library	Provide system facilities used by the core
Python 3	Run source audits and Python-based tests

MPFR development files are optional for ordinary core use and required for the full differential-test and adapter route described below. A development package includes headers as well as a linkable library. Having only a runtime shared library is insufficient for a normal development installation.

On an Ubuntu system with package access, the following installs the basic development tools:

```
sudo apt-get update
sudo apt-get install build-essential cmake libgmp-dev pkg-config python3
```

Check the installed tools before configuring:

```
cc --version
cmake --version
python3 --version
```

Use your distribution's supported package sources. Package versions differ by distribution release; the CMake version must meet the minimum above. The Ubuntu package index documents the packages available for each release.

Windows: enter the Linux environment first

On a supported Windows installation, open PowerShell as an administrator and run:

```
wsl --install
```

Follow the restart and Ubuntu first-run prompts, including creation of a Linux user account. Microsoft maintains the WSL installation instructions, including routes for systems that need additional setup. This bootstrap normally downloads components and is separate from building Polyphase Float.

After Ubuntu opens, run the Linux commands in this chapter **inside Ubuntu**, not in PowerShell. Use an x86-64 Linux environment and keep the project in its Linux home directory. This is a Linux build on a Windows host; it does not provide a native Windows DLL or establish support for an ARM machine. Verify the build and tests in your own WSL environment as you would on any other Linux installation.

Preparing an offline build

The core build does not need to fetch a dependency from the network once the source archive, compiler, CMake, build tool, and GMP development files are present. Arrange those prerequisites before disconnecting. On a managed offline system, obtain the matching distribution packages and their dependencies through your administrator or an approved package cache. Copying only `libgmp.so` from another machine does not supply the headers or ensure an ABI-compatible development environment.

The package-install commands above require access to configured package repositories unless the necessary packages are already cached. They are preparation commands, not a promise of an offline package installation. After preparation, unpack the complete source archive into a writable directory and run the following build commands there.

2.2 Build and run the core

From the directory containing `CMakeLists.txt`:

```
cmake -S . -B build/release \
  -DCMAKE_BUILD_TYPE=Release \
  -DPPF_OPENMP=OFF \
  -DPPF_MPFR_SUPPORT=OFF \
  -DPPF_BUILD_EXAMPLES=ON \
  -DBUILD_TESTING=ON
cmake --build build/release -j2
./build/release/example_basic
ctest --test-dir build/release --output-on-failure -j1
```

The trailing backslash continues a shell command on the next line; do not put characters after it. `-S .` selects the current source directory. `-B build/release` selects a separate build directory. `-j2` permits two build jobs; reduce it to `-j1` on a memory-constrained machine. The test command uses one test job to make failures easier to follow.

The basic example prints:

```
x (256 bits) = 3.10000e1
y (80 bits)  = 3.10000e1
```

Both objects contain the integer 31 exactly. Their different precisions show that storage precision can differ without changing an exactly representable value. The file `examples/basic.c` includes checked initialization, calculation, output, and cleanup.

This configuration builds the core without MPFR or an OpenMP runtime requirement. Both shared and static core libraries are enabled by default. The tests available in a configured build depend on its capabilities; a core-only test run is useful local validation, but it is not the full MPFR-enabled differential matrix.

If configuration fails, fix that error before running the build command. If compilation fails, read the first compiler error before later messages caused by it. A successful `cmake` configuration alone does not mean the library has compiled or its tests have run.

2.3 Install to your user account

A user-local installation avoids requiring administrator privileges:

```
cmake --install build/release --prefix "$HOME/.local"
```

The prefix contains headers, libraries, package metadata, and documentation. Depending on the platform's installation conventions, the library directory can be `lib` or a nested directory such as `lib/x86_64-linux-gnu`. Use the exported CMake package instead of assuming that directory name in application build scripts.

Verify consumption from the installation, rather than accidentally linking back to the source build:

```
cmake -S tests/install_consumer -B build/install-consumer \
  -DCMAKE_PREFIX_PATH="$HOME/.local" \
  -DPPF_CONSUMER_COMPONENTS=core
cmake --build build/install-consumer -j2
ctest --test-dir build/install-consumer --output-on-failure
```

`CMAKE_PREFIX_PATH` tells CMake where to search for installed packages. It does not modify the system package manager. If you maintain several Polyphase Float installations, choose one prefix deliberately and use a fresh consumer build directory when switching between them.

2.4 Create your own CMake application

Make an application directory alongside the library source directory. Place a copy of `intro_first.c` in it as `main.c`, and add this `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.23)
project(my_calculation LANGUAGES C)

find_package(PolyphaseFloat CONFIG REQUIRED COMPONENTS core)
add_executable(my_calculation main.c)
target_compile_features(my_calculation PRIVATE c_std_11)
set_target_properties(my_calculation PROPERTIES C_EXTENSIONS OFF)
target_link_libraries(my_calculation PRIVATE PolyphaseFloat::shared)
```

From that application directory:

```
cmake -S . -B build -DCMAKE_PREFIX_PATH="$HOME/.local"
cmake --build build -j2
./build/my_calculation
```

Expected output is `2 * 3 = 6.00000e0`. The imported target supplies the library's include paths and transitive link requirements. You do not need to copy headers into the application or hand-maintain GMP and thread link flags in this CMake project.

To select the installed static core, replace `PolyphaseFloat::shared` with `PolyphaseFloat::static`, provided that form was built and installed. Static linking still needs the relevant dependency libraries; the imported target expresses those requirements. The word `static` does not imply that every dependency in the final executable is statically linked.

`COMPONENTS core` makes the dependency choice explicit. An application using the optional MPFR adapter should request the `mpfr` component and use its documented adapter target. The independent core has no need for that component.

2.5 Compile the introductory programs directly

For a quick experiment with the serial build above, run this from the library source directory:

```
cc -std=c11 -Wall -Wextra -Werror -pedantic \
  -Iinclude examples/manual/intro_first.c \
  build/release/libpolyphase_float.a \
  -lgmp -lm -pthread -o build/release/intro_first
./build/release/intro_first
```

The source comes before the static library, and the dependency libraries follow it. That order matters to ordinary static linking. `-Iinclude` locates the public header; it does not locate the compiled library. For the other introductory programs, replace `intro_first` in both the source path and executable name.

This command is intentionally tied to the serial build in this chapter. An OpenMP-enabled static build can require additional compiler and runtime flags. For a maintained application, prefer the CMake imported target so that the selected build's dependencies travel with it.

2.6 Add optional capabilities deliberately

Use a new build directory for a different compiler, sanitizer, or major configuration. CMake caches detected paths and compiler properties; reusing a directory after changing toolchains can produce confusing combinations.

For full differential testing on Ubuntu, first install official MPFR development files:

```
sudo apt-get install libmpfr-dev
```

Then configure a separate validation build:

```
cmake -S . -B build/validation \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo \
  -DPPF_OPENMP=AUTO \
  -DPPF_MPFR_SUPPORT=ON \
  -DPPF_BUILD_EXAMPLES=ON \
  -DPPF_WARNINGS_AS_ERRORS=ON \
  -DBUILD_TESTING=ON
cmake --build build/validation -j2
ctest --test-dir build/validation --output-on-failure -j1
```

`AUTO` enables OpenMP when the compiler probe succeeds; `ON` requires it; `OFF` excludes it. For MPFR, `ON` requires support and `OFF` selects the independent core. Use official MPFR headers when installing the adapter. A compatibility-header validation route does not provide the same installable adapter configuration.

Native CPU tuning is off by default. Keep it off when distributing a binary to machines with potentially different instruction sets. The complete option list, packaging staging com-

mands, and tooling configurations are in Building Polyphase Float (`docs/BUILDING.md`); the test labels and qualification boundaries are in Testing (`docs/TESTING.md`).

2.7 Diagnose the first failure

Symptom	First action
`cmake` is not found or is too old	Install a suitable CMake package and check `cmake --version`.
GMP headers or library cannot be found	Install the development package; check that headers and library target the same architecture.
`PolyphaseFloatConfig.cmake` cannot be found	Install the library, then pass its prefix through `CMAKE_PREFIX_PATH`.
The compiler cannot find `polyphase_float.h`	Use the imported CMake target, or check `-Iinclude` and your current directory.
The linker reports undefined `ppf_` symbols	Confirm that the library is linked and matches the header; for direct static linking, check argument order.
The executable cannot load a shared library	Check the installation prefix and runtime library search configuration; do not substitute an unrelated shared object.
A test fails	Keep the `--output-on-failure` output and record compiler, configuration, and platform before changing settings.

A useful problem report includes the first failing command, its full error message, the configured options, compiler version, and whether the failure occurs during configuration, compilation, linking, execution, or testing. Those stages have different remedies.

Exercise. Why can `#include "polyphase_float.h"` compile successfully while the final link fails?

Answer. The header provides declarations to the compiler. The linker still needs the compiled implementations and their dependencies. Linking `PolyphaseFloat::shared` or `PolyphaseFloat::static` through CMake supplies that second part.

Chapter 3

Numbers, contexts, and ownership

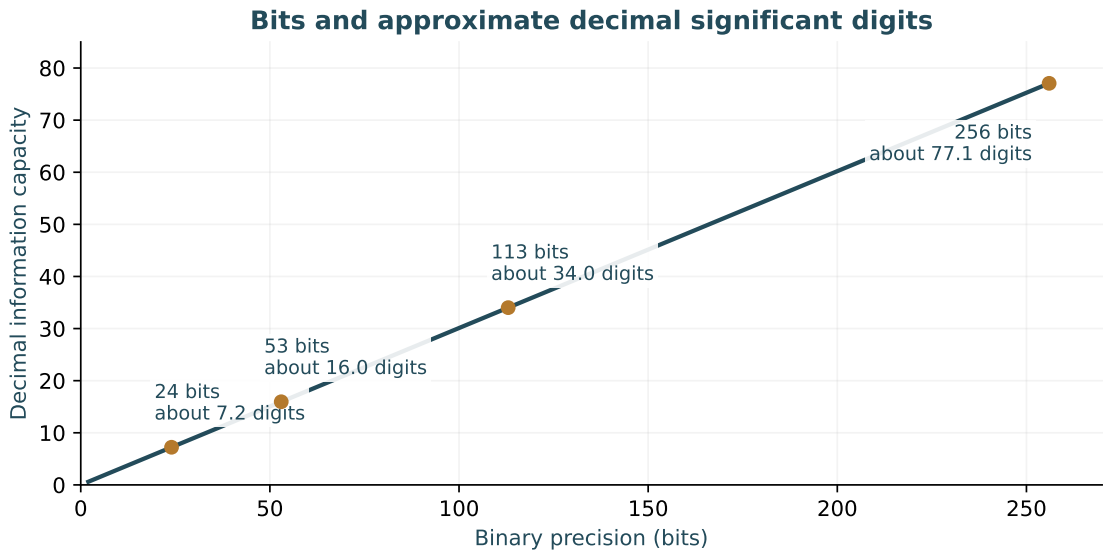


Figure 3.1: Binary precision gives approximately p log10(2) decimal digits of information capacity. This is not an accuracy guarantee for a computed expression.

A `ppf_t` is an owned numerical object, not a plain scalar that you can copy with C assignment. A `ppf_context_t` carries the policy and temporary resources used by operations. Keeping their roles separate makes a program easier to reason about: numbers retain values, while contexts govern how calculations are performed and record what happened.

This chapter uses explicit contexts throughout. That makes the source of precision policy, range limits, flags, and allocations visible at each call site. The thread-local convenience interface is useful too, but mixing the two styles casually is a common source of misleading diagnostics.

3.1 Start with a complete lifetime

A normal object passes through four stages: declare its structure, initialize its storage, assign or calculate a value, and clear it. Here is the pattern in a small scope:

```
ppf_context_t ctx;
ppf_t x = {0};
int status = 1;

if (ppf_context_init(&ctx) != 0) return 1;
if (ppf_init2_ctx(&x, 128, &ctx) != 0) goto cleanup;
if (ppf_set_ui(&x, 7, PPF_RNDN, &ctx) == PPF_TERNARY_ERROR)
    goto cleanup;
status = 0;

cleanup:
ppf_clear(&x);
ppf_context_clear(&ctx);
return status;
```

This is a function-body pattern, not a complete translation unit. The first-programs chapter includes the header, `main`, checked output, and thread cleanup in complete programs.

`ppf_t x = {0}` zero-initializes the C structure so a common cleanup path can safely clear it even if initialization did not complete. It does **not** create a numerical zero or a valid initialized number. Only after successful `ppf_init2_ctx` may ordinary numerical operations use the object.

Successful initialization gives the object its storage and precision and leaves its value as NaN. Assign the intended value before using it as an operand. To make a numerical zero, call `ppf_set_zero(&x, 1)` on an initialized object; the positive sign argument selects positive zero.

If allocating significand storage fails, `ppf_init2_ctx` returns failure and leaves an all-zero structure. Starting from `{0}` also handles early argument validation failures safely. Check the return before proceeding. A nonzero initialization status is not a rounding direction.

The `goto cleanup` statements provide one release path. They do not jump back into a partially initialized calculation. For a C program that owns several resources, this structure keeps an early failure from bypassing cleanup. Clear each object you successfully initialized, and avoid reinitializing an object whose previous storage is still live.

3.2 Read an operation declaration

Consider the multiplication interface:

```
int ppf_mul(ppf_t *rop, const ppf_t *a, const ppf_t *b,
            ppf_rnd_t rnd, ppf_context_t *ctx);
```

`rop` is the result operand, or destination. The `const` qualification on `a` and `b` marks them as input pointers; any permitted overlap with the destination follows the aliasing rules discussed below. `rnd` selects rounding, and `ctx` supplies execution policy and flags. The returned integer is a ternary result, whose interpretation is explained in the next chapter.

In a call such as `ppf_mul(&product, &a, &b, PPF_RNDN, &ctx)`, each `&` supplies an address. The function writes through the destination pointer. There is no C expression `product = a * b` for these objects: C's built-in multiplication operator does not know the library's ownership and rounding rules.

Library functions differ in return conventions. Read the declaration and reference entry instead of treating every `int` as the same kind of answer. Initialization returns a status; multiplication returns a ternary; comparison returns an ordering result; a predicate answers a yes/no question. They can all use the C type `int` while meaning different things.

3.3 Precision belongs to the object

`ppf_init2_ctx(&x, 128, &ctx)` explicitly creates a 128-bit destination. Use `ppf_get_prec(&x)` to inspect that precision. The minimum supported precision is two bits; the maximum practical request also depends on storage and implementation limits. Choose a precision for the numerical work, rather than allocating the largest value the type could express.

Different precision-changing functions have deliberately different purposes:

Function	Effect
`ppf_reserve`	Ensures storage capacity without changing the declared precision or numerical value.
`ppf_set_prec`	Changes the precision and discards the previous value, leaving NaN on success.
`ppf_prec_round`	Changes precision while rounding the current value under the supplied mode and context.
`ppf_context_set_default_precision`	Changes a context's default policy; existing objects retain their precisions.

Use `ppf_prec_round` when the existing value must be retained. Narrowing can lose information. Widening can preserve the stored value exactly when it remains within the applicable range, but cannot reconstruct digits lost before the widening. Always check the return value; a request for more precision can require allocation.

Initialization offers several precision and context choices. `ppf_init` uses the calling thread's default context and its default precision. `ppf_init2` uses an explicit precision with that thread's default context. `ppf_init2_ctx` uses the precision you pass and the explicit context you pass. Setting an explicit context's default precision does not redirect a later `ppf_init` to that context. For introductory code, a named precision and `ppf_init2_ctx` make the intended pairing unambiguous.

A higher-precision source does not resize a lower-precision destination during `ppf_set` or arithmetic. That behavior lets you design a calculation with high-precision intermediate work and a deliberately smaller final output. It also means that a mistakenly small temporary can lose information before the final output is computed.

3.4 Copy values without copying ownership

C structure assignment copies pointers. Consequently, `b = a` and `memcpy(&b, &a, sizeof b)` are not ways to duplicate an owned Polyphase Float number. They can make two structures refer to the same storage and later free that storage twice.

Instead, initialize `b` at its intended precision and call:

```
int ternary = ppf_set(&b, &a, PPF_RNDN, &ctx);
```

This copies the numerical value into `b` with the destination's rounding and precision. The allocations remain independently owned. Check `PPF_TERNARY_ERROR` before interpreting the sign of `ternary`.

`ppf_swap(&a, &b)` exchanges the complete object states, including precision and ownership. It is useful when an algorithm wants to make a completed temporary become the current

result. After a swap, each variable owns the state it received and should still be cleared exactly once. Do not assume that either variable retained its old precision.

Pass object pointers to your own helper functions too. A helper can receive `ppf_t *out` and `const ppf_t *input`; taking a `ppf_t` by value would copy its owning fields. Keep helpers' ownership contracts simple: either they operate on an already initialized destination, or they explicitly initialize a new one and document who clears it.

Aliasing means that two arguments refer to the same object or overlapping storage. Some operations support using an input as the destination; others have stricter rules, especially paired outputs and batch interfaces. The introductory programs use separate objects so their data flow is visible. Check the per-function aliasing contract before replacing those objects with an in-place optimization.

3.5 Give each context a clear owner

A context holds default precision, default rounding, the minimum and maximum exponent, sticky flags, allocation callbacks, and reusable temporary storage. Initialize it with `ppf_context_init`, then adjust policy through the public setters if necessary. `ppf_context_set_rounding` accepts a concrete rounding mode; operations can subsequently request that policy using `PPF_RND_DEFAULT`.

A context is mutable even when the operation's mathematical inputs are read-only: allocation and flags can change it. In concurrent code, give each worker its own context or protect shared use with appropriate synchronization. A shared read-only operand is permitted while no thread mutates, clears, resizes, swaps, or reinitializes it. A destination requires exclusive access for the duration of an operation. OpenMP support inside the library does not remove these application ownership rules.

Use `ppf_context_clone` to create a separate context with copied policy and flags. It initializes independent temporary-storage state rather than sharing the source context's active scratch arena. The clone retains the allocator callbacks and opaque state, so a custom allocator's lifetime and thread-safety requirements still apply. The destination of cloning must be available for initialization; do not clone over a live context without first releasing it.

For ordinary code, clear numbers before clearing the context that created them. Each number actually retains the allocator callbacks needed to free its own allocation, so the context structure itself is not a hidden owner of every number. With custom allocators, however, the opaque allocator state must stay valid until all dependent objects, buffers, and contexts have been released. The straightforward object-first cleanup order makes that obligation easy to meet.

3.6 Use the matching release function

Resource	How it is released
Initialized, normally owned `ppf_t`	`ppf_clear`
Explicit `ppf_context_t`	`ppf_context_clear`
String allocated by `ppf_get_str(NULL, ...)`	`ppf_free_str`
Successful `ppf_asprintf` or `ppf_vasprintf` output	C `free`
Caller-supplied output buffer	Its existing application owner releases it

Those string rules are different on purpose. Do not infer the deallocator merely from the return type `char *`. The API reference states ownership for each allocating function. The introductory examples use stream output so there is no returned string allocation to manage.

`ppf_thread_cleanup` releases the calling thread's default context and internal thread-local workspaces. It may be called repeatedly. It does not clear your user-owned numbers, explicit contexts, plans, or random states. Call it after releasing those resources at the end of a worker or a standalone program when deterministic thread cleanup is desired. Normal thread-exit cleanup is also available where the library successfully registers its POSIX cleanup key; explicit cleanup keeps the example's lifetime visible.

3.7 Keep explicit and default diagnostics together

If a calculation receives `&ctx`, inspect it with `ppf_context_flags(&ctx)` and clear it with `ppf_context_flags_clear(&ctx, mask)`. The shorter `ppf_flags_*` functions refer to the calling thread's default context. They do not automatically inspect whichever explicit context your previous operation used.

Many numerical APIs resolve a null context to the thread-local default context, but lifecycle calls have their own preconditions. Do not generalize the null-context convention to every function. The explicit form used here avoids that ambiguity and works naturally when a program manages multiple numerical policies.

Exercise. An eight-bit object contains an approximation to `0.1`. Which operation lets you increase its precision while retaining its stored value: `ppf_set_prec` or `ppf_prec_round`?

Answer. `ppf_prec_round`, with appropriate range and rounding policy. The widened object still contains the earlier approximation. `ppf_set_prec` discards the value and leaves NaN on success. Parsing the original decimal string again into a wider object is a different operation and can recover more information from that original input.

Exercise. Can clearing a context replace clearing all numbers created through it?

Answer. No. Numbers own their significand allocations. Clear each owned number and then clear the context's own resources. The same distinction applies to thread cleanup.

Chapter 4

Rounding, flags, and exceptional results

A floating-point calculation produces a stored value and information about how that value was obtained. Polyphase Float exposes that information through rounding modes, ternary return values, result categories, and context flags. These mechanisms answer different questions. Using them together makes it possible to distinguish an ordinary rounded answer from an invalid request, an exceptional mathematical result, or a practical implementation limit.

For a first program, use nearest rounding, check the failure sentinel, and inspect any result categories your application cannot accept. Add flag handling according to the calculation's needs. Inexact rounding is expected for many useful calculations; treating it as a program failure would reject values such as one third at every finite binary precision.

4.1 Choose a rounding direction

Imagine the exact real result between two adjacent representable values at the destination precision. A rounding mode decides which value is stored.

Mode	Meaning
`PPF_RNDN`	Nearest; an exact halfway case selects the even significand.
`PPF_RNDZ`	Toward zero.
`PPF_RNDU`	Toward positive infinity.
`PPF_RNDD`	Toward negative infinity.
`PPF_RNDA`	Away from zero.
`PPF_RNDF`	Faithful rounding: an allowed adjacent representable result.
`PPF_RND_DEFAULT`	For context-based numerical operations, use the resolved context's default rounding mode.

The words *up* and *down* refer to numerical order, not magnitude. For a negative result, rounding toward positive infinity makes the answer less negative; rounding toward negative infinity makes it more negative. Rounding toward zero agrees with downward rounding for positive results and upward rounding for negative results.

For nearest-even, "even" refers to the least significant retained significand bit, not the last decimal digit in a printed string. Halfway decisions therefore depend on the binary representation at the destination precision.

Faithful rounding does not request a unique nearest result in every case. Do not use it when your algorithm requires a specifically directed bound or a particular nearest-even bit pattern. The function's own contract describes its supported settlement and reporting behavior. For introductory calculations, `PPF_RNDN`, `PPF_RNDD`, and `PPF_RNDU` make the intent particularly easy to inspect.

Passing a concrete mode on a call makes that choice independent of later changes to the context default. Passing `PPF_RND_DEFAULT` is useful when a whole operation sequence deliberately follows a policy selected by the caller.

Check the individual contract before passing `PPF_RND_DEFAULT` to other interfaces. Rounding-policy setters and `ppf_can_round` require concrete modes. The `ppf_fits_*_p` predicates use a fresh private context, so their default is nearest-even rather than the calling thread's policy. The naming helpers inspect the enumeration: `ppf_rounding_name` returns the name `"PPF_RND_DEFAULT"`, while `ppf_print_rnd_mode` returns `NULL` for that value.

4.2 Check the failure sentinel before the sign

Many assignment and arithmetic functions return a **ternary** integer. For an ordinary finite rounding result:

Return relation	Interpretation
Negative	The stored result is below the exact result.
Zero	The stored result equals the exact result.
Positive	The stored result is above the exact result.

There is also a separate sentinel, `PPF_TERNARY_ERROR`, for an unrecovered failure. That sentinel is a negative integer. Testing `t < 0` before excluding it would misclassify failure as a valid result rounded downward.

Use this order:

```
int t = ppf_div(&result, &numerator, &denominator,
                PPF_RNDN, &ctx);
if (t == PPF_TERNARY_ERROR) {
    /* Handle the failure; do not interpret t as a direction. */
} else if (t < 0) {
    /* The ordinary finite result is below its exact target. */
} else if (t > 0) {
    /* The ordinary finite result is above its exact target. */
} else {
    /* No inexact direction is reported; also check result category. */
}
```

Use the sign of a non-error ternary, not an assumption that the library will always return exactly `-1` or `+1`. Also do not replace the error check with `if (t != 0)`: that would classify successful inexact results as failures.

The exact target is the mathematical operation on the values actually stored in the operands. If a decimal input was rounded earlier, a later zero ternary says nothing about whether that input was the intended decimal real number. This distinction is especially important when an operation subtracts nearly equal approximations: the subtraction can be exact while the result has poor accuracy relative to the original problem.

Special values require their documented category rules rather than an ordinary finite ordering interpretation. For example, division by zero can produce infinity, and an invalid arithmetic combination can produce NaN. A zero return alone is not a general certificate that the result is a finite exact real value.

4.3 Different functions return different kinds of answers

The C type of a return value is not enough to interpret it correctly.

Example	Return convention
`ppf_init2_ctx`, `ppf_context_init`	Status: zero on success, failure otherwise.
`ppf_set`, `ppf_set_ui`, `ppf_mul`, `ppf_div`	Ternary, with `PPF_TERNARY_ERROR` checked first.
`ppf_set_str`	Status: zero for an accepted complete input, `-1` on failure.
`ppf_strtofr`	Ternary plus an end pointer used to inspect parsing progress.
`ppf_out_str`	Output count; zero indicates failure.
`ppf_cmp`	Comparison result, subject to its special-value rules.
`ppf_nan_p`, `ppf_zero_p`	Predicates: nonzero means the condition holds.

In particular, a successful `ppf_set_str` call can involve rounding even though its return is zero. That API reports acceptance of the string, not a ternary direction. If you need parsing progress or rounding direction, use the appropriate parsing interface and validate its end pointer as described in the input/output chapter.

Do not build a helper that treats all functions with return type `int` identically. A useful application helper can be specific: one helper for status-returning setup, another for ternary arithmetic, and a separate policy for exceptional numerical categories.

4.4 Observe sticky flags

A context accumulates diagnostic flags. “Sticky” means that a later successful or exact operation does not automatically erase a flag set by an earlier operation. This allows a program to examine whether any step in a sequence encountered a condition.

Flag	General meaning
`PPF_FLAG_INEXACT`	A result required inexact rounding.
`PPF_FLAG_UNDERFLOW`	An operation encountered its underflow condition.
`PPF_FLAG_OVERFLOW`	An operation encountered its overflow condition.
`PPF_FLAG_DIVBY0`	A division-by-zero condition was reported.
`PPF_FLAG_NAN`	A NaN-related or invalid arithmetic condition was reported.
`PPF_FLAG_ERANGE`	An invalid, out-of-range, or otherwise rejected request was reported.
`PPF_FLAG_ALLOC`	A configured project allocator rejected an allocation request.

These descriptions introduce the flags; individual operation contracts specify their exact triggering rules. For example, exponent-range operations and subnormalization have their own defined underflow semantics. Do not infer every function's behavior from a hardware floating-point exception model.

To isolate one operation, clear the flags you intend to monitor before it, then save the resulting flags immediately afterward:

```
ppf_context_flags_clear(&ctx, PPF_FLAGS_ALL);
int t = ppf_div(&result, &one, &three, PPF_RNDN, &ctx);
ppf_flags_t observed = ppf_context_flags(&ctx);
```

Check `t` before using the numerical result. The saved `observed` value describes the context state at that point. Subsequent conversions or other numerical work may alter the live flags, so saving them before formatting diagnostics avoids accidentally attributing later events to the division.

If an application wants flags for an entire algorithm, clear them once at the algorithm boundary and inspect them at the end. Clearing after every call would lose that accumulated record. If you need both an outer accumulated record and a local observation, use the save/restore interfaces with a deliberate mask and combine the observations according to your application's policy.

An allocation flag does not always mean the calculation ultimately failed. Some operations can recover through an exact fallback after an optional allocation fails, leaving the flag set as a diagnostic. Check the operation's return and result as well as the flag. The allocation contract covers project-controlled paths; GMP, libc, and operating-system allocation behavior are separate domains.

4.5 Recognize special values

Polyphase Float distinguishes regular finite nonzero values, signed zeros, signed infinities, and NaNs. Use predicates such as `ppf_regular_p`, `ppf_zero_p`, `ppf_inf_p`, and `ppf_nan_p` instead of inspecting internal fields or relying on text formatting.

A numerical zero is finite, but it is not a *regular* value in this category terminology. Thus `ppf_regular_p` alone is not a test accepting every finite number. If your application accepts ordinary finite values including zero, accept regular values or zeros and reject the other categories.

Positive and negative zero compare as equal numerical zeros, but their signs can matter for sign-sensitive operations. `ppf_signbit` lets you inspect the retained sign. NaN is not an ordinary ordered number: comparisons involving it need the documented predicate or ordering behavior. Classify a value before using it in a policy that assumes an ordinary finite order.

The introductory flag program demonstrates two different exceptional operations. Dividing positive one by positive zero produces positive infinity and sets `DIVBY0`. Dividing zero by zero produces NaN and sets `NAN`. Neither case should be silently treated as an ordinary finite answer, even if the API follows its special-value rule without returning the failure sentinel.

An application decides whether infinity or NaN is acceptable. A scientific routine may propagate them with diagnostics; a file-format validator may reject them at input; an optimization algorithm may treat them as a failed candidate. Make that policy explicit at the boundary where the value enters the application.

4.6 Understand exponent range separately from precision

Precision determines significand resolution. The context's `emin` and `emax` limit the exponent range used by operations. Increasing precision does not necessarily fix an overflow caused by a restrictive exponent range, and widening the exponent range does not restore precision lost through rounding.

Regular values are normalized; the object model does not store an implicit ladder of subnormal values beneath the minimum normal value. Ordinary range settlement below that boundary chooses signed zero or the minimum normal value according to the applicable rounding rule. Overflow chooses infinity or the largest finite value according to sign and rounding. Advanced range and subnormalization APIs expose additional operations with specific contracts.

For most first programs, retain the default range. If your application changes it, do so deliberately and check the status returned by `ppf_context_set_range`. Existing objects are not automatically recomputed just because policy changes. A subsequent assignment or operation can settle a result under the new range.

4.7 Separate recoverable diagnostics from failed computation

A successful rounded result may be entirely suitable for an application. An unrecovered failure needs an explicit path: return an application error, select a documented alternative, or retry with a justified change of parameters. Do not print the destination as though it were newly valid merely because it still contains something.

Destination preservation depends on the kind of failure and the function's contract. Covered project-allocation failures preserve an initialized destination. Exhaustion of a mathematical function's numerical refinement budget can instead publish NaN with `ERANGE` and `PPF_TERNARY_ERROR`. Invalid input and resource-limit rejection also have operation-specific rules. Consult the relevant reference entry before relying on an old value after a failed call.

Increasing precision is not a universal remedy for rejection: a request can exceed a domain bound, an iteration budget, or a resource policy. Capture the arguments, precision, rounding mode, relevant flags, and result category when diagnosing such a case. That record is more informative than a decimal answer alone.

Exercise. An operation returns a negative integer. What must you test before describing the answer as rounded downward?

Answer. Test whether it equals `PPF_TERNARY_ERROR`. If it does, it is failure. If it does not, interpret the ternary sign under the operation's ordinary-result contract and inspect any special category relevant to the application.

Exercise. An inexact division is followed by an exact assignment, and `INEXACT` remains set. Did the assignment fail?

Answer. No such conclusion follows. The flag is sticky. The division may have set it, and the assignment does not clear it. To diagnose the assignment alone, clear the monitored flags before the assignment or use an explicit save/restore policy.

Chapter 5

First complete programs

The four programs in this chapter turn the preceding concepts into code you can compile and inspect. Each is a complete C11 program with deterministic output. Each checks its own expected numerical relationships and returns a nonzero process status if setup, calculation, verification, or output fails. Their shared structure is intentional: learning one cleanup and error-checking pattern gives you a foundation for a larger application.

The complete source files are in `examples/manual`. The build chapter gives a direct compiler command for the serial core. With examples and tests enabled, CMake also provides a target and a test for each program:

Source file	Build target	Main lesson
`intro_first.c`	`example_manual_intro_first`	A complete checked calculation
`intro_rounding.c`	`example_manual_intro_rounding`	Directed and nearest rounding
`intro_flags.c`	`example_manual_intro_flags`	Sticky flags and special values
`intro_precision.c`	`example_manual_intro_precision`	Widening versus recomputing

For example, from the source directory after configuring `build/release`:

```
cmake --build build/release --target example_manual_intro_first -j2
./build/release/example_manual_intro_first
ctest --test-dir build/release -L manual --output-on-failure
```

The `manual` label selects the manual examples registered in that build. The example results (`docs/manual/EXAMPLE_RESULTS.md`) describe their verification scope. These small demonstrations check their stated behavior; they are not a substitute for the full library test suite or validation of an application's numerical algorithm.

5.1 Program 1: multiply two exact integers

Complete source: `intro_first.c`.

The program computes two times three in a 128-bit destination. Small integers make the arithmetic exact, so any discrepancy points to setup, usage, or execution rather than a subtle decimal approximation.

```c
#include "polyphase_float.h"
#include <stdio.h>

int main(void) {
    ppf_context_t ctx;
    ppf_t a = {0}, b = {0}, product = {0};
    int status = 1;
    int ternary;

    if (ppf_context_init(&ctx) != 0) {
        fputs("Cannot initialize the context.\n", stderr);
        return 1;
    }
    if (ppf_init2_ctx(&a, 128, &ctx) != 0 ||
        ppf_init2_ctx(&b, 128, &ctx) != 0 ||
        ppf_init2_ctx(&product, 128, &ctx) != 0)
        goto cleanup;
    if (ppf_set_ui(&a, 2, PPF_RNDN, &ctx) == PPF_TERNARY_ERROR ||
        ppf_set_ui(&b, 3, PPF_RNDN, &ctx) == PPF_TERNARY_ERROR)
        goto cleanup;

    ternary = ppf_mul(&product, &a, &b, PPF_RNDN, &ctx);
    if (ternary == PPF_TERNARY_ERROR) goto cleanup;
    if (ternary != 0 || ppf_cmp_ui(&product, 6, &ctx) != 0)
        goto cleanup;
    if (fputs("2 * 3 = ", stdout) == EOF ||
        ppf_out_str(stdout, 10, 6, &product, PPF_RNDN, &ctx) == 0 ||
        fputc('\n', stdout) == EOF || fflush(stdout) == EOF)
        goto cleanup;
    status = 0;

cleanup:
    if (status != 0) fputs("The calculation or output failed.\n", stderr);
    ppf_clear(&product);
    ppf_clear(&b);
    ppf_clear(&a);
    ppf_context_clear(&ctx);
    ppf_thread_cleanup();
    return status;
}
```

Expected output:

```
2 * 3 = 6.00000e0
```

The `ui` in `ppf_set_ui` denotes an unsigned C integer input. Both input assignments are exact at this precision. Multiplication returns zero because six is exactly representable. The program then checks the numerical answer using `ppf_cmp_ui`, rather than trusting that a successful call necessarily matches its intended example.

`ppf_out_str(stdout, 10, 6, ...)` prints in base ten with six significant digits. The `e0` exponent means multiplication by ten to the zeroth power. The output precision of six digits is a display choice; the destination still has 128 binary bits of precision.

The example checks both library output and the surrounding C stream operations. A calculation can succeed while writing its answer fails. `fflush` makes a pending buffered output failure visible before the program reports success.

For a C beginner, the `||` operators in setup are short-circuit tests: once one condition reports failure, later initializations in that expression are skipped and execution goes to cleanup. Because all object structures started at `{0}`, clearing the remaining structures is safe. `status` stays one until every required step succeeds.

5.2 Program 2: see where one third lands

Complete source: `intro_rounding.c`.

This program stores one and three exactly, then divides them into three eight-bit destinations. The central calls are:

```
down = ppf_div(&lower, &one, &three, PPF_RNDD, &ctx);
near = ppf_div(&nearest, &one, &three, PPF_RNDN, &ctx);
up = ppf_div(&upper, &one, &three, PPF_RNDU, &ctx);
```

The complete program checks every failure sentinel before inspecting the signs. It also verifies that the lower result is smaller than the upper result and that the nearest result equals the upper result in this particular example.

Expected output:

```
down    = 3.32031250e-1 (below)
nearest = 3.33984375e-1 (above)
up      = 3.33984375e-1 (above)
```

At eight-bit precision, the adjacent representable values around one third are

$$L = \frac{170}{512} = 0.33203125, \qquad U = \frac{171}{512} = 0.333984375.$$

Their distances from one third are $1/768$ and $1/1536$, respectively. The upper neighbor is closer, so nearest rounding chooses it. This is not a halfway case. The directed results bracket the exact quotient of the stored inputs, which are exactly one and three.

The program prints nine decimal significant digits so these particular binary values can be read clearly. It does not require nine accurate decimal digits in the quotient: eight binary

bits have much coarser resolution. Here the extra printed digits expose the exact small dyadic values being stored.

Directed rounding is a tool for constructing bounds, but a complete interval algorithm must account for every operation and the uncertainty in its inputs. Running the last step once downward and once upward does not automatically enclose a mathematical expression whose earlier steps were rounded without bounds.

Try it. Increase all five object precisions from eight to sixteen. The printed endpoints should get closer, but the existing assertions about which endpoint is nearest are specific to the original example and must be reconsidered. When exploring, compare numerical relationships and ternary directions; do not simply change expected output until a test passes.

5.3 Program 3: distinguish flags from return values

Complete source: `intro_flags.c`.

This program performs a sequence whose diagnostic state is easy to predict. It begins with eight-bit one third, performs an exact assignment, clears flags, and then evaluates two exceptional divisions.

Expected output:

```
1 / 3: INEXACT is set
exact assignment: INEXACT is still set
after clearing: no flags are set
1 / 0: positive infinity; DIVBY0 is set
0 / 0: NaN; NAN is set
```

The exact assignment is two. Its zero ternary does not erase the earlier `INEXACT` flag. The program explicitly checks both facts: assignment was exact, and the previous flag remains. It then calls `ppf_context_flags_clear(&ctx, PPF_FLAGS_ALL)` to start a fresh observation period.

For the exceptional divisions, the code checks the failure sentinel, saves the flags, and inspects the result category. It requires positive infinity after one divided by positive zero, and NaN after zero divided by zero. The output text describes those verified conditions; it does not claim that every other possible flag bit is absent.

This is a useful pattern for an application boundary. Decide which categories and flags are acceptable for the task, then test them explicitly. A calculation intended to produce an ordinary finite measurement may reject both infinities and NaNs. A diagnostic tool can display them and their causes. Both policies can use the same library operations.

Try it. Remove the flag-clear call immediately before the final zero-over-zero division. The previous `DIVBY0` condition will remain in the context together with the newly reported NaN condition. That demonstrates why a flag set is a history since the last clear, not an

automatic description of only the most recent call. Restore the clear afterward to retain the original example's intended observation periods.

5.4 Program 4: widening does not restore lost input

Complete source: `intro_precision.c`.

This program compares three objects: a decimal string parsed at eight bits, an exact copy of that stored approximation into 128 bits, and the original string parsed directly at 128 bits.

```
if (ppf_set_str(&coarse, "0.1", 10, PPF_RNDN, &ctx) != 0 ||
    ppf_set_str(&fresh, "0.1", 10, PPF_RNDN, &ctx) != 0)
    goto cleanup;
ternary = ppf_set(&widened, &coarse, PPF_RNDN, &ctx);
```

Notice the two return conventions. `ppf_set_str` returns a status, so nonzero is failure. `ppf_set` returns a ternary, so the complete source checks `PPF_TERNARY_ERROR` first and then verifies that this widening copy is exact.

Expected output:

```
parsed at 8 bits     = 1.00097656250e-1
copied to 128 bits   = 1.00097656250e-1
parsed at 128 bits   = 1.00000000000e-1
changing the default leaves existing precisions unchanged
```

The first two objects are numerically equal. The wider copy has room for more bits, but the copy operation receives only the value already stored in `coarse`. The direct 128-bit parse receives the original decimal input and therefore produces a different, much closer approximation to one tenth. Its displayed twelve digits do not imply that binary one tenth has become exact.

The program also sets the explicit context's default precision to 256 and verifies that existing objects retain their eight-bit and 128-bit precisions. Policy changes do not silently rewrite previously created numbers.

For decimal data, parsing a decimal string directly avoids an intermediate conversion through a C `double`. In contrast, `ppf_set_d(&x, 0.1, ...)` receives the binary value of the C literal after the compiler has represented it as a `double`. A wider destination can preserve that input faithfully but cannot recover the decimal value the literal was intended to suggest. Neither route invents information absent from the input; choose the route that represents your actual data source.

5.5 Adapt the pattern to an application

Before replacing these small calculations with a larger formula, write down the precision of each input, temporary, and final destination. Decide how input is obtained, which rounding modes are required, and which exceptional categories the caller accepts. Place flag-clearing boundaries where they correspond to meaningful units of work.

Keep the numerical operation separate from presentation. A useful calculation helper accepts initialized objects and a context, performs the work, and reports a checked result. The caller decides how many digits to print and where to send them. That separation makes it easier to test numerical relationships without depending on a particular output format.

Finally, keep a meaningful check. Exact small cases, symmetries, independently computed bounds, and comparisons at higher working precision can all be useful, provided you understand what they establish. Repeating one algorithm at more bits is a diagnostic technique, not automatically a proof of the final digits. The later numerical chapters explain the stronger tools available for specific operation families.

Exercise. Why is the widened eight-bit approximation exactly copied even though it is still an inaccurate representation of decimal `0.1`?

Answer. The copy's exact input is the stored eight-bit binary value. That dyadic value fits into 128 bits without rounding. Its discrepancy from the original decimal real number arose during the earlier parse, and the copy has no access to that lost information.

Chapter 6

Arithmetic, rounding, and exponent ranges

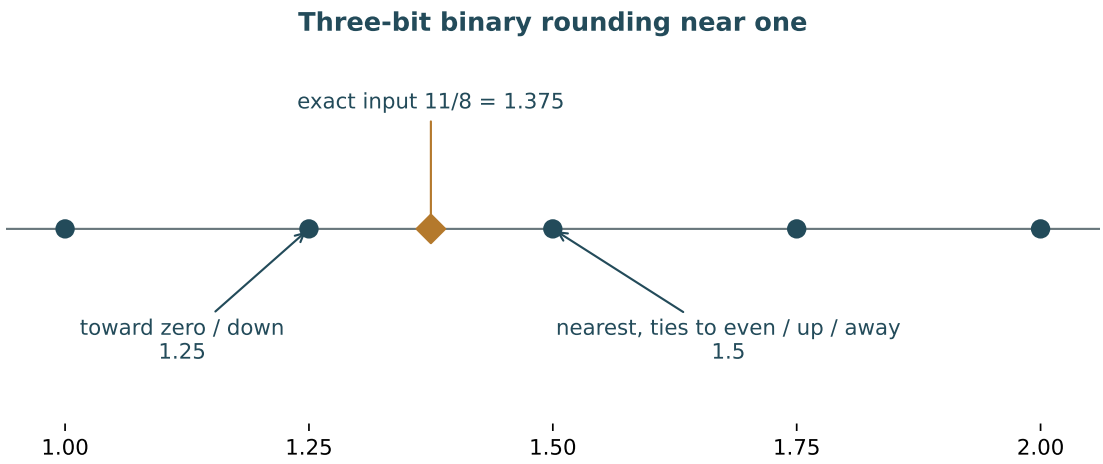


Figure 6.1: At three-bit precision, the positive midpoint 11/8 rounds to 1.5 under nearest, ties to even. Directed modes select the appropriate neighbor.

A reliable calculation starts with three separate choices: what numbers the inputs represent, how many bits each destination will retain, and which rounding rule each operation will use. The context also determines the permitted exponent range. These choices answer different questions. More precision places representable values closer together; a wider exponent range allows larger and smaller magnitudes. Neither change can recover information already discarded when an input was created.

This chapter develops the numerical meaning of an arithmetic call and shows how to read its result. The complete example is numeric_ranges.c. It uses deliberately small precision so that the rounding decisions can be checked by hand.

6.1 The destination determines the precision

For a regular value with precision p, Polyphase Float stores an integer significand M, a sign s, and a normalized exponent e. Its value is

$$x = sM2^{e-p}, \qquad 2^{p-1} \le M < 2^{p}.$$

Thus a regular magnitude lies in $[2^{e-1}, 2^{e})$. Within this interval the spacing is 2^{e-p}. Precision counts significant binary bits, including the leading bit. It does not count decimal places after a decimal point.

In a call such as

```
int t = ppf_div(&quotient, &numerator, &denominator,
                PPF_RNDN, &ctx);
```

`quotient` supplies the result precision. The inputs may have different precisions. Their stored values are the operands of the division; the library does not reinterpret them as unknown higher-precision originals. Increasing the context's default precision affects subsequent initialization using that default. It does not change the precision of objects that already exist.

For planning output, $p \log_{10} 2$ is a useful approximate count of significant decimal digits. For example, 128 bits correspond to about 38.5 decimal digits. This estimate is not an error bound for a long computation. Cancellation, input uncertainty, conditioning, and intermediate rounding still matter.

6.2 Choose the rounding rule deliberately

The five deterministic rounding modes have straightforward geometric meanings. Consider rounding the positive exact value $1/3$ to three binary bits. The neighboring values are $5/16 = 0.3125$ and $3/8 = 0.375$.

Mode	Rule	Three-bit result for $1/3$
`PPF_RNDN`	Nearest; a midpoint selects the even significand	0.3125
`PPF_RNDZ`	Toward zero	0.3125
`PPF_RNDU`	Toward positive infinity	0.375
`PPF_RNDD`	Toward negative infinity	0.3125
`PPF_RNDA`	Away from zero	0.375

For negative values, upward still means toward positive infinity. It therefore moves a negative inexact result toward zero. Downward moves it away from zero. Thinking in terms of the number line avoids a common sign error when constructing bounds.

`PPF_RND_DEFAULT` requests the context's configured mode. Explicit modes are usually easier to review in numerical algorithms, especially when adjacent calls intentionally form lower and upper bounds.

`PPF_RNDF` requests faithful rounding. Code that needs a specified direction, an exactness decision from the return value, or reproducible choice between neighboring values should use a deterministic mode. Do not build a certificate from an assumed faithful ternary convention. The examples in this manual use explicit deterministic modes whenever they inspect the ternary result.

6.3 Read the ternary value before interpreting flags

Most arithmetic functions return a ternary direction. With a deterministic rounding mode and a successful ordinary numerical result, zero means exact, positive means the stored result is greater than the exact mathematical result of that call, and negative means it is smaller. The magnitude of the integer is not an error estimate. A return value of 1 does not mean one bit or one unit in the last place was lost.

Check the failure sentinel first:

```
int t = ppf_mul(&product, &a, &b, PPF_RNDN, &ctx);
if (t == PPF_TERNARY_ERROR) {
    /* Diagnose the failure; product is not a usable new answer. */
} else if (t == 0) {
    /* For an ordinary finite result, the multiplication is exact. */
} else {
    /* A valid rounded result; the sign of t gives its direction. */
}
```

Testing `if (t)` as an error condition rejects ordinary inexact arithmetic. Conversely, testing only `t < 0` confuses an error sentinel with a legitimate downward rounding direction. Special values also require their documented classification rules: a numerical NaN can be the specified result of a domain error rather than a failure to execute the API.

Not every integer-returning function uses this convention. Reuse an error-handling pattern only for functions with the same return contract.

Interface	Return interpretation
Initialization functions and `ppf_set_str`	Status code; successful parsing is not a claim of exact numerical representation
`ppf_get_constant_interval`	Status: 0 for success, -1 for failure
`ppf_div_with_plan_many`	Batch status: 0 for success, -1 for failure; individual rounding directions are written to the optional `ternaries` array
`ppf_set_dyadic_interval`	`DECIDED`, `UNDECIDED`, or `ERROR`; its ternary is a separate output
`ppf_modf`, `ppf_sin_cos`, `ppf_sinh_cosh`	Packed two-output code on success, not one scalar ternary

For a successful paired call in a deterministic mode, each output contributes a field: 0 means exact, 1 means rounded above its exact component, and 2 means rounded below it. The combined code is the first output's field plus four times the second output's field. The output order is integer/fractional for `ppf_modf`, sine/cosine for `ppf_sin_cos`, and hyperbolic sine/cosine for `ppf_sinh_cosh`. Check `PPF_TERNARY_ERROR` before decoding. A positive combined code can therefore include a downward-rounded component; its sign is not a rounding direction. Special-value cases still require numerical classification.

6.4 Precision and range are independent

The context restricts regular exponents to `emin <= e <= emax`. The smallest positive regular value is

$$2^{\mathrm{emin}-1},$$

and, at precision p, the largest positive finite value is

$$(1 - 2^{-p})2^{\mathrm{emax}}.$$

These formulas use the normalized exponent convention above. Treating `emin` as the exponent of the smallest power of two would introduce a factor-of-two mistake.

The default exponent range is intentionally much wider than a native `double` range. The absolute bounds accepted by the context API are exposed through the public constants and range-query functions. Wide representational limits do not imply that every algorithm can economically process every combination of precision and exponent. Some operations impose checked workspace or iteration limits.

Changing a context's range does not rewrite every object previously created with that context. Set the intended range before the calculation where possible. If applying a restricted range to a result calculated in a wider context, use `ppf_check_range` with the correct incoming ternary from that calculation. Inventing an incoming zero discards information needed to avoid a second-rounding error at a boundary.

6.5 A range example you can verify by hand

The example initializes two eight-bit inputs and a two-bit destination, then sets the context range to [-9,9]. It computes

$$\frac{7}{15\,2^{-10}} = \frac{7168}{15} \approx 477.8667.$$

The largest finite destination value is $3 \cdot 2^7 = 384$. The next value in an unbounded exponent system is 512. Their midpoint is 448, so nearest rounding chooses 512 before the exponent restriction is applied. Since that exponent is outside the context range, the final result is positive infinity with overflow and inexact flags.

Toward-zero rounding selects 384 directly. That rounded value fits the exponent range, so this call raises inexact without overflow. This example illustrates why comparing the exact magnitude with the largest finite value is insufficient to predict every range flag: the rounding mode matters.

The program prints:

```
Nearest quotient: +infinity; overflow and inexact.
Toward-zero quotient: 384; inexact only.
Exact subnormal emulation: underflow without inexact.
```

The final line comes from a separate subnormal-emulation calculation described below. Every line is printed only after the program checks the value, return direction, and relevant flags. A failed check produces a nonzero process exit.

6.6 Flags describe events, and they accumulate

Context flags are sticky. An inexact flag can remain set after a later exact operation. Save or clear flags at a meaningful boundary, perform the operation or group of operations, and then inspect the resulting mask. In an explicit-context program, use the `ppf_context_flags_*` functions with that same context. The short forms operate on the calling thread's default context.

Flag	Practical interpretation
`PPF_FLAG_INEXACT`	A rounded operation recorded loss of exact representability
`PPF_FLAG_UNDERFLOW`	The operation encountered its underflow condition; subnormal emulation has a specific rule
`PPF_FLAG_OVERFLOW`	The rounded exponent exceeded the permitted range
`PPF_FLAG_DIVBY0`	A documented pole or division-by-zero case occurred
`PPF_FLAG_NAN`	An invalid numerical input or domain case produced NaN
`PPF_FLAG_ERANGE`	A request could not be completed under the applicable representation, domain, or certification limits
`PPF_FLAG_ALLOC`	A request to the configured Polyphase allocator failed

An inexact result is normal for division by three or evaluation of an irrational constant. It is usually not an application error. Allocation failure, a domain error, and an exhausted certification path require different responses even if more than one flag is present. Record the requested operation and its input policy alongside the flags; the mask alone is not a complete diagnostic message.

6.7 Emulating subnormal arithmetic

A `ppf_t` has regular values and signed zero, but no separate stored subnormal category. `ppf_subnormalize` emulates a fixed-point region by reducing effective precision according to the current exponent and `emin`. In that region the spacing becomes the fixed quantum $2^{\text{emin}-1}$.

The function needs the rounding mode and ternary from the operation that produced its input. It uses that information when a second rounding would otherwise choose the wrong side of a midpoint. Its returned ternary describes the final value relative to the original exact result, not merely relative to the intermediate input.

When the subnormal step actually reduces effective precision, it raises underflow even when the value remains exact. This follows the `mpfr_subnormalize`-style API rule. In the example, precision is two, `emin` is -30, and the input is exactly 2^{-31}. The value remains unchanged and the ternary is zero, yet underflow is set. Inexact remains clear. Do not apply the ordinary “underflow must imply inexact” shortcut to this API.

6.8 Select arithmetic that matches the expression

Use a fused operation when the mathematical specification calls for one rounding of a combined expression. `ppf_fma` evaluates $ab + c$ with a single final rounding; separate multiplication and addition round an intermediate product. Near cancellation, these are different numerical computations. The same design consideration applies to fused two-product operations, sums, dot products, and `ppf_hypot`.

For roots, select the operation that expresses the intended degree and sign convention. Odd roots of negative real values are meaningful; even roots of negative real values are not real. Negative degrees introduce reciprocals and their zero-domain conditions. The reference distinguishes `ppf_rootn_ui`, `ppf_rootn_si`, and the older `ppf_root` interface, including their signed-zero conventions.

Aliasing is a separate contract. Many scalar operations support replacing an input with their output, but this does not authorize arbitrary overlap in paired or batched operations. Use the documented alias pattern, especially when outputs may be written concurrently. Numerical correctness, supported memory overlap, and successful allocation are three requirements that must all hold for a reliable call.

Chapter 7

Constants and certified intervals

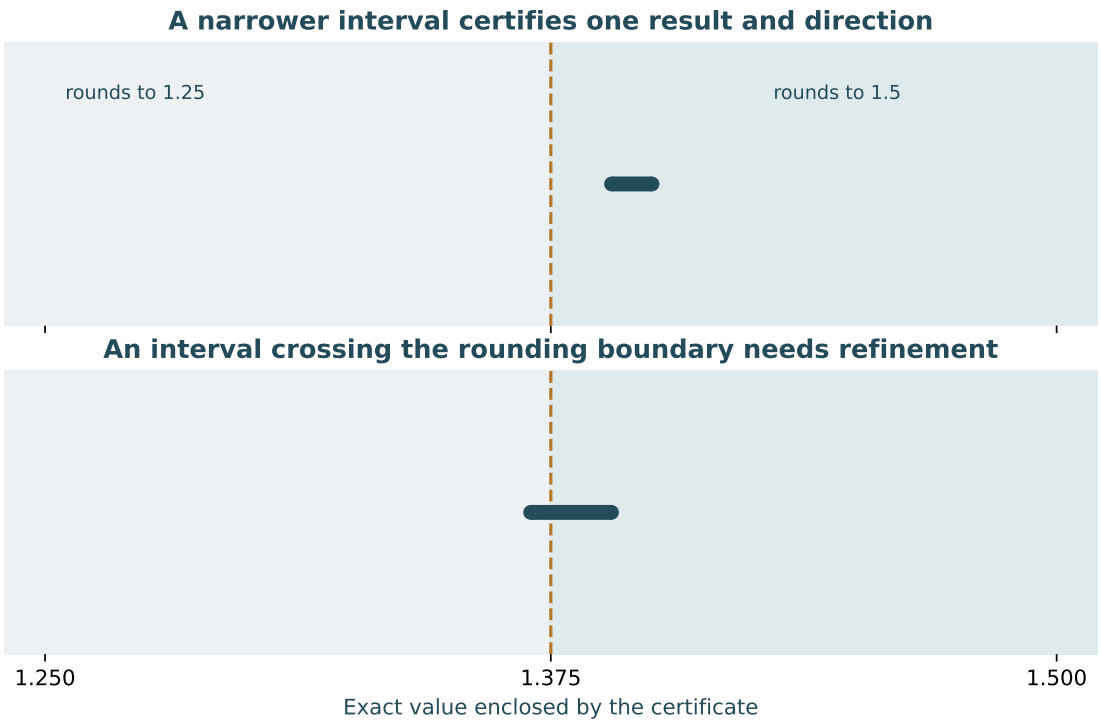


Figure 7.1: A useful interval must certify the rounded value, ternary direction, and range state. Crossing a rounding boundary leaves the result undecided.

A long string of digits is useful for display, but it is a weak foundation for a numerical constant. The last supplied digit may be rounded in the wrong direction for the caller's purpose, and the string may not contain enough information to answer a higher-precision request. Polyphase Float's constant interface is built around enclosures: lower and upper bounds known to contain the mathematical constant.

This chapter explains how to request a constant, how to export its enclosure, and what an interval decision actually certifies. The complete example is numeric_constants.c. The independent square-root refinement example in Numerical recipes shows how an application can supply its own justified enclosure.

7.1 Request the constant you mean

The public constant family includes four values:

Function	Mathematical value
`ppf_const_pi`	π, the circle constant
`ppf_const_log2`	$\log 2$, the natural logarithm of two
`ppf_const_euler`	Euler–Mascheroni constant γ
`ppf_const_catalan`	Catalan's constant G

Euler's constant is not the exponential base e, and `log2` in the constant name denotes the value $\log 2$. By contrast, the function `ppf_log2(x)` evaluates the base-two logarithm of its argument. Keeping these names distinct avoids a subtle substitution error in formulas.

Initialize the destination at the desired precision and call the selected function with an explicit rounding mode and context. In a deterministic mode, a successful ternary has the same direction meaning as for arithmetic. The destination precision, not the length of a later printed decimal string, determines the computed binary result.

For an upper bound on π, request `PPF_RNDU`; for a lower bound, request `PPF_RNDD`. A nearest-rounded approximation does not automatically provide either side. If an application subsequently multiplies a lower bound by a negative number, the inequality direction reverses. Bounds remain meaningful only when every subsequent operation respects signs and monotonicity.

7.2 What a dyadic enclosure contains

The interval-export interface returns GMP integers L, U and a binary exponent e, with

$$L2^e \le c \le U2^e.$$

Both endpoint values are exact because the integer and power-of-two representation is exact. The width is $(U - L)2^e$. It is an absolute width, not a relative-error estimate and not a count of correct decimal digits.

A negative exponent does not mean the constant itself is small. For example, a large integer multiplied by 2^{-S} can represent a number near three. The common scale simply permits both endpoints to be recorded without first rounding them to an ordinary floating-point object.

`ppf_get_constant_interval` accepts a constant identifier, initialized GMP output integers, an exponent output pointer, requested precision, and worker count. It returns status: 0 for success and -1 for failure, rather than a ternary. It has no explicit context argument: its work is governed through the thread-default path. This distinction matters in applications that otherwise use a custom explicit context. Do not assume an unrelated context's allocator or flags govern this export call.

The destination precision used later for rounding the enclosure is independent of the precision requested for interval construction. A practical pattern is to request a somewhat tighter interval, try the desired destination rounding, and refine only if needed. The number of extra bits is a work policy, not a proof by itself.

7.3 The decision requires more than matching endpoints

The public interval assignment call is

```
ppf_interval_status_t s = ppf_set_dyadic_interval(
    &result, lower, upper, exponent, rounding, &ternary, &ctx);
```

Its three outcomes have different meanings:

Status	Meaning	Normal caller response
`PPF_INTERVAL_DECIDED`	The enclosure determines the destination value, ternary direction, and compatible range flags	Use the result and recorded certificate
`PPF_INTERVAL_UNDECIDED`	The available enclosure does not establish the full requested decision	Obtain tighter justified bounds and retry
`PPF_INTERVAL_ERROR`	The request failed its validation or execution requirements	Diagnose the context and arguments

A decided result requires the two rounded endpoints to select the same value. It also requires a proven ternary relation and agreeing underflow and overflow decisions. Endpoint agreement alone is insufficient.

To understand the ternary requirement, suppose both endpoints round to a representable number r, but the interval straddles r. Some possible exact values are below r, others above it. The final floating-point value is determined, but its direction relative to the unknown exact value is not. A non-singleton interval containing r does not establish that the answer is exactly r, either. The correct outcome is undecided.

Likewise, an interval can cross a range-flag boundary while both endpoints happen to select the same stored value. A full decision must resolve that ambiguity. This is why an

application's stopping rule should use the returned interval status instead of comparing two printed numbers.

7.4 Undecided is useful information

An undecided result preserves the destination, the caller's ternary output variable, and the existing context flags. This makes refinement convenient: a previous valid result remains available while a new enclosure is being improved. It does not mean the previous value solves the new problem.

Keep a separate Boolean or application status indicating whether the current request has been certified. A destination containing an old value is still a valid object, but validity of storage is not validity of the new numerical answer. The refinement example starts the destination at one and uses a recognizable ternary marker so that this distinction is explicit.

An error has a different contract. Do not extend the undecided preservation guarantee to every error code or every arithmetic API. In particular, a numerical failure can leave a numerical error result, while a governed project-allocation failure has its own rollback rule. These cases are discussed further in Special functions.

7.5 Walking through the constant example

`numeric_constants.c` creates two 80-bit destinations. It evaluates π directly into the first, exports a 160-bit enclosure, and asks the interval interface to round that enclosure into the second. It checks both value equality and equality of ternary signs.

The expected output is:

```
Pi: direct evaluation and exported interval agree at 80 bits.
The interval proves the value and its rounding direction.
```

The program deliberately does not test a particular decimal spelling or a cache-hit count. Those observations are not the property being demonstrated. Its useful invariant is that the direct API and the exported enclosure establish the same 80-bit answer and direction.

This agreement is a consistency check within the constant subsystem. The two paths share the constant construction and finalization architecture, so the example is not an independent proof of the formula for π. The enclosure's mathematical justification comes from the builder's documented construction; release qualification supplies additional finite tests. Keep those kinds of evidence distinct when describing results to users.

7.6 How the constant builders justify their bounds

The constant implementations use exact integer or rational construction together with outward rounding. The π builder uses a Chudnovsky series and exact binary splitting. Its enclosure accounts for the omitted series tail and the rounding steps used in the scaled result. The natural-logarithm constant uses an exact series derived from

$$\log 2 = 2\,\operatorname{atanh}(1/3).$$

The finite sum is paired with a bound for the positive remainder, then converted outward to dyadic endpoints. Euler's and Catalan's constants use their documented series and asymptotic interval machinery. The common finalizer does not accept a result merely because the last two approximations agree.

These descriptions explain why an enclosure is available; they are not a substitute for the detailed derivations and implementation reviews. A client usually needs to rely on the public contract, retain the actual exported endpoints when reproducibility matters, and avoid presenting a finite validation suite as a universal theorem about every execution environment.

7.7 Caches retain reusable mathematical information

A constant cache stores a certified interval, allowing it to serve different destination precisions and rounding modes. It does not merely store a nearest-rounded number and reuse that number as if it were exact. A separate rounded-result cache may answer a repeated request more cheaply.

This makes precomputation useful when many calls will request the same constants. `ppf_precompute_constant` and the constant-specific precompute functions allow the application to arrange that work before a latency-sensitive phase. A higher-precision cached enclosure can also help later lower-precision requests. The cache-precision query reports cache state; it is not a certificate for an unrelated expression that uses the constant.

Cache construction and access are synchronized. Clearing a cache deliberately discards reusable work. Reserve explicit cache cleanup for a lifecycle boundary, controlled benchmark, or memory policy rather than clearing it after every call in a repeated numerical workload. The small standalone example clears caches at exit to demonstrate complete cleanup.

Worker-count entry points allow parallel construction where the implementation supports it. A worker request is not a promise of a speedup. Initialization, synchronization, cache hits, problem size, and the selected algorithm affect actual cost. When reporting timing, distinguish cold construction from a warm lookup and record precision, mode, worker policy, and cache state.

7.8 A certificate needs an enclosure argument

`ppf_set_dyadic_interval` validates and rounds the interval supplied by the caller. It cannot establish that an arbitrary interval encloses the mathematical quantity the application has in mind. If a program supplies bounds around a poor approximation without a justified error estimate, a decided result certifies only the decision implied by those supplied bounds.

For a complete application-level certificate, record the target quantity, exact input interpretation, derivation of the bounds, endpoint integers and exponent, destination precision, rounding mode, and interval decision. Include the output ternary and relevant flags if these are part of the claimed result. A rounded decimal rendering can accompany this record for readability, but it should not replace the exact endpoints.

This discipline is especially useful when composing several operations. Each successful scalar call concerns its own exact stored inputs. An enclosure for a complete expression must propagate input uncertainty and intermediate bounds through that expression. The library gives the application reliable building blocks; the application must still connect them to the mathematical question it is answering.

Chapter 8

Elementary functions without avoidable information loss

Choosing a mathematically equivalent formula can change the accuracy and cost of a floating-point calculation. A dedicated `log1p` call preserves a small increment that would disappear if the program first formed a rounded `1+x`. A scaled-angle function preserves an exact turn fraction that would be perturbed by multiplying by a rounded value of π. These are choices about the problem presented to the library, not merely about the number of guard bits used internally.

The complete program numeric_elementary.c demonstrates both effects. It uses exact dyadic inputs, explicit nearest rounding, and a 53-bit destination, making the examples independent of decimal input conversion.

8.1 Start with the intended expression

The elementary families include natural, base-two, and base-ten logarithms; exponentials and their small-increment forms; integer and general powers; trigonometric functions; inverse angles; and hyperbolic functions. The most useful first question is whether the family already contains a function that expresses the mathematical quantity directly.

Intended quantity	Prefer	Avoid forming first when accuracy matters
$\log(1+x)$ for small x	`ppf_log1p`	A destination-rounded `1+x`
$\exp(x)-1$ for small x	`ppf_expm1`	A destination-rounded exponential
$\log_2(1+x)$	`ppf_log2p1`	A rounded sum followed by `ppf_log2`
2^x-1	`ppf_exp2m1`	A rounded power followed by subtraction
$(1+x)^n$ with integer n	`ppf_compound_si`	A prematurely rounded increment
$\sin(\pi x)$	`ppf_sinpi`	Multiplication by a rounded constant
$\sin(2\pi x/u)$	`ppf_sinu`	Conversion of an exact turn fraction to radians

The dedicated functions keep the defining expression inside their numerical evaluation. This gives them an opportunity to preserve relationships that would already be lost in a sequence of public calls. It does not make the external input exact if the application originally imported an approximation.

8.2 A tiny increment can vanish before the logarithm starts

At 53-bit precision, the spacing immediately above one is 2^{-52}. Take the exactly represented input

$$x = 2^{-80}.$$

It is far smaller than half that spacing. Nearest rounding of `1+x` therefore produces exactly one. A subsequent call to `ppf_log` correctly returns zero for the input it actually receives. The logarithm has not made a mistake; the addition changed the question.

`ppf_log1p` receives the original dyadic x and evaluates the combined expression. Its result remains positive. `ppf_expm1` similarly preserves the small positive difference between $\exp(x)$ and one.

The example prints:

```
log of the rounded 1 + 2^-80 is zero at 53 bits.
log1p(2^-80) and expm1(2^-80) remain positive.
sinpi(1) is exactly zero; sin of rounded pi is not.
```

The positive result may still be inexact. Positivity, representability, and exactness are different properties. The example checks the property relevant to the cancellation problem instead of interpreting every nonzero ternary as failure.

Increasing only the logarithm destination precision cannot undo the first addition's rounding. To recompute the original expression at greater precision, rebuild the inputs and intermediate steps whose information was lost. A higher-precision container holding the same rounded one still represents one.

8.3 Preserve exact angle structure

Ordinary `ppf_sin`, `ppf_cos`, and `ppf_tan` take angles in radians. Their input is the exact stored binary number. If the program obtains an approximation `pi` from `ppf_const_pi`, then calls `ppf_sin(pi)`, it asks for the sine of that binary approximation. Since the approximation is not the mathematical constant π, the result need not be zero.

`ppf_sinpi(x)` means $\sin(\pi x)$. The factor of π is part of the function definition, so the implementation can recognize integer and other exact grid cases before ordinary numerical reduction. At $x = 1$, the result is exactly zero. This is the third observation checked by the example.

For degrees, an exact unit-scaled input can be particularly convenient: `ppf_sinu(x,360,...)` represents $\sin(2\pi x/360)$. For a system that counts phase in turns, use a consistent exact turn scale. The unit parameter is part of the API's domain and must satisfy its documented requirements; do not substitute a native rounded conversion constant when an exact integer scale is available.

Large angles deserve an input-quality check. A binary value of large magnitude can have many significant bits while still having little resolution in its fractional-turn position. Accurate range reduction computes the function of that stored value. It cannot determine which nearby physical angle was intended if measurement or import discarded the needed low bits.

8.4 Inverse angles and quadrants

Use `ppf_atan2` when both Cartesian coordinates are available and the quadrant matters. Computing an ordinary quotient and passing it to `ppf_atan` can lose quadrant information, can encounter division by zero unnecessarily, and introduces an intermediate rounding. The two-argument function has explicit rules for axes, signed zeros, infinities, and NaNs.

Scaled inverse-angle variants return answers in the corresponding π or unit convention. They are useful when the caller ultimately wants a turn fraction rather than radians. State the convention in interfaces and stored data; the same numerical value can mean a different angle under a different scale.

For `asin` and `acos`, check that a real input is in [-1,1]. A dot product of nominal unit vectors can drift slightly outside this interval after earlier rounding or because the vectors were not exactly normalized. Silently clamping the input changes the mathematical problem. If the application intentionally clamps to accommodate a known modeling tolerance, document that tolerance and retain the original residual for diagnosis.

A similar distinction applies to `atanh`: the finite real interior is (-1,1), with poles at the endpoints. `acosh` has a real domain beginning at one. A domain check should compare the actual `ppf_t` value, not a conversion to `double` that can round a nearby out-of-domain value back onto the boundary.

8.5 Integer powers and general powers have different information

When the exponent is an integer in the application's model, pass it through an integer-power interface such as `ppf_pow_ui`, `ppf_pow_si`, or `ppf_pow_z`, according to its representation. This retains exact parity and avoids treating a rounded floating exponent as if it were the intended integer.

A general negative base has a real result for integer exponents, with sign determined by parity. The expression $(-8)^{1/3}$ illustrates a different issue: a finite binary approximation to one third is not exactly one third. A general real-power call with that exponent does not communicate the intention "take the real cube root." Use `ppf_cbrt` or an appropriate integer-degree root function when that is the intended operation.

`ppf_powr` has a stricter domain than `ppf_pow`. It is not simply an alternative spelling. In particular, cases involving a negative base or ambiguous zero/infinity exponents follow its documented invalid-value rules. Choose the function based on the mathematical convention needed by the application, then handle its special values explicitly.

Some noninteger powers are exact, such as $9^{1/2} = 3$ and $81^{1/4} = 3$. The implementation contains exact-case routes so that a numerical enclosure need not be forced to prove equality by indefinitely narrowing around a representable answer. An application should still inspect the returned ternary rather than infer exactness from a familiar-looking decimal rendering.

8.6 What the internal enclosure establishes

For nontrivial cases, the implementation constructs directed bounds at a working precision, attempts to settle the requested output, and refines when the decision remains ambiguous. Depending on the function and argument, the first path may use fixed-point integer endpoints or floating interval objects. The public meaning is the requested function of the exact stored inputs, rounded to the destination precision and mode.

For a deterministic rounding mode, a completed endpoint decision includes the value and the applicable rounding direction and range information. Merely obtaining identical nearest-rounded approximations at two working precisions would not establish these facts. The library's certificate-oriented paths therefore use enclosure finalization rather than a "digits stopped changing" test.

There are still finite implementation limits. Iteration counts, workspace sizes, transformed-argument alignment, and available memory constrain execution. The

current logarithm/exponential interval layer caps a particular dyadic alignment step at 1,000,000 bits. This is a workspace bound on that representation step, not a universal limit on input precision or exponent. An exact shortcut or a proved saturation path may avoid it; another request can be rejected before an enormous temporary is allocated.

A representation or certification rejection returns the documented error sentinel and status flags. Repeating the same request without changing its representation or work policy is not a general remedy. Nor should an application remove a guard simply to obtain an unchecked approximation. Record which function and inputs failed, then choose an explicitly supported alternative formulation or report the limitation to the caller.

8.7 Compose functions with a clear error model

Successful scalar calls do not automatically give a final expression the same single-rounding contract. For example, `log(x)` followed by multiplication by `y`, followed by `exp`, introduces visible intermediate roundings. `ppf_pow(x,y)` can keep those stages inside one evaluation of the power. Both programs can be useful, but they answer different floating-point specifications.

For an application-level bound, use directed operations and valid interval formulas throughout. Monotone functions allow endpoint propagation only on a domain where the monotonicity assertion holds. Sine over an interval spanning a maximum cannot be bounded by evaluating just its two endpoints. Division requires a denominator interval that avoids zero, or a more elaborate treatment of the singularity.

For an application-level approximation, record input precision and the complete formula, perform a precision study, and describe the result as observed stability unless an error bound has been proved. This distinction is useful even in ordinary engineering work: it tells the next reader whether the uncertainty is from input data, numerical rounding, or an unresolved mathematical bound.

Chapter 9

Special functions and difficult domains

Special functions often expose a gap between a formula that exists mathematically and a numerical request that can be completed economically. Poles, sign changes, cancellation, large parameters, and finite implementation bounds all affect how an application should use the interface. A useful wrapper must distinguish these cases instead of reporting every unusual outcome as “not enough precision.”

The complete program numeric_special_domains.c demonstrates four different situations: an exact Gamma value, a finite logarithmic magnitude with a separate sign, a real-domain NaN, and an explicit practical-domain rejection. These are representative calling patterns, not a claim that the example covers every special-function branch.

9.1 Choose the quantity and its convention

The special-function surface includes the Gamma family, beta, zeta, exponential integral, dilogarithm, error functions, Bessel functions of integer order, the arithmetic–geometric mean, and Airy Ai. Consult the function’s mathematical convention before replacing a formula from another library. Similar names can refer to a logarithm, a normalized quantity, or a different branch.

Interface	Quantity or convention
`ppf_gamma`	Real Gamma function, with documented poles and special values
`ppf_lgamma`	$\log \lvert\Gamma(x)\rvert$, plus a separate Gamma sign output
`ppf_lngamma`	Real $\log \Gamma(x)$, where Gamma is positive
`ppf_digamma`	Logarithmic derivative of Gamma
`ppf_gamma_inc`	Upper incomplete Gamma function
`ppf_beta`	Beta function with the implementation's stated real-domain rules
`ppf_erf`, `ppf_erfc`	Error function and complementary error function
`ppf_jn`, `ppf_yn`	Bessel functions of integer order
`ppf_ai`	Airy Ai, subject to a practical finite-argument limit

For positive shape and nonnegative lower limit, the upper incomplete Gamma convention is

$$\Gamma(a, x) = \int_x^\infty t^{a-1} e^{-t}\, dt.$$

It is not the lower integral from zero to x, and it is not automatically divided by $\Gamma(a)$. If a probability formula needs a regularized ratio, that normalization is part of the application's expression and error analysis. The implementation also has real-parameter continuation paths; their existence should not be read as an unrestricted promise of cheap completion for every extreme parameter.

9.2 Gamma: magnitude and sign are separate information

For positive integers, $\Gamma(n) = (n-1)!$. The example uses the exactly representable input five and verifies that the 80-bit result is exactly 24 with zero ternary. This is a useful sanity check, but it is much easier than evaluating a point close to a negative-integer pole.

For negative nonintegral arguments, Gamma changes sign between poles. At $x = -1/2$,

$$\Gamma(-1/2) = -2\sqrt{\pi}.$$

Its absolute value has a finite real logarithm. `ppf_lgamma` therefore provides a finite logarithmic magnitude and a separate sign of -1. `ppf_lngamma` cannot return a real logarithm of that negative value and reports a numerical NaN under its real-domain convention.

These outcomes are not interchangeable. Exponentiating the magnitude returned by `lgamma` loses the Gamma sign unless the application restores it. Treat the auxiliary sign as part of the result, validate it only after a successful call, and preserve it with the numerical output when storing or passing the result onward.

For large positive arguments, a logarithmic representation often avoids an otherwise unnecessary overflow. An application computing ratios of large Gamma values may benefit from a logarithmic formulation, but cancellation in the difference of logarithms still needs analysis. Subtracting two separately rounded large logarithms is not automatically equivalent to one certified beta or ratio evaluation.

9.3 Poles and nearby points require exact input awareness

A negative integer and a nearby negative noninteger can produce radically different Gamma behavior. The distinction must be made using the stored high-precision value. Converting it to a native floating type for an "is this an integer?" check can destroy the very offset that distinguishes the two cases.

The same principle applies when generating a point near a pole. If the intended input is $-n + 2^{-k}$, construct that dyadic expression at sufficient precision. Writing a long decimal number through a native `double` temporary may round it to $-n$. A correctly rounded special-function result for the rounded integer would then answer the wrong input question.

Increasing output precision does not repair that change. Reconstruct the original input from its exact integer/rational description or original text at the new precision. Record whether a test is probing the mathematical pole itself, a representable neighboring value, or a decimal value rounded onto one of those points.

A wrapper should classify NaN and infinity explicitly. A documented pole may return an infinity and set a division-by-zero flag without returning `PPF_TERNARY_ERROR`. A real-domain invalid value may return NaN and set the NaN flag. These are specified numerical outcomes, while the sentinel identifies a request that did not produce its ordinary requested result.

9.4 A practical bound is different from a mathematical domain

Airy Ai is mathematically meaningful for every finite real argument. The current implementation nevertheless restricts its ordinary finite-input evaluation to $|x| \leq 500$. A finite argument outside that interval produces `PPF_TERNARY_ERROR` with `PPF_FLAG_ERANGE`. This is an implementation boundary, not a claim that Ai is undefined there.

The bound is checked on the exact stored input. A number slightly greater than 500 remains out of range even if conversion to `double` or `long double` would round it back to 500. Applications that validate this boundary should therefore use the library comparison functions, as the example does with `ppf_cmpabs_ui`.

The example deliberately calls Ai at 501 after identifying that the input exceeds the supported interval. It verifies the documented rejection and checks that no project-allocation failure was reported. Its output is:

```
Gamma(5) = 24 exactly.
lgamma(-1/2) returns a finite log magnitude and sign -1.
lngamma(-1/2) reports a real-domain NaN.
Ai(501) exceeds the implementation domain: ERANGE, not ALLOC.
```

An application would normally turn the same precheck into a clear user-facing message or choose a different supported method. The example makes the rejected call so that the distinction between numerical range status and allocation failure is executable and reviewable.

Integer-order Bessel evaluation has another explicit practical bound: the general nontrivial `jn` and `yn` paths reject order magnitude above 100,000. Special inputs can be resolved before that work is needed. Even below the bound, a large order or argument can be expensive. A numeric cutoff is not a promise of uniform running time throughout the accepted region.

For upper incomplete Gamma at a finite positive lower limit, the negative-shape recurrence admits shape parameters $a \geq -100000$. A smaller shape is rejected before converting it into a recurrence count. The special cases at a zero or infinite lower limit are handled separately. This is another practical recurrence bound, not the mathematical domain of the defining integral for positive lower limit. Exact-case recognition can avoid some expensive general routes: for example, negative even integer zeta inputs can be recognized from their compact dyadic representation without materializing an enormous integer.

9.5 Complementary functions and cancellation

When the desired quantity is a small complementary tail, prefer its dedicated interface. For example, evaluating `erf(x)` and subtracting its rounded value from one can erase a small `erfc(x)` value. A direct complementary-function call keeps the defining relationship within the evaluator and avoids this particular external rounding step.

The same caution applies to incomplete Gamma formulas assembled from a complete Gamma value minus a lower contribution. Such differences can be much smaller than either term. A formula that is algebraically correct can be numerically fragile if its terms are rounded independently before subtraction.

Near a zero of an oscillatory special function, absolute error and relative error tell very different stories. A small absolute uncertainty can correspond to a large relative uncertainty when the true value is near zero. Choose the accuracy measure that matters to the application. If the sign near a root is important, seek an interval that excludes zero rather than assuming that a displayed nonzero decimal digit proves the sign.

9.6 Certification rejection and allocation rollback

The special-function evaluators use finite working-precision schedules and bounded internal algorithms. An unresolved enclosure, an unsupported practical domain, or an internal representation limit can cause an ordinary numerical rejection. The error sentinel tells the application not to use the destination as a newly computed answer. It does not, by itself, promise that the destination still contains its previous value.

The project-allocation contract is more specific. For the governed public transcendental and special-function entry points, an unrecovered failure of a configured Polyphase allocation raises the sticky allocation flag, returns the sentinel, and preserves initialized numerical destinations. Paired outputs commit together under that contract, and the `lgamma` auxiliary sign is protected with its numerical output. The staging machinery can distinguish a newly failed allocation from an allocation flag that was already sticky before the call.

A numerical rejection without a project-allocation failure may commit a staged NaN or other documented error result. Therefore, code that requires its previous numerical value after *any* unsuccessful request should arrange its own temporary destination and copy only after applying the application's full success test. That test may include classification and domain policy in addition to the sentinel.

This allocation guarantee concerns project-controlled requests and valid initialized objects. It does not intercept GMP's internal allocation behavior or promise recovery from process termination or invalid allocator callbacks. Keep those ownership boundaries visible in a service's resource policy without conflating them with numerical correctness.

9.7 A useful failure-handling sequence

First decide whether the input belongs to the intended real domain and whether it satisfies known practical bounds. Then clear or save the relevant context flags, execute the operation, and check its return contract. If the result is numerical, classify it before treating it as a finite quantity. If the call fails, inspect whether a new allocation failure occurred and retain the original input description for diagnosis.

The response should fit the failure. Correct a mistaken domain convention rather than merely increasing precision. Reconstruct an input that lost its offset during import. Use a logarithmic quantity if the application actually needs a log magnitude. Report a practical-domain limit when the implementation rejects the requested region. Escalate working precision only when the algorithm and application have a justified refinement policy and a finite resource budget.

Tests should include ordinary values, exact cases, points near poles or zeros, practical-boundary neighbors, and invalid-domain cases. A comparison with an independent implementation is valuable evidence over the tested set. Agreement at a few convenient values does not establish all-input correctness or resource-independent completion. The

manual examples are intentionally small; the release's broader validation records state their own exact tested ranges and limitations.

Chapter 10

Numerical recipes and reproducible results

A numerical recipe should say what its inputs mean, what result it promises, and how it decides that enough work has been done. This chapter connects the library's arithmetic and interval interfaces to those application-level questions. The recipes distinguish three increasingly strong observations: a value was computed successfully, repeated calculations look stable, and a justified enclosure certifies a particular rounding decision.

Two complete programs accompany the discussion: numeric_import.c compares decimal text with a native floating input, and numeric_refinement.c certifies a 53-bit square root using an independently constructed exact interval. Both programs check their claims before printing their expected output.

10.1 Recipe: import the information you actually possess

Suppose a data file contains the decimal text `0.1`. There are at least three different objects the application might intend:

1. the exact rational number $1/10$;
2. the nearest binary value at a selected Polyphase precision;
3. the binary value already stored in a native `double`.

These are related, but they are not identical. A binary floating-point number with finite precision cannot equal $1/10$, because the reduced denominator contains a factor of five. A direct decimal parse can round that rational to the selected precision. Importing an existing native `double` instead preserves the native value, including its earlier rounding.

The import example assumes a 53-bit binary `double`, checks that platform condition, and creates two 256-bit destinations. It parses the text into one and imports the C literal through `ppf_set_d` into the other:

```
ppf_set_str(&decimal, "0.1", 10, PPF_RNDN, &ctx);
ppf_set_d(&native, 0.1, PPF_RNDN, &ctx);
```

These two calls have different return contracts. The complete program checks status zero for `ppf_set_str`. It checks exact ternary zero for this particular `ppf_set_d`, because the already-rounded native binary value fits exactly into 256 bits. That exact import does not mean the native input equals the intended decimal rational.

To compare errors without another floating approximation, the example exports both stored values to GMP rationals and subtracts the exact rational $1/10$. It compares the absolute rational errors. The 256-bit decimal parse is closer, though still not exact. It then widens the native-import object to 512 bits and verifies that its rational value is unchanged.

Expected output:

```
Neither binary value is exactly 1/10.
The 256-bit decimal parse is closer to 1/10.
Widening the native import preserves its old value.
```

If the input is already a `double` supplied by another API, preserving that value may be precisely the right operation. The problem arises only when the program claims to have imported the original decimal or measured quantity more accurately than the available data allows. Keep original text or an exact integer/rational representation when later recomputation at higher precision is part of the workflow.

10.2 Recipe: separate computation precision from output formatting

Choose the computational precision before the calculation. Choose the number of display digits afterward, based on what the output is intended to communicate. Printing more digits does not add information, and printing fewer digits can hide a meaningful difference between two internal values.

For reproducibility, retain the binary value or an exact dyadic export in addition to a human-readable decimal string. `ppf_get_z_2exp` expresses a finite regular value as an integer times a power of two. That representation makes the stored value unambiguous. It is especially useful when a later reviewer must distinguish input rounding from algorithmic differences.

For user-facing results, avoid implying that all printed digits are independently established digits of the mathematical target. A formatter can accurately print a stored approximation whose input or modeling error is much larger than its last displayed digit. Report an error bound or uncertainty separately when one is available, and state how it was obtained.

10.3 Recipe: use precision refinement as a diagnostic

A practical precision study recomputes the entire expression at several precisions, for example 64, 128, and 256 bits. Recreate inputs from their original description at each stage. Reusing a low-precision intermediate after widening it defeats the purpose of the study.

Compare results at a common target precision or compare their exact difference. Otherwise the display itself can make unrelated outputs look equal or make harmless low-order changes look alarming. Record whether every call completed successfully, whether the result remained finite, and whether the same range policy was used.

Stability is useful evidence. It can reveal an accidental native conversion, insufficient precision in one intermediate, a cancellation problem, or an unexpected dependence on evaluation order. However, two calculations can share the same input error, formula defect, or biased approximation. Agreement at two precisions does not prove a correct final rounding decision.

Give an approximation-based algorithm a stopping rule that matches its claim. If the application only requires observed stability to a stated display tolerance, label that outcome accordingly and set a finite retry limit. If it requires a certified binary result, construct or obtain justified bounds and use a certification test. Increasing precision is a work strategy; the stopping test supplies the evidence.

10.4 Recipe: build an exact enclosure for a square root

The refinement example targets $\sqrt{2}$. For a positive integer scale k, form the exact integer

$$R = 2 \cdot 2^{2k} = 2^{2k+1}$$

and use GMP integer square root with remainder to obtain

$$L = \lfloor \sqrt{R} \rfloor.$$

Because R is a power of two with an odd exponent, it is not an integer square. Thus the returned remainder is positive and

$$L^2 < R < (L+1)^2.$$

Taking positive square roots and dividing by 2^k proves

$$L2^{-k} < \sqrt{2} < (L+1)2^{-k}.$$

This is an enclosure argument, not an estimate inferred from agreement of approximations. It uses exact integer arithmetic and supplies endpoints whose width is exactly 2^{-k}. The

program checks the nonzero remainder explicitly rather than silently assuming that every future variant of the recipe has the same irrational target.

The central construction is short:

```
mpz_set_ui(radicand, 2);
mpz_mul_2exp(radicand, radicand, 2u * k);
mpz_sqrtrem(lower, remainder, radicand);
mpz_add_ui(upper, lower, 1);

ppf_interval_status_t s = ppf_set_dyadic_interval(
    &result, lower, upper, -(ppf_exp_t)k,
    PPF_RNDN, &ternary, &ctx);
```

The complete program supplies initialization, checks, a finite sequence of scales, and cleanup. It never treats a failed allocation or an interval error as an instruction to continue with unchecked bounds.

10.5 Recipe: refine until the requested decision is established

The destination stays at 53 bits throughout the square-root example. Only the enclosure scale changes. The program tries 8, 16, 32, 64, and, if necessary, 128 bits of fixed-point scale. This separates *target precision* from *work used to establish the target result*.

At the first three scales, the enclosure is too wide to settle the 53-bit request. The program verifies that the undecided call leaves its initial destination value and ternary marker unchanged. At scale 64, the enclosure determines the result and proves that the nearest 53-bit value is above the exact square root.

Expected output:

```
scale 8: refine
scale 16: refine
scale 32: refine
scale 64: certified 53-bit sqrt(2), rounded upward
```

Here “rounded upward” describes the sign of the result’s error under nearest rounding. It does not mean that the requested mode was `PPF_RNDU`. A nearest-rounded answer can lie on either side of its exact target; the ternary records which side was selected in this particular case.

If the finite scale schedule ends without a decision, the program exits unsuccessfully and says that it did not obtain its certificate. It does not promote the last approximation to a certified answer. An application can choose a larger schedule or a different enclosure method, but it should report the outcome actually established by the work it performed.

10.6 Recipe: record a certificate that another reader can inspect

For the square-root recipe, a compact reproducibility record consists of the exact target expression, scale k, integer endpoints, endpoint exponent $-k$, target precision, rounding mode, interval status, ternary, and context range. The integer-square-root remainder supplies a direct check of the enclosure construction.

For a constant, the record can contain the exported enclosure and the constant identifier, together with the construction/validation provenance on which the enclosure relies. For a general expression, record how input intervals and intermediate bounds were propagated. A collection of endpoints without an enclosure argument is not yet a certificate for the intended mathematical expression.

A numerical report should also state its boundaries: tested input ranges, platform and dependency versions, random seed if sampling was used, any retry or resource cap, and whether an independent oracle was involved. These details let another reader reproduce a finite assertion without mistaking it for a universal result.

10.7 Recipe: preserve an application result across rejected requests

Applications sometimes need to keep the last accepted value while attempting a new calculation. Use a separate initialized temporary of the intended destination precision. Run the new calculation into that temporary, inspect the sentinel, numerical classification, and application-specific domain requirements, and copy into the persistent result only after acceptance.

This pattern complements the library's allocation rollback guarantees. It also covers numerical rejections and domain outcomes for which retaining the old value is an application preference rather than an API guarantee. Keep an explicit validity or freshness marker: an unchanged value can be a valid old result while the current request remains unresolved.

For a service or long-running calculation, pair that marker with a concise diagnostic record: the operation, exact or original input description, requested precision and mode, flags, and reason for rejection. Avoid returning a plausible old number without saying that the new calculation failed. A useful numerical interface communicates both the value it has and the question that value actually answers.

Chapter 11

Text, formatting, and binary interchange

Choose an output representation according to what the next reader needs. A person usually needs a short decimal result with a stated precision. A parser needs an unambiguous token and a checked conversion. A saved numerical object needs its value, precision, and special-value information preserved. These are different tasks, and the library provides separate interfaces for them.

Task	Interface	What to check
Read one complete string	`ppf_set_str`	Zero means success; a nonzero result means failure
Read a number at the beginning of a string	`ppf_strtofr`	Error sentinel, end pointer, and any flags of interest
Obtain radix digits and a separate exponent	`ppf_get_str`	Non-null returned pointer; release allocated output with `ppf_free_str`
Produce a presentation string	`ppf_snprintf` or `ppf_asprintf`	Negative result means failure; check truncation separately
Read or write one text token on a stream	`ppf_inp_str`, `ppf_out_str`	Nonzero byte count, then the stream's error state when relevant
Save or restore a numerical object	`ppf_fpif_export`, `ppf_fpif_import`	Zero means success

11.1 Reading numbers deliberately

Use `ppf_set_str` for a configuration field that must contain exactly one number. It accepts leading and trailing white space, but rejects an unconsumed suffix. For example, `"1.25"` is a complete decimal input, while `"1.25 volts"` requires either application-level parsing or the prefix interface. An empty or white-space-only string is not a successful complete conversion. The destination must already be initialized.

```
if (ppf_set_str(&value, "1.25", 10, PPF_RNDN, &ctx) != 0) {
    /* Report an invalid or unsuccessful conversion. */
}
```

The return value here is a status, not the ternary rounding relation returned by arithmetic. Successful parsing can round the input to the destination's precision. Inspect `PPF_FLAG_INEXACT` when the application needs to distinguish exact conversion from rounding, clearing the relevant context flag before the operation if earlier work must not affect that decision.

Use `ppf_strtofr` when a larger grammar follows the number. On return, compare the end pointer with the original input pointer to detect no conversion, then validate the remaining text. A successful prefix does not validate a unit, delimiter, or trailing field. Check `PPF_TERNARY_ERROR` separately before using the result. A pointer into the input remains valid only while that input buffer remains valid.

Positive input bases range from 2 through 62. Base zero requests prefix detection for binary and hexadecimal forms and otherwise selects decimal. The parser recognizes `@` as

a radix-exponent marker in every base, `e` or `E` in bases at most 10, and `p` or `P` as a binary-exponent marker in bases 2 and 16. These distinctions matter: a hexadecimal digit may itself be the letter `e`, so treating every `e` as an exponent marker would be ambiguous. Use an explicit base for a persistent text protocol, and define its special-value policy rather than accepting NaN or infinity accidentally.

Short input does not necessarily mean cheap input. A small exponent token can request a large exact radix power. The parser has construction limits and range checks, but an application accepting untrusted text should also bound input length, requested precision, and its own permitted exponent range. A resource rejection is a conversion failure to handle, not permission to use a partially computed value.

11.2 Digits and the separate exponent

`ppf_get_str` returns significand digits without a radix point. For an ordinary finite value, interpret the result as a fraction beginning immediately before the first digit, multiplied by the base raised to the returned exponent. The sign, when present, precedes the digits. Thus the five decimal digits `12500` and exponent `1` describe `0.12500 × 10`, or `1.25`. They do not describe the integer 12,500.

The `digits` argument requests the number of significant radix digits. Zero asks the library to choose a count from the object's binary precision; `ppf_get_str_ndigits` exposes that sizing calculation. This choice is useful for general output, but a protocol should still specify its base and rounding policy. If exact binary text is required, base 2 with enough digits to retain the stored significand avoids an additional decimal rounding step. For a complete saved object, use the binary interchange interface below.

With `buffer == NULL`, the library allocates the string and retains the allocator provenance needed to release it. Call `ppf_free_str` exactly once. Do not pass that allocation to `free`. Conversely, a buffer supplied by the caller remains caller-owned and must not be passed to `ppf_free_str` merely because `ppf_get_str` wrote into it. The caller-buffer form has no capacity argument; allocated output is usually the easier choice unless storage bounds are already established.

Special values have token representations rather than an ordinary significand/exponent pair. Handle their categories explicitly when consuming the digits API. Do not interpret an exponent variable left from an earlier call as the exponent of NaN or infinity.

11.3 Formatting for people

The `ppf_*printf` family accepts ordinary C conversions, GMP conversions, and Polyphase floating conversions beginning with `%R`. Pass the address of the initialized `ppf_t`, not a copied structure or a C floating value. For example, `"%.2RNf"` prints two digits after the decimal point, using nearest rounding. The `N` belongs to the formatting conversion; it is not a change to the object's stored precision or to the context's default rounding mode.

Conversion	Typical purpose
`%Rf`, `%RF`	A fixed number of decimal places
`%Re`, `%RE`	Scientific decimal notation
`%Rg`, `%RG`	A compact decimal presentation
`%Ra`, `%RA`	Hexadecimal floating notation
`%Rb`	Binary floating notation
`%Pu`	A `ppf_prec_t` value as an unsigned quantity

After `R`, the selectors `N`, `Z`, `U`, `D`, and `Y` request nearest, toward zero, toward positive infinity, toward negative infinity, and away from zero, respectively. A `*` selector consumes a rounding-mode argument immediately before the floating-object pointer. Ordinary dynamic field width and precision consume their own integer arguments in the usual order. Prefer a literal, simple format until those argument positions are familiar.

Choose `ppf_snprintf` for fixed-capacity buffers. Its nonnegative result is the length that would have been produced, excluding the terminating null byte. A result greater than or equal to the supplied capacity means truncation; it is not an allocation error. Test for a negative result before converting that result to an unsigned type. Choose `ppf_asprintf` when the required length is not known in advance. Its successful allocation must be released with ordinary `free`, and failure leaves its output pointer null.

Allocation source	Required release
`ppf_get_str(NULL, ...)`	`ppf_free_str(text)`
`ppf_asprintf` or `ppf_vasprintf`	`free(text)`
Caller-supplied array	No library release; follow the array's own lifetime

Formatting interfaces without an explicit context use the calling thread's default context for their internal diagnostics. Inspecting an unrelated explicit context will not explain their failure. Keep format strings under application control: pass user text through `"%s"`, never as the format itself. Conversions such as `%n` and `%Rn` write through pointers. Their earlier side effects are not rolled back if a later conversion fails. Positional `$` arguments are rejected. Large field widths can demand substantial work even when the final destination buffer is small, so bound user-selected widths and precisions before formatting.

11.4 Streams, locales, and commit boundaries

`ppf_inp_str` skips initial white space and reads a non-white-space token. `ppf_out_str` writes a formatted numeric token. A null input stream selects standard input; a null output stream selects standard output. Applications should normally pass their stream explicitly so an accidental null pointer does not redirect data to a terminal.

Do not confuse numerical rollback with stream rollback. Failed input may have consumed bytes while leaving the destination unchanged. Output staging can fail before any write, but once writing begins a short write or an I/O error can leave a prefix in the stream. Use application-level record framing, temporary files, and a checked file replacement when an entire saved document must be committed atomically. Check flushing and closing too: buffered output can report an error after a successful formatting call.

The text stream formatter consults the active locale's decimal point. The number parser recognizes the library's numeric syntax, including a dot radix point. Keep machine-readable text in the C locale with a defined grammar, or use PPFIF. Avoid changing process locale while other threads format data.

11.5 PPFIF records

The binary representation is called `PPFIF01`. It records value category, sign, binary precision, binary exponent, significand bytes in a defined byte order, and a CRC-32 check. It is independent of the in-memory limb layout and of host byte order. It is not MPFR's experimental FPIF representation, and a raw dump of `ppf_t` is neither a PPFIF record nor a portable file.

Import restores the encoded precision as well as the value. It does not round the record into the destination's previous precision. If an application needs a particular working precision or range, import into a temporary object, validate application-specific limits, and perform a separate rounded assignment into the working destination. Category policies also belong to the application: a valid NaN record can still be inappropriate for a price or a physical measurement.

The default importer bounds the encoded payload and the rounded-up significand storage independently at 64 MiB each. This also prevents a special record with no payload from requesting unlimited storage through a claimed precision. The private build macro `PPF_FPIF_MAX_IMPORT_BYTES` can change that positive construction bound. It is not a per-call memory budget, a limit on the total file size, or a guarantee of peak process memory. Oversized resource requests return `-1`, raise `PPF_FLAG_ERANGE`, and preserve the destination. Malformed, truncated, or checksum-invalid records likewise do not publish a partially decoded object. Not every malformed record necessarily raises the same flag; always check the return value.

A CRC detects accidental corruption; it does not authenticate a sender or protect against deliberate modification. Apply authentication and transport limits at the surrounding protocol layer when they are needed.

The complete example systems_io.c prints the digits and formatted value of `1.25`, then exports and imports a PPFIF record through a temporary stream. It deliberately initializes the receiving object at a different precision and checks that import restores the stored precision. Its cleanup shows the two different string-release rules in one place.

Chapter 12

Memory ownership and in-place operations

A `ppf_t` owns or refers to significand storage; a context owns mutable execution policy and reusable scratch space. Treat both as managed C objects with explicit lifetimes. The most reliable application structure initializes them once, checks each fallible call, and releases them through one cleanup path. That structure is also the foundation for safe in-place operations and for recovery from allocation failure.

12.1 Establish ownership before computing

For local numbers, zero-initialize the structure before a sequence of initialization calls. This allows a common cleanup path to call `ppf_clear` on objects whose initialization was never attempted. Successfully initialize the context before using it to initialize numbers. A failure to allocate a significand during valid `ppf_init2_ctx` initialization leaves a safe-zero object; it does not leave a usable numerical value.

```c
ppf_context_t ctx;
ppf_t x = {0}, y = {0};

if (ppf_context_init(&ctx) != 0) return 1;
if (ppf_init2_ctx(&x, 128, &ctx) != 0 ||
    ppf_init2_ctx(&y, 128, &ctx) != 0) {
    ppf_clear(&y);
    ppf_clear(&x);
    ppf_context_clear(&ctx);
    return 1;
}
/* Assign values and perform checked calculations here. */
ppf_clear(&y);
ppf_clear(&x);
ppf_context_clear(&ctx);
```

Do not initialize an already live object again to change its precision. That can abandon its existing allocation. `ppf_set_prec` changes precision but discards the old value on success, leaving NaN. Use `ppf_prec_round` when retaining and rounding the old value is the intention. `ppf_reserve` reserves capacity; it is not itself a request to change the numerical precision or value.

Do not copy a live `ppf_t` with C structure assignment or `memcpy`. That copies the storage pointer and ownership metadata, so two apparent objects can refer to one allocation and later free it twice. Use `ppf_set` to assign a value into an initialized destination, and `ppf_swap` when ownership of two complete objects should be exchanged. Similarly, use `ppf_context_clone` to create a context with copied policy and its own initially empty scratch state; a raw copy can duplicate ownership of arena blocks.

12.2 Supplying an allocator

`ppf_context_init_custom` accepts a `ppf_allocator_t` containing allocation, reallocation, and release callbacks plus an opaque pointer. Supply all three callbacks. The reallocation callback receives old and new byte sizes; it must leave the old block valid when it returns null. The release callback receives the size associated with that allocation. Respect these sizes when integrating a pool, accounting system, or debugging allocator.

Objects retain the allocator needed to release their own significands. An allocated `ppf_get_str` result likewise retains allocator provenance. This does not extend the lifetime of the opaque application state. Keep that state, and any supporting pool or callback code, alive until every dependent number, string, arena, and other owned resource has been released. Clearing the context is not a substitute for clearing the numbers created through it.

The complete systems_allocation.c example uses a small allocator wrapper that can reject its next request. It first stores `37`, forces the temporary-object allocation in `ppf_set_str` to fail, and checks that `37` remains stored. After clearing the allocation flag, a retry reads `1.25` successfully. The program also checks that all blocks allocated through the wrapper

have been released. This is an example of the project allocator contract, not a simulation of every possible system-wide out-of-memory condition.

If several contexts share one allocator and opaque pointer, the allocator's own synchronization must support the way those contexts are used. Separate contexts do not make a shared allocation counter or pool thread-safe.

12.3 What failure preserves

The library distinguishes initialization, assignment, and temporary resource construction. Their failure outcomes are useful only when the destination was valid and the function's preconditions were met.

Operation class	Useful failure rule
Significand allocation during `ppf_init2_ctx`	The object is safe to clear; there is no previous value to preserve
Covered numerical transaction with an initialized destination	An unrecovered project-allocation failure preserves that destination
`ppf_reserve` or `ppf_set_prec` reallocation	Failure preserves the original object and allocation
Initializer followed by assignment	A later setter failure may leave an initialized object that still needs clearing
Stream input	The number can be preserved although input bytes were consumed
Stream output	Staging can fail before writing; an I/O failure after writing starts can leave a prefix

`PPF_FLAG_ALLOC` is sticky. It says that a project-controlled allocation was refused; it does not by itself say that the operation failed. Some operations recover through an exact fallback and return a normal result with the flag still set. Check the function's result first, then interpret the flag according to the application's resource policy. For an isolated measurement, clear the flag on the same context before the operation. The short convenience flag functions inspect the thread-default context, which may be different from an explicit context passed to the calculation.

The transactional rule covers the specified project-allocation paths. It is not a general promise to preserve the old value for every numerical error. Domain errors, unresolved numerical certification, invalid arguments, and ordinary overflow have their own documented outcomes. Nor does it promise rollback of all externally visible state: flags can change, input streams can advance, and formatting `%n` conversions can already have written a count.

GMP allocations form a separate domain from `ppf_context_t`. A custom Polyphase allocator does not intercept every temporary integer or rational allocated by GMP. C-library stream buffering, some process allocations, and operating-system resources are separate too. Plan application memory limits with those domains in mind. Do not replace GMP's process-level allocation functions casually inside an application that already has live GMP objects or other GMP users.

12.4 In-place arithmetic without accidental overlap

An in-place call such as `ppf_add(&x, &x, &y, ...)` means that one initialized object appears in an allowed destination and input position. It does not mean that unrelated objects may share arbitrary portions of their limb arrays. Consult the function's aliasing classification when building a generic wrapper or using an unfamiliar operation; similarity between function names is not a storage contract.

Ordinary arithmetic families support their documented in-place cases. The destination's precision still controls the rounded result, even when the destination is also an input. If different precision is needed for the result, create or prepare that destination explicitly rather than assuming an input's precision will be inherited.

Multi-output operations need special attention. The two floating outputs of `ppf_modf`, `ppf_sin_cos`, and `ppf_sinh_cosh` must be distinct, although either may individually alias the input where the contract permits it. `ppf_grandom` also requires distinct floating outputs when both are present; its second output may instead be null. `ppf_get_constant_interval` requires distinct lower and upper GMP integer objects. Two pointer variables pointing to the same object are not distinct outputs.

For `ppf_div_with_plan_many`, a destination may coincide with its numerator at the same index. Duplicate destinations and cross-index destination/input overlap are rejected before the batch begins. This prevents a result written early in the batch from silently replacing an operand needed later. Check array relationships before submitting a batch, especially when arrays are assembled from slices or views.

Lifecycle functions are not ordinary in-place assignments. An initializer needs unused destination storage; initializing it from itself cannot preserve a value that initialization destroys. Direct-storage functions likewise have their own capacity, alignment, normalization, and overlap requirements.

12.5 Scratch space and caller-owned limbs

The context arena is useful when an integration needs temporary storage with a shared scope. Take a mark before allocations and release back to it after the temporary work. Growing the arena uses linked blocks and does not move earlier returned addresses. A failed growth request leaves the earlier block chain and mark usable. Restoring a mark ends the lifetime of allocations made after that mark; no saved pointer into those allocations may be used afterward.

`ppf_arena_reset` keeps reusable storage rather than necessarily returning every byte to the operating system. `ppf_context_clear` releases the context's arena. Do not use an arena mark as a permanent bookmark across context cleanup, and do not mix marks from different contexts.

Advanced external-significand storage avoids ordinary ownership of limb memory. Its backing allocation must have enough capacity and remain valid while the object uses it. Clearing an externally backed object does not transfer ownership of that backing allocation to the library. Growing, relocating, or editing the buffer requires the appropriate direct-storage procedure; bypassing it can break normalization and expose readers to inconsistent data. Start with ordinary owned objects unless the integration has a measured need for direct storage and can enforce these lifetimes.

At shutdown, release returned strings and user-owned numbers, destroy plans and random states, clear explicit contexts, and then perform any desired thread-local cache cleanup. That order keeps allocator state alive until its dependents are gone and makes repeated computations easier to reason about.

Chapter 13

Threads, runtime policy, and cache lifetime

The simplest concurrency arrangement gives each worker its own context and destination objects, while sharing only inputs that remain immutable until all workers finish. This arrangement works whether the library was built with internal OpenMP support or with its serial numerical paths. Application threads and internal numerical workers are separate layers of concurrency.

13.1 Share values, own mutations

Resource	Concurrent-use rule
Initialized number used only as an input	May be shared while its storage and value remain unchanged
Destination number	Requires exclusive ownership during mutation
Explicit context	Requires one-thread-at-a-time ownership or an application lock
Thread-default context	Private to its calling thread
Completed divisor plan	May be applied concurrently with separate destinations and contexts
GMP random state	Requires exclusive ownership or an application lock
Constant and transcendental caches	Internally synchronized; purges may wait for active users

An input is no longer read-only when another thread resizes it, swaps it, clears it, edits its limbs, or uses it as an aliased output. Keep those actions outside the readers' lifetime. `const ppf_t *` helps express the intended use, but it cannot stop a different pointer from mutating the same object. The application must enforce the ownership rule.

An explicit context is mutable even when every numerical operand is read-only. An operation can update its flags or use its scratch arena. Passing one shared context to otherwise independent calculations therefore needs synchronization. Giving each worker a context is usually simpler than locking around every arithmetic call. If the workers need common settings, initialize their contexts from a stable policy or clone a context before starting the concurrent work. A clone copies flags as well as policy, so clear inherited flags when a clean diagnostic interval is wanted.

The complete systems_threads.c example starts two POSIX threads. They both read one initialized value, `7`, and independently square it into private destinations. Each checks that its result is `49`. Only the main thread prints, after joining both workers, so the example's output is deterministic. The shared input is cleared after the joins. The same pattern scales to a larger worker pool: worker-local context, local output, immutable shared input, and an explicit completion boundary.

13.2 Default contexts and flags

The convenience default context is thread-local. Changing one thread's default precision, exponent range, rounding mode, or flags does not change another thread's defaults. This is useful for a small program, but it also means that a worker cannot report its numerical flags merely by having the main thread inspect the main thread's default context.

Collect the status and flags in the worker that performed the operation, then publish an application result record after the work is complete. Such a record can contain the operation status, a copied flag mask, and the resulting value or a reference whose ownership is transferred to the receiving thread. Do not publish a pointer to a worker's stack-allocated context or to an object that the worker clears before the receiver finishes reading it.

Formatting functions without a context argument use the calling thread's default context. A worker using an explicit numerical context can therefore have two relevant diagnostic sources: its explicit calculation context and the default context used for formatting. Keeping presentation in a separate stage can make this distinction easier to manage.

13.3 Choosing the level of parallelism

The CMake option `PPF_OPENMP` accepts `OFF`, `ON`, and `AUTO`. `OFF` builds without the OpenMP runtime dependency. `ON` requires usable OpenMP support for the chosen compiler. `AUTO` enables support when the compiler and runtime probe succeeds. These options do not change the ownership rules for objects, contexts, or custom allocator state.

Operations exposing a worker-count argument receive a request, not a promise to launch that many workers. The selected algorithm may have fewer useful tasks; small inputs may use a serial route; and nested parallel execution can be limited. A request of many workers is therefore not evidence that the operation used the whole machine. Consult operation statistics where the interface provides them and retain the build configuration when measuring parallel behavior.

The constant-family default worker setting is process-wide and atomic. The trigonometric default worker preference is thread-local. Configure these preferences deliberately, preferably before the relevant workload starts. Changing a process-wide preference during a timing campaign makes the campaign harder to interpret even when the change is safely synchronized. Explicit worker-count forms are useful when one call needs a policy different from the surrounding application's default.

Avoid oversubscription. For a server processing many independent requests, several application workers each requesting a large OpenMP team may compete for the same cores and memory bandwidth. Begin with one parallel layer, then measure whether nested or mixed execution improves the actual workload. A throughput-oriented server and a single very-high-precision calculation can need different policies. Neither a worker count nor an environment setting alone guarantees CPU affinity or exclusive access to a core.

13.4 Reproducible results and reproducible execution

Specify precision, rounding mode, exponent range, and input values explicitly when reproducibility matters. Record the library and dependency builds too. Numerical reproducibility is different from repeating the same algorithm, cache state, schedule, allocation count, or elapsed time. A planner can choose an exact computational route that differs from another route while producing the same rounded numerical result.

Faithful rounding needs particular care: its contract permits an allowed neighboring result rather than the unique result selected by a specified directed or nearest mode. Do not use a byte-for-byte comparison across implementations as the sole validity test for faithful rounding. Likewise, special-function acceptance remains subject to documented precision and construction limits; a worker policy is not a guarantee that every possible input can be certified within those limits.

For randomized calculations, keep a separate initialized GMP random state per worker or lock a shared state. Choose and record the seeding scheme at the application level. Reusing the same seed independently in every worker can produce repeated streams, which may be undesirable even though the calls are free of data races. Thread scheduling should not silently determine the meaning of a reproducible experiment's sample numbering.

13.5 Cache reuse and cleanup

Caches reduce repeated construction work for constants and related calculations. They also retain memory after an individual function returns. That retained memory is not automatically a leak: it can be intentionally owned by a cache or a reusable thread workspace. Distinguish an increasing unreachable allocation from a stable reusable allocation before changing an application's memory policy.

`ppf_clear_constant_cache` releases a selected constant's cached value; `ppf_free_constant_caches` clears the constant caches collectively. `ppf_free_cache2(PPF_FREE_GLOBAL_CACHE)` covers the global-cache cleanup interface. Purging shared caches is supported while other threads calculate, but it takes the corresponding locks and may block. A later call can rebuild what was removed. Periodic purging in a latency-sensitive request path can therefore make both that request and subsequent requests slower.

`ppf_free_cache2(PPF_FREE_LOCAL_CACHE)` affects the calling thread's local scratch pool and product workspaces. It cannot release another live worker's local storage. If workers retain large temporary workspaces, schedule local cleanup in those workers at a sensible boundary, such as after an unusually large job or when returning a worker to an idle pool.

`ppf_thread_cleanup` provides explicit cleanup of heap-backed state owned by the calling thread and may be called repeatedly. On the POSIX baseline, ordinary thread exit also performs cleanup when the thread-specific destructor key was successfully registered. `ppf_`

`buildopt_thread_cleanup_p` reports that key's availability. Explicit cleanup remains useful for controlled shutdown, for the main thread, and before unloading the library.

Thread cleanup does not destroy application-owned numbers, explicit contexts, divisor plans, or GMP random states. Clear those resources through their own lifecycle interfaces. Before unloading a shared library, stop new calls, wait for active workers to finish, release their resources and local state, and only then unload it. Automatic thread cleanup cannot make a callback into an already unloaded library safe.

Use runtime identity functions such as `ppf_get_version`, `ppf_abi_tag`, and the build-option queries when constructing a diagnostic report. They help identify the library actually loaded, which can differ from a header found first on the compiler's include path. Keep these diagnostics separate from numerical output that another program expects to parse.

Chapter 14

Application integration and performance measurement

Integrate the installed library first, then tune a representative workload. This sequence separates packaging mistakes from numerical mistakes and gives measurements a clear dependency configuration. The production core uses GMP for integer machinery and does not require MPFR. The optional MPFR adapter is a separate library for applications that need that boundary.

14.1 Build an installation you can identify

Use CMake for an installed package. A serial core build is a useful starting point because it has fewer optional dependencies and is easy to reproduce:

```
cmake -S . -B build/core \
  -DCMAKE_BUILD_TYPE=Release \
  -DPPF_OPENMP=OFF \
  -DPPF_MPFR_SUPPORT=OFF
cmake --build build/core -j2
cmake --install build/core --prefix "$HOME/.local"
```

Run these commands from the Polyphase source directory. `-j2` controls build parallelism, not the number of workers used by a numerical operation. The installation prefix is an application deployment choice; it need not be the example shown here. On the supported Linux baseline, development headers and libraries for GMP, a C11 compiler, POSIX threads, and the platform math library are required. Install the development files for the architecture actually being compiled, rather than relying only on a runtime shared library.

Both shared and static library forms are normally built. Use `PPF_BUILD_SHARED=OFF` or `PPF_BUILD_STATIC=OFF` when only one form is wanted, leaving at least one enabled. Use a fresh build directory when changing the compiler, sanitizer configuration, or major de-

pendency settings. A stale CMake cache can otherwise continue to select an old compiler or dependency path despite changes in the shell environment.

Native instruction tuning is disabled by default. Enable `PPF_ENABLE_NATIVE_OPTIMIZATIONS` only when the deployment machines support the instructions selected for the build machine. A binary that performs well locally but fails with an illegal instruction on a recipient's computer is not a successful portable optimization.

14.2 Consume exported CMake targets

In a separate application directory, use an installed package rather than copying internal source files into the application:

```
cmake_minimum_required(VERSION 3.23)
project(polyphase_application LANGUAGES C)
find_package(PolyphaseFloat CONFIG REQUIRED COMPONENTS core)
add_executable(polyphase_application main.c)
target_link_libraries(polyphase_application PRIVATE PolyphaseFloat::shared)
```

Set `CMAKE_PREFIX_PATH` to the installation prefix when configuring that application. It should not point at the Polyphase source tree. The exported target supplies include paths and its declared link dependencies. Select `PolyphaseFloat::static` for the archive form, provided that form was installed. This is preferable to maintaining a handwritten list of transitive dependencies for each OpenMP and platform configuration.

Request `COMPONENTS core` when the application does not use MPFR. An unqualified package request can load an installed adapter as well. Request `COMPONENTS mpfr` when the adapter is required; the package then loads the core and exposes the installed `PolyphaseFloat::mpfr_shared` and/or `PolyphaseFloat::mpfr_static` targets. A missing required component should fail configuration early instead of becoming an unexplained link error later.

An installed header being found does not prove that the intended library is loaded at runtime. Configure the platform loader for the installed shared library directory using the application's deployment policy. Build-tree runtime paths are convenient during development but are not a replacement for a distributable application's installation layout. When diagnosing an unexpected version, compare runtime identity with the headers and package configuration selected during compilation.

14.3 Using pkg-config or Make

The core package name is `polyphase-float`; the adapter package name is `polyphase-float-mpfr`. For a small application with installed dependencies:

```
cc -std=c11 main.c -o my_app \
  $(pkg-config --cflags --libs polyphase-float)
```

If the prefix is outside the system search path, add the directory containing the installed `.pc` files to `PKG_CONFIG_PATH`. Library directories can be nested or architecture-specific; use the actual installation layout instead of assuming that every system installs into a directory literally named `lib`. The installed pkg-config metadata is designed to follow a relocated complete prefix when relative install directories are used.

`pkg-config --static --libs polyphase-float` adds the private dependency flags needed by static linking. It does not itself force the linker to select an archive when a shared library is also available. Use the explicit CMake static target, a static-only installation, or a deliberate linker selection when that distinction matters. Fully static linking of all system dependencies is a separate platform question from selecting Polyphase's static archive.

The source Makefile supports development commands such as `make test`, `make audit`, and named `bin/` targets. CMake is the canonical installation route. Keep independent compiler or sanitizer builds in separate directories or clean copies; reusing old objects with different instrumentation can produce misleading failures. A successful source-tree test does not by itself exercise downstream discovery of an installed package.

14.4 Crossing the MPFR boundary

Build and install the adapter with official MPFR development headers when an application needs `<polyphase_float_mpfr.h>`. Some local validation modes can use a minimal compatibility declaration set with an existing MPFR shared library. That arrangement does not supply the installable adapter contract for downstream applications. Prefer an explicit `PPF_MPFR_SUPPORT=ON` when the adapter is required, so missing development dependencies are reported at configuration time.

The three conversion directions have distinct precision policies:

Interface	Precision policy	Return convention
`ppf_import_mpfr_exact`	Adopts the MPFR source precision for exact import	`0` on success, `-1` on failure
`ppf_set_mpfr`	Keeps the initialized Polyphase destination precision and applies the requested rounding and context range	Numerical ternary; check `PPF_TERNARY_ERROR` before its sign
`ppf_get_mpfr`	Rounds into the initialized MPFR destination's precision using an MPFR rounding mode	MPFR numerical ternary

All arguments must be valid, and destinations must satisfy their respective initialization requirements. A return value is not a substitute for these preconditions. Exact import may change the destination precision before a later failure; it does not promise to preserve the previous destination on every failure. Check the function-specific return convention rather than assuming that every adapter call returns the same kind of status. MPFR's own flags and exponent policy remain part of the MPFR side of the boundary. A Polyphase context is not a replacement for MPFR's runtime state.

The adapter also provides two enumeration translators: `ppf_rounding_from_mpfr` maps an MPFR rounding mode to its corresponding Polyphase mode, and `ppf_rounding_to_mpfr` maps in the opposite direction. Both translate the six concrete modes: nearest, toward zero, upward, downward, away from zero, and faithful. Both fall back to nearest for an unrecognized value; they do not return an error indicator. In particular, `ppf_rounding_to_mpfr(PPF_RND_DEFAULT)` yields `MPFR_RNDN`. It does not consult a Polyphase context. If the intended policy is a context's current default, resolve that policy to its concrete rounding mode before translating it.

The complete systems_mpfr_bridge.c example imports an exact MPFR value `5/4`, verifies adoption of its 128-bit precision, multiplies by `8` in Polyphase, and exports the exact result `10` to a 53-bit MPFR destination. The small exact values make the example's precision changes visible without confusing them with unavoidable rounding.

For migration of a larger calculation, convert at a well-defined boundary, perform a substantial sequence of operations in one representation, and convert back when needed. Alternating representations at every elementary operation can add allocation, conversion, and repeated-rounding costs. Measure the complete application path if frequent conversion is unavoidable. Do not attribute all of that path's cost to either arithmetic engine alone.

14.5 Make the workload stable before tuning

Start by reusing initialized objects and choosing realistic working precisions. Repeated initialization and teardown inside a tight loop may dominate a small arithmetic operation. Separate object setup, cache warmup, plan construction, and timed application according to what the real workload does. A reusable divisor plan is valuable only when enough applications amortize its construction and storage costs.

Preserve the intended numerical calculation while optimizing it. A fused operation and a sequence with an intermediate rounding are different computations. Choosing fewer printed digits improves presentation costs but does not reduce the precision of an already computed result. Likewise, raising a worker count is a scheduling request, not an assurance of lower latency. Record which question a measurement is intended to answer before changing several variables at once.

The portable product planner selects algorithms from operation, size, and worker conditions. Optional machine-specific profiles can change those choices; explicit algorithm requests have their own selection semantics. Keep any custom profile with the build record. A profile chosen on one CPU or one cache-placement policy is not automatically suitable for another.

14.6 Interpret measurements at their actual scope

A useful timing record identifies the operation, inputs or generation rule, precision, rounding mode, worker count, build flags, dependency builds, hardware, and correctness check. Retain raw samples rather than only a best time. State whether the result measures a cold call, a warmed cache, a planned application, or an end-to-end task including setup and conversion.

Compare equal questions. A many-worker latency measurement and a one-worker measurement answer a scaling question; they are not automatically an equal-resource throughput comparison. Include plan construction when claiming benefit for a one-off division. Include output formatting when the application spends most of its time writing results. If system load or CPU placement changes between runs, an observed difference may have several causes.

The repository's benchmark pipeline retains correctness-gated samples and uses independent process launches as the experimental units. Multiple samples inside one process describe that process's variability; they do not establish independent reproducibility. A correctness gate checks the tested values and conditions, not every mathematical input. Report performance as a result for the measured configuration and workload, with its uncertainty and omitted costs stated. No benchmark smoke test alone establishes a general speedup.

Chapter 15

Troubleshooting and diagnostic workflow

Begin with the failing call, its actual return convention, and the context used by that call. Record the input values and their precisions before changing code or increasing precision. Many apparent arithmetic failures are caused by reading the wrong flags, changing a destination's precision in a way that discards its value, or loading a library different from the one used at compile time.

15.1 First distinguish status from rounding

An arithmetic ternary result is negative when the stored result lies below the exact result, zero when exact, and positive when above, subject to the operation's documented special cases. `PPF_TERNARY_ERROR` is a distinct error sentinel. Therefore a condition such as `if (ppf_add(...))` treats ordinary inexact arithmetic as failure. Check the sentinel first, then interpret the ternary relation if needed.

Other interfaces use other conventions. `ppf_set_str` returns zero on successful complete conversion, while `ppf_strtofr` returns a ternary relation or its error sentinel and also reports an end pointer. PPFIF import/export use zero for success. Formatting returns a byte count or a negative failure result. Stream text functions return counts. A generic wrapper must preserve these distinctions instead of converting every nonzero integer to an error.

Flags are sticky and describe the context on which they were raised. Clear the relevant flags before an isolated diagnostic call, read them immediately afterward, and preserve them before unrelated output formatting can change another context. A normal return together with `PPF_FLAG_ALLOC` can describe a successful fallback after a preferred allocation was refused.

15.2 Numerical symptoms

Symptom	What to examine	Practical next step
A correct-looking result is reported as an error	Was a nonzero ternary result treated as failure?	Compare with `PPF_TERNARY_ERROR`; report inexactness separately
A value unexpectedly becomes NaN after changing precision	Was `ppf_set_prec` used?	Use `ppf_prec_round` when the old value must be retained and rounded
Increasing an input's precision does not improve the result	What is the destination precision, and how were the inputs originally obtained?	Increase working and destination precision deliberately; reconstruct inputs from exact source data
Decimal input differs from its decimal spelling	Is the value exactly representable in binary at this precision?	Inspect inexactness and rounding; use an exact integer/rational source when appropriate
Printing fewer digits seems to change the result	Is the discrepancy only in the formatted representation?	Compare stored values independently of their display strings
A result overflows or becomes tiny unexpectedly	Is the intended context range active?	Record `emin`, `emax`, and the explicit/default context choice before the call
Flags appear unrelated to the current operation	Were earlier sticky flags retained, or was the wrong context queried?	Clear and inspect the same context around the operation
A special function returns an error on finite input	Is the input admitted, and could the bounded evaluator resolve it?	Check the function's domain and construction limits; retain the flags and exact input for diagnosis
Faithful results differ in the last bit	Is the comparison expecting a unique nearest result?	Check the faithful-rounding contract or choose a specified directed/nearest mode

More precision cannot repair information lost before the value entered the library. Passing a decimal literal through a C `double` first fixes it to that binary approximation. Reading the original string or constructing the exact rational gives the high-precision calculation the intended starting data. Similarly, a later high-precision destination cannot undo an earlier rounded intermediate. Identify the first irreversible rounding step before adding precision everywhere.

For a suspected numerical defect, reduce the calculation while retaining the same rounding mode, precision, range, and special-value categories. Compare out-of-place and permitted in-place forms when aliasing is involved. If an independent reference is used, match precision and exponent policy on both sides and distinguish a reference disagreement from a proven library defect. Keep the original failing input even if a smaller reproducer is found.

15.3 Text, files, and ownership symptoms

Symptom	Likely check	Practical next step
A parsed field accepts an unwanted suffix	Was the prefix parser used without checking `endptr`?	Use `ppf_set_str`, or validate the suffix explicitly
Text output cannot be read back	Base, exponent marker, locale, or too few digits may differ	Define the text grammar and locale; use PPFIF for saved objects
`ppf_get_str` digits look too large by a power of the base	The separate exponent may have been ignored	Place the radix point before the first digit and apply the returned exponent
A formatted buffer is silently incomplete	Was the `snprintf` required length checked?	Treat a nonnegative length at least as large as capacity as truncation
Freeing a returned string crashes	Which interface allocated it?	Pair `ppf_get_str` with `ppf_free_str`, and `ppf_asprintf` with `free`
An imported object's precision changes	PPFIF restores encoded precision	Import into a temporary, then assign into the desired working precision
A small binary record is rejected for resources	The encoded precision can require large significand storage	Check the 64 MiB default import bounds and the producer's record metadata
Retrying failed input starts at the wrong place	Input consumption is not rolled back with the number	Reposition a seekable stream or restore an application-owned input buffer

Do not diagnose a saved record solely from its extension. PPFIF has its own identifier, layout, and integrity check. MPFR FPIF, a raw structure dump, and a text number are different representations. A checksum mismatch is a reason to reject a record; removing the check to make an import succeed conceals the problem. Retain a bounded failing record and the producer's identity when investigating interoperability.

15.4 Allocation and concurrency symptoms

Symptom	What to examine	Practical next step
Clearing one number damages another	Were live structures copied by assignment or `memcpy`?	Use initialized destinations with `ppf_set`, or exchange ownership with `ppf_swap`
A custom allocator fails during cleanup	Did its opaque state or pool expire too early?	Keep allocator state alive until all dependent objects, strings, and contexts are cleared
A failed resize destroys the original block	Does the custom reallocator preserve its input on failure?	Correct the callback contract before retrying numerical work
Memory remains after a large calculation	Is it an arena, thread workspace, or global cache?	Clean the relevant owner at a deliberate lifecycle boundary
Several threads occasionally produce corrupted state	Are a context, destination, random state, or raw buffer shared mutably?	Give workers private state or protect the whole affected operation with a lock
More workers make a workload slower	Are internal teams competing with application workers?	Measure one parallel layer first, then compare controlled configurations
Shutdown or library unload crashes	Can another thread still call the library or run its destructor?	Join workers and release their state before unloading

An allocation flag does not identify every process allocation. If a GMP or system allocation fails outside the recoverable project allocator domain, the ordinary destination-preservation guarantee is not a complete recovery strategy. Likewise, a custom allocator that returns an invalid pointer or reports a false capacity breaks the caller contract. Start fault-injection tests with a simple, correctly accounting allocator and a valid initialized destination, as in the allocation example.

15.5 Build and deployment symptoms

Symptom	Practical next step
GMP header or library discovery fails	Install matching development files and check compiler architecture and search paths
A core-only consumer unexpectedly requests MPFR	Use `find_package` with `COMPONENTS core`
The adapter is unavailable after installation	Build it with official MPFR development headers and request `COMPONENTS mpfr` in the consumer
A static link has unresolved dependencies	Use the exported static target or pkg-config's static dependency flags; remember that selecting an archive is a separate step
A program links but cannot load a shared library	Correct its installed runtime-library search configuration
OpenMP configuration or linking fails	Select the matching compiler runtime, or build with `PPF_OPENMP=OFF`
An executable fails with an illegal instruction elsewhere	Check native compiler tuning and the target CPU instruction set
Tests or examples are absent from a build	Review CMake feature detection and build options, then list configured tests

When a build behaves inconsistently after a compiler or sanitizer change, configure a new directory before investigating numerical code. Run one small installed consumer to distinguish package discovery and loader problems from failures inside a larger application. Capture the actual library path and runtime version, not only the version requested in a build script.

15.6 Prepare a useful diagnostic record

A concise reproducer should initialize its own objects and contexts, contain the exact input construction, perform the failing operation, check the result, and clean up. Include the precision of every relevant object, the rounding mode, exponent limits, return value, and flag mask. For a parser problem, include the bytes or a safely bounded file, the base, and the first unconsumed position. For an aliasing problem, state which pointers refer to the same object. For a concurrency problem, identify which resources are shared and the point at which readers finish.

Also record compiler and dependency versions, build options, optional interfaces, worker settings, and whether the failure occurs in a fresh process. Keep the original diagnostic

output and distinguish an actual failure from a test that was never executed. A missing executable permission, an unavailable sanitizer runtime, or a timeout is useful evidence, but none is a passing numerical result. These details make a report actionable without requiring another reader to reconstruct the application’s entire environment.

Chapter 16

Numerical limits and admission policies

Arbitrary precision makes the precision a parameter. It does not remove finite storage, bounded certification, or the need to validate an argument before constructing a large intermediate. This chapter explains how to treat those limits in an application. The source-level inventory in `docs/NUMERICAL_LIMITS.md` records the individual policies and the available boundary evidence.

16.1 Range, domain, and work are different constraints

A mathematical domain restriction concerns whether a real-valued answer exists under the operation's definition. A configured exponent range concerns which answers can be represented in the current context. A work limit bounds a particular implementation route. These restrictions can produce different status values, flags, and destinations. Do not infer one from another.

A shortcut can decline an input while a general evaluator remains available. An evaluator can also exhaust its finite certification budget and return a numerical error. Raising the destination precision may make a later attempt more expensive without settling the original uncertainty. Check the actual result rather than treating a larger precision as a universal remedy.

Path	Current policy	Application consequence
Parsed textual exponents	Direct power construction has an absolute exponent budget of 2,000,000. Larger cases require a justified range classification or may be rejected.	Validate input sizes and check parse status.
Ordinary periodic trigonometric reduction	The general path can reject a request needing more than 1,000,000 extra reduction bits.	Prefer exact scaled-grid interfaces when the input is naturally specified in that form.
Finite Airy Ai input	The admitted absolute input is at most 500, compared exactly.	Check the domain before requesting work outside it.
General integer-order Bessel paths	Finite order magnitude is bounded by 100,000.	Do not interpret rejection as a proof that the mathematical function is undefined.
PPFIF import	The default maximum is 64 MiB for both payload and limb-rounded significand storage.	Keep the default for untrusted data unless the application deliberately chooses another positive cap.
General dyadic certificate construction	Endpoint alignment is limited to 1,000,000 extra bits by default.	An unusable enclosure can cause refinement or numerical rejection.
Exponential-minus-one construction	The same extra-bit cap bounds an intermediate endpoint's exponent gap from one before subtraction.	A compact enormous input need not be expanded into an enormous exact intermediate.
Negative-parameter incomplete Gamma at finite positive second argument	Recurrence admission requires the first argument to be at least -100,000. Separate special-value paths have their own rules.	Test the admission condition before relying on this recurrence path.
Negative-integer zeta	Large negative even integers are recognized directly as exact zeros. The small odd-integer Bernoulli shortcut is admitted before integer expansion.	A shortcut threshold is not a claim that all other zeta arguments are outside the mathematical domain.

The default import limit can be changed through the documented `PPF_FPIF_MAX_IMPORT_BYTES` build configuration. The dyadic extra-bit budget is an alignment policy, not a global ceiling on destination precision. A series term count is likewise not the same quantity as an input exponent or a byte allocation limit.

16.2 Attempts and fallback routes

The implementation uses distinct budgets for distinct algorithms. General transcendental and special-function certification, root refinement, interval arithmetic, constant builders, text conversion, and fast shortcuts do not all share a single retry count. Individual series and recurrences have their own iteration and admission checks. The source inventory identifies these separately instead of presenting one number as a library-wide limit.

Many of those numbers control efficiency or a particular route. A fast path may decline and fall back. A builder can stop without a certificate. A final evaluator may reject after bounded attempts. Only a proven enclosure and a successful settlement justify a certified returned result. Counting attempts, or observing that a last term is small, does not supply that proof.

The numerical-arguments reference in `docs/NUMERICAL_ARGUMENTS.md` connects reviewed paths to their enclosure or remainder arguments and records remaining obligations. Its distinction between a written argument and finite execution evidence matters: both are useful, and neither should be silently substituted for the other.

16.3 Handling a numerical rejection

For an operation whose contract uses the numerical ternary convention:

1. Check for `PPF_TERNARY_ERROR` before reading its sign.
2. Inspect the operation's flags and documented output state.
3. If the operation committed NaN on certification or admission failure, do not keep using the old destination as if it were still present.
4. Decide whether to choose another supported representation, change a justified precision or range parameter, report the unsupported request, or use another algorithm with its own checked contract.

This sequence does not turn every error into the same case. Invalid arguments, an unsupported resource request, and a recoverable project-allocation failure can follow different paths. Transactional allocation rollback preserves the prior numerical destination on an unrecovered project-allocation failure, but sticky flags may change. GMP allocations, I/O side effects, and application callbacks remain outside that rollback guarantee.

16.4 Designing application limits

Give the application its own input, precision, time, and output-size budgets. These should reflect its purpose and the resources actually available. Check string lengths and exponent syntax before accepting a large batch. Cap import bytes and total record counts. Estimate the total working set before assigning one large object to every worker. Treat output formatting as work too: a small stored value can still be asked to produce a very large number of digits.

When retaining a failure report, record the function, exact input encoding, destination precision, rounding mode, context range, flags, and return value. Keep the source version and build features with that record. An exact binary or hexadecimal input is more useful than a shortened decimal display when reproducing a rounding boundary.

16.5 Interpreting validation

The example suite checks its complete programs and expected outputs. Numerical regressions exercise selected domains and resource boundaries. Sanitizers inspect the executions they observe. Differential comparisons test agreement on their enumerated cases. These results support concrete engineering claims. They do not imply support for an untested platform, unbounded precision, or every admitted input.

A stable public contract can state its finite limits plainly. It should also state which mathematical and execution obligations remain open. The release readiness record carries those obligations separately from this user's guide.

Chapter 17

Core API reference

This reference covers every core function declared in `<polyphase_float.h>`. The five optional MPFR adapter functions are described separately in the integration chapter. Entries are arranged by task, then by name. Use PDF search or the alphabetical index to find a symbol.

All ordinary arithmetic objects must be initialized and valid. Unless an operation explicitly changes precision, the destination's precision governs its result. Mutating an object or an explicit context requires exclusive access. No function transfers ownership unless its entry says so.

A recorded allocation contract describes recoverable project allocation, not every allocation made by GMP, libc, or the operating system. Numerical range rejection and certification exhaustion have their own output rules. In particular, an error sentinel alone is not a promise of destination preservation.

The numerical-limits appendix applies throughout this reference. Absence of a function-specific limit in an entry is not a claim of unlimited work, a termination proof, or universal correct rounding. The full source and individual numerical arguments remain the authority for implementation details.

17.1 Contexts, allocators, flags, and scratch memory

ppf_allocflag_p

Report whether the allocation-failure flag is set in the calling thread default context.

```
int ppf_allocflag_p(void);
```

Result. A predicate result: zero means false and a nonzero value means true.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_arena_alloc

Allocate temporary aligned memory from a context scratch arena.

```
void *ppf_arena_alloc(ppf_context_t *ctx, size_t bytes, size_t alignment);
```

Result. A pointer with the ownership conditions below. Failure indicator: `NULL where documented`.

Context and rounding. A null `ctx` selects the calling thread's default context.

Ownership. storage remains owned by the context until release, reset, or clear.

Allocation and failure. On unrecovered project-allocation failure, returns NULL without adding a new arena block or changing the existing arena allocation position; earlier arena allocations remain valid.

ppf_arena_mark

Mark the current scratch-arena position.

```
ppf_arena_mark_t ppf_arena_mark(const ppf_context_t *ctx);
```

Result. The value described above, with the declared C return type.

ppf_arena_release

Release scratch allocations made after a saved arena mark.

```
void ppf_arena_release(ppf_context_t *ctx, ppf_arena_mark_t mark);
```

Result. No return value.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_arena_reset

Reset the context scratch arena for reuse.

```
void ppf_arena_reset(ppf_context_t *ctx);
```

Result. No return value.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_clear_allocflag

Clear the allocation-failure flag in the calling thread default context.

```
void ppf_clear_allocflag(void);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_context_clear

Release memory owned by an explicit context.

```
void ppf_context_clear(ppf_context_t *ctx);
```

Result. No return value.

Context and rounding. The supplied context must be non-null and initialized.

ppf_context_clone

Create a new context with the same policy settings as another context.

```
int ppf_context_clone(ppf_context_t *dst, const ppf_context_t *src);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Context and rounding. The supplied context must be non-null and initialized.

ppf_context_flags

Read all exception flags from a context.

```
ppf_flags_t ppf_context_flags(const ppf_context_t *ctx);
```

Result. The value described above, with the declared C return type.

ppf_context_flags_clear

Clear selected exception flags in a context.

```
void ppf_context_flags_clear(ppf_context_t *ctx, ppf_flags_t mask);
```

Result. No return value.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_context_flags_restore

Restore selected context flags from a saved value.

```
void ppf_context_flags_restore(
    ppf_context_t *ctx,
    ppf_flags_t flags,
    ppf_flags_t mask
);
```

Result. No return value.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_context_flags_save

Copy the current context flags for later restoration.

```
ppf_flags_t ppf_context_flags_save(const ppf_context_t *ctx);
```

Result. The value described above, with the declared C return type.

ppf_context_flags_set

Set selected exception flags in a context.

```
void ppf_context_flags_set(ppf_context_t *ctx, ppf_flags_t mask);
```

Result. No return value.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_context_init

Initialize an explicit context with default precision, range, rounding, flags, allocator, and scratch arena.

```
int ppf_context_init(ppf_context_t *ctx);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Context and rounding. The context destination must be non-null writable unused storage. This call initializes it; do not overwrite a live context.

ppf_context_init_custom

Initialize a context that uses caller-supplied allocation callbacks.

```
int ppf_context_init_custom(
    ppf_context_t *ctx,
    const ppf_allocator_t *allocator
);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Context and rounding. The context destination must be non-null writable unused storage. This call initializes it; do not overwrite a live context.

ppf_context_set_default_precision

Change the precision used for new default-precision objects in a context.

```
int ppf_context_set_default_precision(
    ppf_context_t *ctx,
    ppf_prec_t precision
);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_context_set_range

Set the minimum and maximum binary exponents for a context.

```
int ppf_context_set_range(
    ppf_context_t *ctx,
    ppf_exp_t emin,
    ppf_exp_t emax
);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_context_set_rounding

Set the context's default rounding mode.

```
int ppf_context_set_rounding(ppf_context_t *ctx, ppf_rnd_t rnd);
```

Result. Returns 0 when the concrete rounding mode is accepted and -1 otherwise; `PPF_RND_DEFAULT` is rejected. Failure indicator: `-1`.

Context and rounding. A null `ctx` selects the calling thread's default context. select a concrete mode from PPF_RNDN through PPF_RNDF; PPF_RND_DEFAULT is rejected rather than resolved.

ppf_default_allocator

Return the library's standard allocator callbacks.

```
ppf_allocator_t ppf_default_allocator(void);
```

Result. The value described above, with the declared C return type.

Ownership. returns allocator callbacks by value.

ppf_default_context

Return the calling thread's default context.

```
ppf_context_t *ppf_default_context(void);
```

Result. A borrowed pointer. Do not free the returned storage. Failure indicator: `NULL where documented`.

Ownership. pointer remains owned by the calling thread and is invalid after thread cleanup.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_default_context_cleanup

Release the calling thread's default-context resources.

```
void ppf_default_context_cleanup(void);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_set_allocflag

Set the allocation-failure flag in the calling thread default context.

```
void ppf_set_allocflag(void);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_thread_cleanup

Release heap-backed state owned by the calling thread. Safe to call repeatedly.

```
void ppf_thread_cleanup(void);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

17.2 Exponent range and exception flags

ppf_check_range

Apply the active exponent range to an already rounded result and update flags and ternary direction.

```
int ppf_check_range(
    ppf_t *x,
    int ternary,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the designated object is mutated in place.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_clear_divby0

Clear the divide-by-zero flag in the thread-local default context.

```
void ppf_clear_divby0(void);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_clear_erangeflag

Clear the range-error flag in the thread-local default context.

```
void ppf_clear_erangeflag(void);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_clear_flags

Clear all exception flags in the thread-local default context.

```
void ppf_clear_flags(void);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_clear_inexflag

Clear the inexact flag in the thread-local default context.

```
void ppf_clear_inexflag(void);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_clear_nanflag

Clear the NaN flag in the thread-local default context.

```
void ppf_clear_nanflag(void);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_clear_overflow

Clear the overflow flag in the thread-local default context.

```
void ppf_clear_overflow(void);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_clear_underflow

Clear the underflow flag in the thread-local default context.

```
void ppf_clear_underflow(void);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_divby0_p

Test the divide-by-zero flag in the thread-local default context.

```
int ppf_divby0_p(void);
```

Result. A predicate result: zero means false and a nonzero value means true.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_erangeflag_p

Test the range-error flag in the thread-local default context.

```
int ppf_erangeflag_p(void);
```

Result. A predicate result: zero means false and a nonzero value means true.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_flags_clear

Clear selected flags in the thread-local default context.

```
void ppf_flags_clear(ppf_flags_t mask);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_flags_restore

Restore selected flags in the thread-local default context.

```
void ppf_flags_restore(ppf_flags_t flags, ppf_flags_t mask);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_flags_save

Save all flags from the thread-local default context.

```
ppf_flags_t ppf_flags_save(void);
```

Result. The value described above, with the declared C return type.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_flags_set

Set selected flags in the thread-local default context.

```
void ppf_flags_set(ppf_flags_t mask);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_flags_test

Return the selected flags currently set in the thread-local default context.

```
ppf_flags_t ppf_flags_test(ppf_flags_t mask);
```

Result. The value described above, with the declared C return type.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_get_emax

Read the maximum exponent of the thread-local default context.

```
ppf_exp_t ppf_get_emax(void);
```

Result. The value described above, with the declared C return type.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_get_emax_max

Return the largest supported maximum exponent setting.

```
ppf_exp_t ppf_get_emax_max(void);
```

Result. The value described above, with the declared C return type.

ppf_get_emax_min

Return the smallest supported maximum exponent setting.

```
ppf_exp_t ppf_get_emax_min(void);
```

Result. The value described above, with the declared C return type.

ppf_get_emin

Read the minimum exponent of the thread-local default context.

```
ppf_exp_t ppf_get_emin(void);
```

Result. The value described above, with the declared C return type.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_get_emin_max

Return the largest supported minimum exponent setting.

```
ppf_exp_t ppf_get_emin_max(void);
```

Result. The value described above, with the declared C return type.

ppf_get_emin_min

Return the smallest supported minimum exponent setting.

```
ppf_exp_t ppf_get_emin_min(void);
```

Result. The value described above, with the declared C return type.

ppf_inexflag_p

Test the inexact flag in the thread-local default context.

```
int ppf_inexflag_p(void);
```

Result. A predicate result: zero means false and a nonzero value means true.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_nanflag_p

Test the NaN flag in the thread-local default context.

```
int ppf_nanflag_p(void);
```

Result. A predicate result: zero means false and a nonzero value means true.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_overflow_p

Test the overflow flag in the thread-local default context.

```
int ppf_overflow_p(void);
```

Result. A predicate result: zero means false and a nonzero value means true.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_set_divby0

Set the divide-by-zero flag in the thread-local default context.

```
void ppf_set_divby0(void);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_set_emax

Set the default context maximum exponent.

```
int ppf_set_emax(ppf_exp_t exponent);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_set_emin

Set the default context minimum exponent.

```
int ppf_set_emin(ppf_exp_t exponent);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_set_erangeflag

Set the range-error flag in the thread-local default context.

```
void ppf_set_erangeflag(void);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_set_inexflag

Set the inexact flag in the thread-local default context.

```
void ppf_set_inexflag(void);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_set_nanflag

Set the NaN flag in the thread-local default context.

```
void ppf_set_nanflag(void);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_set_overflow

Set the overflow flag in the thread-local default context.

```
void ppf_set_overflow(void);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_set_underflow

Set the underflow flag in the thread-local default context.

```
void ppf_set_underflow(void);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_subnormalize

Emulate subnormal or fixed-point rounding at the active minimum exponent.

```
int ppf_subnormalize(
    ppf_t *x,
    int ternary,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the designated object is mutated in place.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_underflow_p

Test the underflow flag in the thread-local default context.

```
int ppf_underflow_p(void);
```

Result. A predicate result: zero means false and a nonzero value means true.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

17.3 Initialization and precision

ppf_clear

Release memory owned by one number.

```
void ppf_clear(ppf_t *x);
```

Result. No return value.

Overlap. the designated object is mutated in place.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_clears

Clear several numbers; terminate the variadic list with `(ppf_t *)0`.

```
void ppf_clears(ppf_t *first, ...);
```

Result. No return value.

Overlap. destination storage satisfies the function's initialization or destruction precondition; Forbidden: using an already initialized object as an initialization destination unless explicitly documented.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_get_default_prec

Return the thread-local default precision.

```
ppf_prec_t ppf_get_default_prec(void);
```

Result. The value described above, with the declared C return type.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_get_prec

Return an object's precision in bits.

```
ppf_prec_t ppf_get_prec(const ppf_t *x);
```

Result. The value described above, with the declared C return type.

ppf_init

Initialize a number at the thread-local default precision.

```
int ppf_init(ppf_t *x);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Overlap. destination storage satisfies the function's initialization or destruction precondition; Forbidden: using an already initialized object as an initialization destination unless explicitly documented.

Allocation and failure. On a valid initialization request, project-allocation failure leaves a safe zero-state object. Invalid arguments can be rejected before zeroing; follow the initializer's preconditions and cleanup lifecycle.

ppf_init2

Initialize a number at an explicit precision.

```
int ppf_init2(ppf_t *x, ppf_prec_t precision);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Overlap. destination storage satisfies the function's initialization or destruction precondition; Forbidden: using an already initialized object as an initialization destination unless explicitly documented.

Allocation and failure. On a valid initialization request, project-allocation failure leaves a safe zero-state object. Invalid arguments can be rejected before zeroing; follow the initializer's preconditions and cleanup lifecycle.

ppf_init2_ctx

Initialize a number at an explicit precision using an explicit context allocator.

```
int ppf_init2_ctx(ppf_t *x, ppf_prec_t precision, ppf_context_t *ctx);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Context and rounding. The supplied context must be non-null and initialized.

Overlap. destination storage satisfies the function's initialization or destruction precondition; Forbidden: using an already initialized object as an initialization destination unless explicitly documented.

Allocation and failure. On a valid initialization request, project-allocation failure leaves a safe zero-state object. Invalid arguments can be rejected before zeroing; follow the initializer's preconditions and cleanup lifecycle.

ppf_inits

Initialize several numbers at the default precision; terminate with `(ppf_t *)0`.

```
int ppf_inits(ppf_t *first, ...);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Overlap. destination storage satisfies the function's initialization or destruction precondition; Forbidden: using an already initialized object as an initialization destination unless explicitly documented.

Allocation and failure. On failure the initialized prefix is cleared and unvisited variadic destinations are untouched; every observed project allocation is tested; libc prefix-tracking realloc remains outside the recoverable project-allocator domain.

ppf_inits2

Initialize several numbers at one precision; terminate the variadic list with `(ppf_t *)0`.

```
int ppf_inits2(ppf_prec_t precision, ppf_t *first, ...);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Overlap. destination storage satisfies the function's initialization or destruction precondition; Forbidden: using an already initialized object as an initialization destination unless explicitly documented.

Allocation and failure. On failure the initialized prefix is cleared and unvisited variadic destinations are untouched; every observed project allocation is tested; libc prefix-tracking realloc remains outside the recoverable project-allocator domain.

ppf_prec_round

Change precision while preserving and rounding the current value.

```
int ppf_prec_round(
    ppf_t *x,
    ppf_prec_t precision,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the designated object is mutated in place.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_reserve

Ensure an object has storage for at least the requested precision.

```
int ppf_reserve(ppf_t *x, ppf_prec_t precision);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Overlap. the designated object is mutated in place.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_set_default_prec

Set the thread-local default precision used by `ppf_init`.

```
int ppf_set_default_prec(ppf_prec_t precision);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_set_prec

Change object precision and leave a deterministic NaN rather than preserving the old value.

```
int ppf_set_prec(ppf_t *x, ppf_prec_t precision);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Overlap. the designated object is mutated in place.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_set_prec_raw

Change the exposed precision field for MPF-compatibility workflows; advanced use only.

```
void ppf_set_prec_raw(ppf_t *x, ppf_prec_t precision);
```

Result. No return value.

Overlap. the designated object is mutated in place.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_swap

Exchange two complete floating objects without numerical rounding.

```
void ppf_swap(ppf_t *a, ppf_t *b);
```

Result. No return value.

Overlap. the designated object is mutated in place.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

17.4 Direct significand access (advanced)

ppf_commit_regular

Validate and commit a directly written regular significand, sign, and exponent.

```
int ppf_commit_regular(
    ppf_t *x,
    int sign,
    ppf_exp_t exponent,
    ppf_context_t *ctx
);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Context and rounding. A null `ctx` selects the calling thread's default context.

Overlap. overlap is governed by the advanced direct-storage preconditions; Forbidden: overlap that violates the documented storage-size, commit, or ownership preconditions.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_limb_count

Return the number of significand limbs currently used by an object.

```
size_t ppf_limb_count(const ppf_t *x);
```

Result. The operation-specific count described above, rather than a numerical ternary. Failure indicator: `0 when the operation cannot produce a count`.

Overlap. overlap is governed by the advanced direct-storage preconditions; Forbidden: overlap that violates the documented storage-size, commit, or ownership preconditions.

ppf_limbs_read

Return read-only access to normalized significand limbs.

```
const mp_limb_t *ppf_limbs_read(const ppf_t *x);
```

Result. A borrowed pointer. Do not free the returned storage. Failure indicator: `NULL where documented`.

Ownership. pointer is invalidated by mutation, reallocation, or clear.

Overlap. overlap is governed by the advanced direct-storage preconditions; Forbidden: overlap that violates the documented storage-size, commit, or ownership preconditions.

ppf_limbs_write

Begin controlled writable access to significand limbs; the caller must later commit a valid regular value.

```
mp_limb_t *ppf_limbs_write(ppf_t *x);
```

Result. A borrowed pointer. Do not free the returned storage. Failure indicator: `NULL where documented`.

Ownership. pointer is writable internal storage and must be followed by a valid commit when representation fields change.

Overlap. overlap is governed by the advanced direct-storage preconditions; Forbidden: overlap that violates the documented storage-size, commit, or ownership preconditions.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

17.5 Special values, signs, and rounding control

ppf_can_round

Ask whether an approximation with a stated error can be safely rounded to a target precision and mode.

```
int ppf_can_round(
    const ppf_t *approximation,
    ppf_exp_t error_bits,
    ppf_rnd_t source_rounding,
    ppf_rnd_t target_rounding,
    ppf_prec_t target_precision
);
```

Result. A predicate result: zero means false and a nonzero value means true.

Context and rounding. source_rounding and target_rounding must both be concrete modes from PPF_RNDN through PPF_RNDF; PPF_RND_DEFAULT in either parameter is rejected with return 0, without consulting the calling thread default context.

ppf_copysign

Copy the magnitude of one value and the sign of another.

```
int ppf_copysign(
    ppf_t *rop,
    const ppf_t *op1,
    const ppf_t *op2,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_get_default_rounding_mode

Return the thread-local default rounding mode.

```
ppf_rnd_t ppf_get_default_rounding_mode(void);
```

Result. The value described above, with the declared C return type.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_inf_p

Test whether a value is infinite.

```
int ppf_inf_p(const ppf_t *x);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_nan_p

Test whether a value is NaN.

```
int ppf_nan_p(const ppf_t *x);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_number_p

Test whether a value is finite, including zero.

```
int ppf_number_p(const ppf_t *x);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_print_rnd_mode

Return the short printable spelling of a rounding mode.

```
const char *ppf_print_rnd_mode(ppf_rnd_t rnd);
```

Result. Returns a static name for a concrete rounding mode from `PPF_RNDN` through `PPF_RNDF`. `PPF_RND_DEFAULT` and invalid enumeration values return NULL. Failure indicator: `NULL`.

Context and rounding. the enumeration is inspected without resolving a context; PPF_RND_DEFAULT and invalid values return NULL.

Ownership. returned string is library-owned static storage.

ppf_regular_p

Test whether a value is a finite nonzero regular number.

```
int ppf_regular_p(const ppf_t *x);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_round_nearest_away_finish

Finish the extra-bit helper used to implement nearest rounding with ties away from zero.

```
int ppf_round_nearest_away_finish(
    ppf_t *rop,
    const ppf_t *extended_result,
    int extended_ternary,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context.

Overlap. destination precision is exactly one bit below the extended source precision; Forbidden: destination and extended source are the same object.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_default_rounding_mode

Set the thread-local default rounding mode.

```
int ppf_set_default_rounding_mode(ppf_rnd_t rnd);
```

Result. Returns 0 when the concrete rounding mode is accepted and -1 otherwise; `PPF_RND_DEFAULT` is rejected. Failure indicator: `-1`.

Context and rounding. select a concrete mode from PPF_RNDN through PPF_RNDF; PPF_RND_DEFAULT is rejected rather than resolved.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_set_inf

Set signed infinity.

```
void ppf_set_inf(ppf_t *x, int sign);
```

Result. No return value.

Overlap. the designated object is mutated in place.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_nan

Set a signed NaN.

```
void ppf_set_nan(ppf_t *x, int sign);
```

Result. No return value.

Overlap. the designated object is mutated in place.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_zero

Set signed zero.

```
void ppf_set_zero(ppf_t *x, int sign);
```

Result. No return value.

Overlap. the designated object is mutated in place.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_setsign

Copy a value while explicitly selecting its sign bit.

```
int ppf_setsign(
    ppf_t *rop,
    const ppf_t *op,
    int signbit,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_sgn

Return the mathematical sign of a non-NaN value and update context state when needed.

```
int ppf_sgn(const ppf_t *x, ppf_context_t *ctx);
```

Result. A sign result for the operation described above.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_signbit

Return the stored sign bit, including for zero, infinity, and NaN.

```
int ppf_signbit(const ppf_t *x);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_zero_p

Test whether a value is zero.

```
int ppf_zero_p(const ppf_t *x);
```

Result. A predicate result: zero means false and a nonzero value means true.

17.6 Assignment and parsing

ppf_set

Copy and round one Polyphase number into another.

```
int ppf_set(ppf_t *rop, const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_d

Round a double into an existing Polyphase number.

```
int ppf_set_d(ppf_t *rop, double value, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_decimal128

Round a decimal128 value into an existing Polyphase number.

```
int ppf_set_decimal128(
    ppf_t *rop,
    ppf_decimal128_t value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

Availability. This declaration depends on `PPF_HAVE_DECIMAL_FLOAT`; test the installed header's capability macro.

ppf_set_decimal64

Round a decimal64 value into an existing Polyphase number.

```
int ppf_set_decimal64(
    ppf_t *rop,
    ppf_decimal64_t value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

Availability. This declaration depends on `PPF_HAVE_DECIMAL_FLOAT`; test the installed header's capability macro.

ppf_set_f

Round a GMP float (`mpf_t`) into an existing Polyphase number.

```
int ppf_set_f(
    ppf_t *rop,
    mpf_srcptr value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_float128

Round a binary128 value into an existing Polyphase number.

```
int ppf_set_float128(
    ppf_t *rop,
    ppf_float128_t value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

Availability. This declaration depends on `PPF_HAVE_FLOAT128`; test the installed header's capability macro.

ppf_set_flt

Round a float into an existing Polyphase number.

```
int ppf_set_flt(ppf_t *rop, float value, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_ld

Round a long double into an existing Polyphase number.

```
int ppf_set_ld(
    ppf_t *rop,
    long double value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_q

Round a GMP rational (`mpq_t`) into an existing Polyphase number.

```
int ppf_set_q(
    ppf_t *rop,
    mpq_srcptr value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_si

Round a signed long into an existing Polyphase number.

```
int ppf_set_si(ppf_t *rop, long value, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_si_2exp

Assign the exact value (signed long) times a binary power, then round once.

```
int ppf_set_si_2exp(
    ppf_t *rop,
    long value,
    ppf_exp_t exponent,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_sj

Round an intmax_t into an existing Polyphase number.

```
int ppf_set_sj(
    ppf_t *rop,
    intmax_t value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_sj_2exp

Assign the exact value (intmax_t) times a binary power, then round once.

```
int ppf_set_sj_2exp(
    ppf_t *rop,
    intmax_t value,
    ppf_exp_t exponent,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_str

Parse an entire string. Return `0` only when the complete non-whitespace input is consumed; return `-1` on parse failure or trailing text.

```
int ppf_set_str(
    ppf_t *rop,
    const char *text,
    int base,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. absolute parsed exponent magnitudes above 2,000,000 are constructed only when the result is already unambiguously outside the configured binary range. Consult the numerical-limits appendix for the complete cap inventory.

ppf_set_ui

Round an unsigned long into an existing Polyphase number.

```
int ppf_set_ui(
    ppf_t *rop,
    unsigned long value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_ui_2exp

Assign the exact value (unsigned long) times a binary power, then round once.

```
int ppf_set_ui_2exp(
    ppf_t *rop,
    unsigned long value,
    ppf_exp_t exponent,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_uj

Round an uintmax_t into an existing Polyphase number.

```
int ppf_set_uj(
    ppf_t *rop,
    uintmax_t value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_uj_2exp

Assign the exact value (uintmax_t) times a binary power, then round once.

```
int ppf_set_uj_2exp(
    ppf_t *rop,
    uintmax_t value,
    ppf_exp_t exponent,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_z

Round a GMP integer (`mpz_t`) into an existing Polyphase number.

```
int ppf_set_z(
    ppf_t *rop,
    mpz_srcptr value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_z_2exp

Assign the exact value (GMP integer (`mpz_t`)) times a binary power, then round once.

```
int ppf_set_z_2exp(
    ppf_t *rop,
    mpz_srcptr value,
    ppf_exp_t exponent,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_strtofr

Parse as much of a string as possible, return the ternary rounding relation, and report the first unparsed character.

```
int ppf_strtofr(
    ppf_t *rop,
    const char *text,
    char **endptr,
    int base,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. absolute parsed exponent magnitudes above 2,000,000 are constructed only when the result is already unambiguously outside the configured binary range. Consult the numerical-limits appendix for the complete cap inventory.

17.7 Initialize-and-set shortcuts

ppf_init_set

Initialize a number and copy-round another Polyphase number into it.

```
int ppf_init_set(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. destination is uninitialized and distinct from the initialized source; Forbidden: destination and source are the same object.

Allocation and failure. Every observed project-allocation failure is tested; initialization failure leaves a safe-zero object, setter failure leaves an initialized clearable object; caller must clear after any initialized outcome.

ppf_init_set_d

Initialize a number and assign from a double in one call.

```
int ppf_init_set_d(
    ppf_t *rop,
    double value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. destination storage satisfies the function's initialization or destruction precondition; Forbidden: using an already initialized object as an initialization destination unless explicitly documented.

Allocation and failure. Every observed project-allocation failure is tested; initialization failure leaves a safe-zero object, setter failure leaves an initialized clearable object; caller must clear after any initialized outcome.

ppf_init_set_f

Initialize a number and assign from a GMP float (`mpf_t`) in one call.

```
int ppf_init_set_f(
    ppf_t *rop,
    mpf_srcptr value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. destination storage satisfies the function's initialization or destruction precondition; Forbidden: using an already initialized object as an initialization destination unless explicitly documented.

Allocation and failure. Every observed project-allocation failure is tested; initialization failure leaves a safe-zero object, setter failure leaves an initialized clearable object; caller must clear after any initialized outcome.

ppf_init_set_flt

Initialize a number and assign from a float in one call.

```
int ppf_init_set_flt(
    ppf_t *rop,
    float value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. destination storage satisfies the function's initialization or destruction precondition; Forbidden: using an already initialized object as an initialization destination unless explicitly documented.

Allocation and failure. Every observed project-allocation failure is tested; initialization failure leaves a safe-zero object, setter failure leaves an initialized clearable object; caller must clear after any initialized outcome.

ppf_init_set_ld

Initialize a number and assign from a long double in one call.

```
int ppf_init_set_ld(
    ppf_t *rop,
    long double value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. destination storage satisfies the function's initialization or destruction precondition; Forbidden: using an already initialized object as an initialization destination unless explicitly documented.

Allocation and failure. Every observed project-allocation failure is tested; initialization failure leaves a safe-zero object, setter failure leaves an initialized clearable object; caller must clear after any initialized outcome.

ppf_init_set_q

Initialize a number and assign from a GMP rational (`mpq_t`) in one call.

```
int ppf_init_set_q(
    ppf_t *rop,
    mpq_srcptr value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. destination storage satisfies the function's initialization or destruction precondition; Forbidden: using an already initialized object as an initialization destination unless explicitly documented.

Allocation and failure. Every observed project-allocation failure is tested; initialization failure leaves a safe-zero object, setter failure leaves an initialized clearable object; caller must clear after any initialized outcome.

ppf_init_set_si

Initialize a number and assign from a signed long in one call.

```
int ppf_init_set_si(
    ppf_t *rop,
    long value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. destination storage satisfies the function's initialization or destruction precondition; Forbidden: using an already initialized object as an initialization destination unless explicitly documented.

Allocation and failure. Every observed project-allocation failure is tested; initialization failure leaves a safe-zero object, setter failure leaves an initialized clearable object; caller must clear after any initialized outcome.

ppf_init_set_sj

Initialize a number and assign from an intmax_t in one call.

```
int ppf_init_set_sj(
    ppf_t *rop,
    intmax_t value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. destination storage satisfies the function's initialization or destruction precondition; Forbidden: using an already initialized object as an initialization destination unless explicitly documented.

Allocation and failure. Every observed project-allocation failure is tested; initialization failure leaves a safe-zero object, setter failure leaves an initialized clearable object; caller must clear after any initialized outcome.

ppf_init_set_str

Initialize a number and assign from a str in one call.

```
int ppf_init_set_str(
    ppf_t *rop,
    const char *text,
    int base,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. destination storage satisfies the function's initialization or destruction precondition; Forbidden: using an already initialized object as an initialization destination unless explicitly documented.

Allocation and failure. Every observed project-allocation failure is tested; initialization failure leaves a safe-zero object, setter failure leaves an initialized clearable object; caller must clear after any initialized outcome.

Resource policy. absolute parsed exponent magnitudes above 2,000,000 are constructed only when the result is already unambiguously outside the configured binary range. Consult the numerical-limits appendix for the complete cap inventory.

ppf_init_set_ui

Initialize a number and assign from an unsigned long in one call.

```
int ppf_init_set_ui(
    ppf_t *rop,
    unsigned long value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. destination storage satisfies the function's initialization or destruction precondition; Forbidden: using an already initialized object as an initialization destination unless explicitly documented.

Allocation and failure. Every observed project-allocation failure is tested; initialization failure leaves a safe-zero object, setter failure leaves an initialized clearable object; caller must clear after any initialized outcome.

ppf_init_set_uj

Initialize a number and assign from an uintmax_t in one call.

```
int ppf_init_set_uj(
    ppf_t *rop,
    uintmax_t value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. destination storage satisfies the function's initialization or destruction precondition; Forbidden: using an already initialized object as an initialization destination unless explicitly documented.

Allocation and failure. Every observed project-allocation failure is tested; initialization failure leaves a safe-zero object, setter failure leaves an initialized clearable object; caller must clear after any initialized outcome.

ppf_init_set_z

Initialize a number and assign from a GMP integer (`mpz_t`) in one call.

```
int ppf_init_set_z(
    ppf_t *rop,
    mpz_srcptr value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. destination storage satisfies the function's initialization or destruction precondition; Forbidden: using an already initialized object as an initialization destination unless explicitly documented.

Allocation and failure. Every observed project-allocation failure is tested; initialization failure leaves a safe-zero object, setter failure leaves an initialized clearable object; caller must clear after any initialized outcome.

17.8 Conversion to C and GMP types

ppf_free_str

Release a string returned by `ppf_get_str` using the correct allocator.

```
void ppf_free_str(char *text);
```

Result. No return value.

Ownership. releases a string allocated by ppf_get_str.

ppf_frexp

Split a value into a Polyphase mantissa and binary exponent.

```
int ppf_frexp(
    ppf_exp_t *exponent,
    ppf_t *mantissa,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_get_d

Convert a Polyphase value to double using the requested rounding mode.

```
double ppf_get_d(const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. The value described above, with the declared C return type.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

ppf_get_d_2exp

Return a `double` mantissa and store a separate binary exponent.

```
double ppf_get_d_2exp(
    long *exponent,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. The value described above, with the declared C return type.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

ppf_get_decimal128

Convert a Polyphase value to decimal128 value using the requested rounding mode.

```
ppf_decimal128_t ppf_get_decimal128(
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. The value described above, with the declared C return type.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Availability. This declaration depends on `PPF_HAVE_DECIMAL_FLOAT`; test the installed header's capability macro.

ppf_get_decimal64

Convert a Polyphase value to decimal64 value using the requested rounding mode.

```
ppf_decimal64_t ppf_get_decimal64(
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. The value described above, with the declared C return type.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Availability. This declaration depends on `PPF_HAVE_DECIMAL_FLOAT`; test the installed header's capability macro.

ppf_get_f

Round a value into an initialized GMP float.

```
int ppf_get_f(
    mpf_ptr value,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. Returns the numerical ternary for conversion to the GMP floating-point destination. Test for `PPF_TERNARY_ERROR` before interpreting its sign. NaN maps to zero with ERANGE and return 0; infinity maps to a finite GMP value with ERANGE and returns +1 for negative infinity or -1 for positive infinity.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

ppf_get_float128

Convert a Polyphase value to binary128 value using the requested rounding mode.

```
ppf_float128_t ppf_get_float128(
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. The value described above, with the declared C return type.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Availability. This declaration depends on `PPF_HAVE_FLOAT128`; test the installed header's capability macro.

ppf_get_flt

Convert a Polyphase value to float using the requested rounding mode.

```
float ppf_get_flt(const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. The value described above, with the declared C return type.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

ppf_get_ld

Convert a Polyphase value to long double using the requested rounding mode.

```
long double ppf_get_ld(const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. The value described above, with the declared C return type.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

ppf_get_ld_2exp

Return a `long double` mantissa and store a separate binary exponent.

```
long double ppf_get_ld_2exp(
    long *exponent,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. The value described above, with the declared C return type.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

ppf_get_q

Export a finite value exactly as a GMP rational.

```
void ppf_get_q(mpq_ptr value, const ppf_t *op, ppf_context_t *ctx);
```

Result. No return value.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_get_si

Convert a Polyphase value to signed long using the requested rounding mode.

```
long ppf_get_si(const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. The value described above, with the declared C return type.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

ppf_get_sj

Convert a Polyphase value to intmax_t using the requested rounding mode.

```
intmax_t ppf_get_sj(const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. The value described above, with the declared C return type.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

ppf_get_str

Format a significand into a supplied or library-allocated buffer and return its radix exponent separately.

```
char *ppf_get_str(
    char *buffer,
    ppf_exp_t *exponent,
    int base,
    size_t digits,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A pointer with the ownership conditions below. Failure indicator: `NULL`.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Ownership. when the caller supplies no buffer, the returned string retains the resolved context allocator provenance and must be released only with ppf_free_str.

Allocation and failure. On unrecovered project-allocation failure, returns NULL and releases temporary project-owned storage; the source number is unchanged; recovered fallback allocation failures may still yield a valid string.

ppf_get_str_ndigits

Return the number of base-`b` digits normally needed to print an object at its precision.

```
size_t ppf_get_str_ndigits(int base, ppf_prec_t precision);
```

Result. The operation-specific count described above, rather than a numerical ternary. Failure indicator: `0 when the operation cannot produce a count.`

ppf_get_ui

Convert a Polyphase value to unsigned long using the requested rounding mode.

```
unsigned long ppf_get_ui(
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. The value described above, with the declared C return type.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

ppf_get_uj

Convert a Polyphase value to uintmax_t using the requested rounding mode.

```
uintmax_t ppf_get_uj(const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. The value described above, with the declared C return type.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

ppf_get_z

Round a value to a GMP integer.

```
int ppf_get_z(
    mpz_ptr value,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. Returns the numerical ternary for the GMP integer written to rop. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

ppf_get_z_2exp

Export the exact integer significand and binary exponent of a finite value.

```
ppf_exp_t ppf_get_z_2exp(mpz_ptr value, const ppf_t *op);
```

Result. The value described above, with the declared C return type.

17.9 Text I/O, PPFIF interchange, formatting, and fit tests

ppf_asprintf

Allocate a formatted string with `malloc`; release the returned buffer with `free`.

```
int ppf_asprintf(char **text, const char *format, ...);
```

Result. A formatting count; a negative result reports failure.

Ownership. on success *text is allocated with the process allocator and must be released with free.

Allocation and failure. Sets *text to NULL before formatting; failure releases temporary storage and leaves* text NULL, rather than preserving its prior pointer; %n writes already performed while formatting are not rolled back.

ppf_fits_intmax_p

Test whether the value fits in intmax_t under the library fit rules.

```
int ppf_fits_intmax_p(const ppf_t *op, ppf_rnd_t rnd);
```

Result. A predicate result: zero means false and a nonzero value means true.

Context and rounding. PPF_RND_DEFAULT resolves to PPF_RNDN through a newly initialized private context, independently of the calling thread default rounding mode.

ppf_fits_sint_p

Test whether the value fits in signed int under the library fit rules.

```
int ppf_fits_sint_p(const ppf_t *op, ppf_rnd_t rnd);
```

Result. A predicate result: zero means false and a nonzero value means true.

Context and rounding. PPF_RND_DEFAULT resolves to PPF_RNDN through a newly initialized private context, independently of the calling thread default rounding mode.

ppf_fits_slong_p

Test whether the value fits in signed long under the library fit rules.

```
int ppf_fits_slong_p(const ppf_t *op, ppf_rnd_t rnd);
```

Result. A predicate result: zero means false and a nonzero value means true.

Context and rounding. PPF_RND_DEFAULT resolves to PPF_RNDN through a newly initialized private context, independently of the calling thread default rounding mode.

ppf_fits_sshort_p

Test whether the value fits in signed short under the library fit rules.

```
int ppf_fits_sshort_p(const ppf_t *op, ppf_rnd_t rnd);
```

Result. A predicate result: zero means false and a nonzero value means true.

Context and rounding. PPF_RND_DEFAULT resolves to PPF_RNDN through a newly initialized private context, independently of the calling thread default rounding mode.

ppf_fits_uint_p

Test whether the value fits in unsigned int under the library fit rules.

```
int ppf_fits_uint_p(const ppf_t *op, ppf_rnd_t rnd);
```

Result. A predicate result: zero means false and a nonzero value means true.

Context and rounding. PPF_RND_DEFAULT resolves to PPF_RNDN through a newly initialized private context, independently of the calling thread default rounding mode.

ppf_fits_uintmax_p

Test whether the value fits in uintmax_t under the library fit rules.

```
int ppf_fits_uintmax_p(const ppf_t *op, ppf_rnd_t rnd);
```

Result. A predicate result: zero means false and a nonzero value means true.

Context and rounding. PPF_RND_DEFAULT resolves to PPF_RNDN through a newly initialized private context, independently of the calling thread default rounding mode.

ppf_fits_ulong_p

Test whether the value fits in unsigned long under the library fit rules.

```
int ppf_fits_ulong_p(const ppf_t *op, ppf_rnd_t rnd);
```

Result. A predicate result: zero means false and a nonzero value means true.

Context and rounding. PPF_RND_DEFAULT resolves to PPF_RNDN through a newly initialized private context, independently of the calling thread default rounding mode.

ppf_fits_ushort_p

Test whether the value fits in unsigned short under the library fit rules.

```
int ppf_fits_ushort_p(const ppf_t *op, ppf_rnd_t rnd);
```

Result. A predicate result: zero means false and a nonzero value means true.

Context and rounding. PPF_RND_DEFAULT resolves to PPF_RNDN through a newly initialized private context, independently of the calling thread default rounding mode.

ppf_fpif_export

Write the endian-independent `PPFIF01` binary interchange representation.

```
int ppf_fpif_export(FILE *stream, const ppf_t *op);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Allocation and failure. The complete record is staged before its first stream write; project-allocation failure leaves the stream and source number unchanged, but a later stream write failure may leave a partial record.

ppf_fpif_import

Transactionally import a `PPFIF01` record; malformed input leaves the destination unchanged.

```
int ppf_fpif_import(ppf_t *op, FILE *stream, ppf_context_t *ctx);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Context and rounding. A null `ctx` selects the calling thread's default context.

Allocation and failure. Unrecovered project-allocation failure preserves the numerical destination and releases temporary project-owned storage; bytes already consumed from the input stream are not restored.

Resource policy. default 64 MiB cap independently bounds imported payload and rounded significand storage (536870912 ordinary precision bits on GMP64); `PPF_FPIF_MAX_IMPORT_BYTES` may configure another positive cap; initial staging is at most 4096 bytes, growing only after reserved bytes arrive; range rejection returns -1, raises ERANGE and preserves the destination. Consult the numerical-limits appendix for the complete cap inventory.

ppf_fprintf

Formatted output to a stream with C, GMP, precision, and Polyphase conversions.

```
int ppf_fprintf(FILE *stream, const char *format, ...);
```

Result. A formatting count; a negative result reports failure.

Allocation and failure. Formatting completes in temporary storage before stream output; unrecovered project-allocation failure during formatting leaves the stream unchanged; %n writes already performed are not rolled back, and a later stream write failure may leave partial output.

ppf_inp_str

Read one whitespace-delimited number from a stream; a null stream means standard input.

```
size_t ppf_inp_str(
    ppf_t *rop,
    FILE *stream,
    int base,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. The operation-specific count described above, rather than a numerical ternary. Failure indicator: `0 when the operation cannot produce a count.`

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Unrecovered project-allocation failure preserves the numerical destination and releases temporary project-owned storage; bytes already consumed from the input stream are not restored.

ppf_out_str

Write a value in an arbitrary base to a stream; a null stream means standard output.

```
size_t ppf_out_str(
    FILE *stream,
    int base,
    size_t digits,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. The operation-specific count described above, rather than a numerical ternary. Failure indicator: `0 when the operation cannot produce a count`.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. Unrecovered project-allocation failure during text construction occurs before stream output; the source number is unchanged; a later stream write failure may leave a partial record and is not rolled back.

ppf_printf

Formatted output to standard output.

```
int ppf_printf(const char *format, ...);
```

Result. A formatting count; a negative result reports failure.

Allocation and failure. Formatting completes in temporary storage before stream output; unrecovered project-allocation failure during formatting leaves the stream unchanged; %n writes already performed are not rolled back, and a later stream write failure may leave partial output.

ppf_snprintf

Length-bounded formatted output.

```
int ppf_snprintf(char *buffer, size_t size, const char *format, ...);
```

Result. A formatting count; a negative result reports failure.

Allocation and failure. Formatting completes in temporary storage before the character buffer is copied; unrecovered project-allocation failure leaves that buffer unchanged; %n writes already performed while formatting are not rolled back.

ppf_sprintf

Format into a caller-provided buffer without a length limit; use only when the buffer is known to be large enough.

```
int ppf_sprintf(char *buffer, const char *format, ...);
```

Result. A formatting count; a negative result reports failure.

Allocation and failure. Formatting completes in temporary storage before the character buffer is copied; unrecovered project-allocation failure leaves that buffer unchanged; %n writes already performed while formatting are not rolled back.

ppf_vasprintf

`va_list` form of `ppf_asprintf`; release the returned buffer with `free`.

```
int ppf_vasprintf(char **text, const char *format, va_list ap);
```

Result. A formatting count; a negative result reports failure.

Ownership. on success *text is allocated with the process allocator and must be released with free.

Allocation and failure. Sets *text to NULL before formatting; failure releases temporary storage and leaves* text NULL, rather than preserving its prior pointer; %n writes already performed while formatting are not rolled back.

ppf_vfprintf

`va_list` form of `ppf_fprintf`.

```
int ppf_vfprintf(FILE *stream, const char *format, va_list ap);
```

Result. A formatting count; a negative result reports failure.

Allocation and failure. Formatting completes in temporary storage before stream output; unrecovered project-allocation failure during formatting leaves the stream unchanged; %n writes already performed are not rolled back, and a later stream write failure may leave partial output.

ppf_vprintf

`va_list` form of `ppf_printf`.

```
int ppf_vprintf(const char *format, va_list ap);
```

Result. A formatting count; a negative result reports failure.

Allocation and failure. Formatting completes in temporary storage before stream output; unrecovered project-allocation failure during formatting leaves the stream unchanged;

%n writes already performed are not rolled back, and a later stream write failure may leave partial output.

ppf_vsnprintf

`va_list` form of `ppf_snprintf`.

```
int ppf_vsnprintf(
    char *buffer,
    size_t size,
    const char *format,
    va_list ap
);
```

Result. A formatting count; a negative result reports failure.

Allocation and failure. Formatting completes in temporary storage before the character buffer is copied; unrecovered project-allocation failure leaves that buffer unchanged; %n writes already performed while formatting are not rolled back.

ppf_vsprintf

`va_list` form of `ppf_sprintf`.

```
int ppf_vsprintf(char *buffer, const char *format, va_list ap);
```

Result. A formatting count; a negative result reports failure.

Allocation and failure. Formatting completes in temporary storage before the character buffer is copied; unrecovered project-allocation failure leaves that buffer unchanged; %n writes already performed while formatting are not rolled back.

17.10 Comparison, ordering, sign, and absolute value

ppf_abs

Take an absolute value with requested destination rounding.

```
int ppf_abs(ppf_t *rop, const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_cmp_d

Compare with a double without rounding the Polyphase operand.

```
int ppf_cmp_d(const ppf_t *op, double value, ppf_context_t *ctx);
```

Result. An ordering result. NaNs and unordered comparisons require the operation's documented handling.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_cmp_f

Compare with a GMP float (`mpf_t`) without rounding the Polyphase operand.

```
int ppf_cmp_f(const ppf_t *op, mpf_srcptr value, ppf_context_t *ctx);
```

Result. An ordering result. NaNs and unordered comparisons require the operation's documented handling.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_cmp_ld

Compare with a long double without rounding the Polyphase operand.

```
int ppf_cmp_ld(const ppf_t *op, long double value, ppf_context_t *ctx);
```

Result. An ordering result. NaNs and unordered comparisons require the operation's documented handling.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_cmp_q

Compare with a GMP rational (`mpq_t`) without rounding the Polyphase operand.

```
int ppf_cmp_q(const ppf_t *op, mpq_srcptr value, ppf_context_t *ctx);
```

Result. An ordering result. NaNs and unordered comparisons require the operation's documented handling.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_cmp_si

Compare with a signed long without rounding the Polyphase operand.

```
int ppf_cmp_si(const ppf_t *op, long value, ppf_context_t *ctx);
```

Result. An ordering result. NaNs and unordered comparisons require the operation's documented handling.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_cmp_si_2exp

Compare with an exact signed long times a binary power.

```
int ppf_cmp_si_2exp(
    const ppf_t *op,
    long value,
    ppf_exp_t exponent,
    ppf_context_t *ctx
);
```

Result. An ordering result. NaNs and unordered comparisons require the operation's documented handling.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_cmp_ui

Compare with a unsigned long without rounding the Polyphase operand.

```
int ppf_cmp_ui(const ppf_t *op, unsigned long value, ppf_context_t *ctx);
```

Result. An ordering result. NaNs and unordered comparisons require the operation's documented handling.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_cmp_ui_2exp

Compare with an exact unsigned long times a binary power.

```
int ppf_cmp_ui_2exp(
    const ppf_t *op,
    unsigned long value,
    ppf_exp_t exponent,
    ppf_context_t *ctx
);
```

Result. An ordering result. NaNs and unordered comparisons require the operation's documented handling.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_cmp_z

Compare with a GMP integer (`mpz_t`) without rounding the Polyphase operand.

```
int ppf_cmp_z(const ppf_t *op, mpz_srcptr value, ppf_context_t *ctx);
```

Result. An ordering result. NaNs and unordered comparisons require the operation's documented handling.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_cmpabs

Compare absolute values without rounding either input.

```
int ppf_cmpabs(const ppf_t *a, const ppf_t *b, ppf_context_t *ctx);
```

Result. An ordering result. NaNs and unordered comparisons require the operation's documented handling.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_cmpabs_ui

Compare a value's absolute magnitude with an unsigned long.

```
int ppf_cmpabs_ui(const ppf_t *op, unsigned long value, ppf_context_t *ctx);
```

Result. An ordering result. NaNs and unordered comparisons require the operation's documented handling.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_greater_p

IEEE-style greater-than predicate.

```
int ppf_greater_p(const ppf_t *a, const ppf_t *b);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_greaterequal_p

IEEE-style greater-than-or-equal predicate.

```
int ppf_greaterequal_p(const ppf_t *a, const ppf_t *b);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_less_p

IEEE-style less-than predicate.

```
int ppf_less_p(const ppf_t *a, const ppf_t *b);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_lessequal_p

IEEE-style less-than-or-equal predicate.

```
int ppf_lessequal_p(const ppf_t *a, const ppf_t *b);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_lessgreater_p

Test whether two values are ordered and unequal.

```
int ppf_lessgreater_p(const ppf_t *a, const ppf_t *b);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_neg

Negate a value with requested destination rounding.

```
int ppf_neg(ppf_t *rop, const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_total_order_p

Apply a total ordering that includes signed zeros, infinities, and signed NaNs.

```
int ppf_total_order_p(const ppf_t *a, const ppf_t *b);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_unordered_p

Test whether a comparison is unordered because at least one operand is NaN.

```
int ppf_unordered_p(const ppf_t *a, const ppf_t *b);
```

Result. A predicate result: zero means false and a nonzero value means true.

17.11 Integer rounding, fractional parts, and remainders

ppf_ceil

Round to an integral value toward positive infinity.

```
int ppf_ceil(ppf_t *rop, const ppf_t *op, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. the function name selects the integer-rounding rule.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_floor

Round to an integral value toward negative infinity.

```
int ppf_floor(ppf_t *rop, const ppf_t *op, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. the function name selects the integer-rounding rule.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_fmod

Compute the truncating-quotient remainder.

```
int ppf_fmod(
    ppf_t *rop,
    const ppf_t *x,
    const ppf_t *y,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_fmod_ui

Compute the truncating-quotient remainder by an unsigned long divisor.

```
int ppf_fmod_ui(
    ppf_t *rop,
    const ppf_t *x,
    unsigned long y,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_fmodquo

Compute `fmod` and return selected low quotient bits.

```
int ppf_fmodquo(
    ppf_t *rop,
    long *quotient_bits,
    const ppf_t *x,
    const ppf_t *y,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_frac

Return the signed fractional part.

```
int ppf_frac(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_integer_p

Test whether a finite value is mathematically integral.

```
int ppf_integer_p(const ppf_t *op);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_min_prec

Return the minimum precision needed to represent the current value exactly.

```
ppf_prec_t ppf_min_prec(const ppf_t *op);
```

Result. The value described above, with the declared C return type.

ppf_modf

Split one value into integral and fractional Polyphase outputs.

```
int ppf_modf(
    ppf_t *integer_part,
    ppf_t *fractional_part,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. Test for `PPF_TERNARY_ERROR` before decoding. Otherwise the return is code(first) + 4 * code(second), where 0 means exact, 1 means above the exact result, and 2 means below it: the integral output first and the fractional output second. The sign of the packed return is not a numerical ternary. Failure indicator: `PPF_TERNARY_ERROR`.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. either floating output may alias the floating input individually; Forbidden: the two floating outputs must not be the same object.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_remainder

Compute the IEEE nearest-integer-quotient remainder.

```
int ppf_remainder(
    ppf_t *rop,
    const ppf_t *x,
    const ppf_t *y,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_remquo

Compute the IEEE remainder and selected low quotient bits.

```
int ppf_remquo(
    ppf_t *rop,
    long *quotient_bits,
    const ppf_t *x,
    const ppf_t *y,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_rint

Round to an integral Polyphase value using a selected rounding mode.

```
int ppf_rint(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_rint_ceil

`ppf_rint` convenience form using upward rounding.

```
int ppf_rint_ceil(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_rint_floor

`ppf_rint` convenience form using downward rounding.

```
int ppf_rint_floor(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_rint_round

`ppf_rint` convenience form using nearest with ties away from zero.

```
int ppf_rint_round(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_rint_roundeven

`ppf_rint` convenience form using nearest with ties to even.

```
int ppf_rint_roundeven(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_rint_trunc

`ppf_rint` convenience form using truncation.

```
int ppf_rint_trunc(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_round

Round to nearest integer with halfway cases away from zero.

```
int ppf_round(ppf_t *rop, const ppf_t *op, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. the function name selects the integer-rounding rule.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_roundeven

Round to nearest integer with halfway cases to even.

```
int ppf_roundeven(ppf_t *rop, const ppf_t *op, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. the function name selects the integer-rounding rule.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_trunc

Round to an integral value toward zero.

```
int ppf_trunc(ppf_t *rop, const ppf_t *op, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. the function name selects the integer-rounding rule.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

17.12 Neighboring values, min/max, and exponent access

ppf_dim

Compute the positive difference `max(a-b, 0)`.

```
int ppf_dim(
    ppf_t *rop,
    const ppf_t *a,
    const ppf_t *b,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_get_exp

Return the stored binary exponent of a regular value.

```
ppf_exp_t ppf_get_exp(const ppf_t *x, ppf_context_t *ctx);
```

Result. The value described above, with the declared C return type.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_max

Return the selected maximum of two values with defined NaN and signed-zero behavior.

```
int ppf_max(
    ppf_t *rop,
    const ppf_t *a,
    const ppf_t *b,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_min

Return the selected minimum of two values with defined NaN and signed-zero behavior.

```
int ppf_min(
    ppf_t *rop,
    const ppf_t *a,
    const ppf_t *b,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_nextabove

Move one representable step toward positive infinity.

```
void ppf_nextabove(ppf_t *x, ppf_context_t *ctx);
```

Result. No return value.

Context and rounding. A null `ctx` selects the calling thread's default context.

Overlap. the designated object is mutated in place.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_nextbelow

Move one representable step toward negative infinity.

```
void ppf_nextbelow(ppf_t *x, ppf_context_t *ctx);
```

Result. No return value.

Context and rounding. A null `ctx` selects the calling thread's default context.

Overlap. the designated object is mutated in place.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_nexttoward

Move one representable step from one value toward another.

```
void ppf_nexttoward(ppf_t *x, const ppf_t *y, ppf_context_t *ctx);
```

Result. No return value.

Context and rounding. A null `ctx` selects the calling thread's default context.

Overlap. the designated object is mutated in place.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_set_exp

Replace the stored binary exponent after range validation.

```
int ppf_set_exp(ppf_t *x, ppf_exp_t exponent, ppf_context_t *ctx);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Context and rounding. A null `ctx` selects the calling thread's default context.

Overlap. the designated object is mutated in place.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

17.13 Random values, build information, cache control, and custom storage

ppf_buildopt_decimal_p

Report whether decimal64/decimal128 conversion support was compiled.

```
int ppf_buildopt_decimal_p(void);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_buildopt_float128_p

Report whether binary128 conversion support was compiled.

```
int ppf_buildopt_float128_p(void);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_buildopt_gmpinternals_p

Report whether optional GMP-internal integration is active.

```
int ppf_buildopt_gmpinternals_p(void);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_buildopt_sharedcache_p

Report whether shared-cache support is active.

```
int ppf_buildopt_sharedcache_p(void);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_buildopt_tls_p

Report whether thread-local storage support is active.

```
int ppf_buildopt_tls_p(void);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_buildopt_tune_case

Return the build tuning-case description.

```
const char *ppf_buildopt_tune_case(void);
```

Result. A borrowed pointer. Do not free the returned storage. Failure indicator: `NULL where documented`.

Ownership. returned string is library-owned static storage.

ppf_custom_get_exp

Return the external object exponent.

```
ppf_exp_t ppf_custom_get_exp(const ppf_t *x);
```

Result. The value described above, with the declared C return type.

Overlap. overlap is governed by the advanced direct-storage preconditions; Forbidden: overlap that violates the documented storage-size, commit, or ownership preconditions.

ppf_custom_get_kind

Return the public kind code of an externally stored value.

```
int ppf_custom_get_kind(const ppf_t *x);
```

Result. The value described above, with the declared C return type.

Overlap. overlap is governed by the advanced direct-storage preconditions; Forbidden: overlap that violates the documented storage-size, commit, or ownership preconditions.

ppf_custom_get_significand

Return the address of an external object's significand storage.

```
void *ppf_custom_get_significand(const ppf_t *x);
```

Result. A borrowed pointer. Do not free the returned storage. Failure indicator: `NULL where documented`.

Ownership. returns the currently attached application-owned significand storage.

Overlap. overlap is governed by the advanced direct-storage preconditions; Forbidden: overlap that violates the documented storage-size, commit, or ownership preconditions.

ppf_custom_get_size

Return the application-owned byte count required for an externally backed object.

```
size_t ppf_custom_get_size(ppf_prec_t precision);
```

Result. The operation-specific count described above, rather than a numerical ternary. Failure indicator: `0 when the operation cannot produce a count`.

Overlap. overlap is governed by the advanced direct-storage preconditions; Forbidden: overlap that violates the documented storage-size, commit, or ownership preconditions.

ppf_custom_init

Initialize raw external storage for a chosen precision.

```
void ppf_custom_init(void *significand, ppf_prec_t precision);
```

Result. No return value.

Ownership. initializes caller-provided storage without taking ownership.

Overlap. overlap is governed by the advanced direct-storage preconditions; Forbidden: overlap that violates the documented storage-size, commit, or ownership preconditions.

ppf_custom_init_set

Attach a `ppf_t` view to application-owned storage and set its initial kind, exponent, and precision.

```
int ppf_custom_init_set(
    ppf_t *x,
    int kind,
    ppf_exp_t exponent,
    ppf_prec_t precision,
    void *significand
);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Ownership. attaches caller-provided storage without taking ownership.

Overlap. overlap is governed by the advanced direct-storage preconditions; Forbidden: overlap that violates the documented storage-size, commit, or ownership preconditions.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_custom_move

Move an existing value into application-owned storage.

```
void ppf_custom_move(ppf_t *x, void *new_position);
```

Result. No return value.

Ownership. moves the object view to replacement caller-owned storage.

Overlap. overlap is governed by the advanced direct-storage preconditions; Forbidden: overlap that violates the documented storage-size, commit, or ownership preconditions.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_div_2exp

Divide by an unsigned binary power for MPF-style compatibility.

```
int ppf_div_2exp(
    ppf_t *rop,
    const ppf_t *op,
    unsigned long exponent,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_eq

Test equality to a requested number of leading bits for MPF-style compatibility.

```
int ppf_eq(const ppf_t *op1, const ppf_t *op2, unsigned long bits);
```

Result. Returns nonzero when the operands agree to the requested number of leading bits, and 0 otherwise.

ppf_erandom

Generate an exponentially distributed random value with certified rounding.

```
int ppf_erandom(
    ppf_t *rop,
    gmp_randstate_t state,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. adaptive random-distribution rounding uses at most 12 precision-escalation attempts. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. Application locking is required for shared mutable use.

ppf_free_cache

Release the calling thread's local caches.

```
void ppf_free_cache(void);
```

Result. No return value.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_free_cache2

Release selected local and/or global caches.

```
void ppf_free_cache2(ppf_free_cache_t way);
```

Result. No return value.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_free_pool

Release reusable memory-pool resources.

```
void ppf_free_pool(void);
```

Result. No return value.

Concurrency. This operation concerns the calling thread's state; it is not a mechanism for sharing a mutable context.

ppf_get_patches

Return the patch-level description string.

```
const char *ppf_get_patches(void);
```

Result. A borrowed pointer. Do not free the returned storage. Failure indicator: `NULL where documented`.

Ownership. returned string is library-owned static storage.

ppf_get_version

Return the library version string.

```
const char *ppf_get_version(void);
```

Result. A borrowed pointer. Do not free the returned storage. Failure indicator: `NULL where documented`.

Ownership. returned string is library-owned static storage.

ppf_grandom

Generate two normally distributed values as a pair.

```
int ppf_grandom(
    ppf_t *rop1,
    ppf_t *rop2,
    gmp_randstate_t state,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. Test for `PPF_TERNARY_ERROR` before decoding. Otherwise the return is code(first) + 4 * code(second), where 0 means exact, 1 means above the exact result, and 2 means below it: rop1 first and rop2 second; when rop2 is NULL its component code is 0. The sign of the packed return is not a numerical ternary. Failure indicator: `PPF_TERNARY_ERROR`.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the optional second output may be NULL; Forbidden: the two floating outputs must not be the same object.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. adaptive random-distribution rounding uses at most 12 precision-escalation attempts. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. Application locking is required for shared mutable use.

ppf_mp_memory_cleanup

Run the library's broad multiprecision memory cleanup.

```
int ppf_mp_memory_cleanup(void);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

ppf_mul_2exp

Multiply by an unsigned binary power for MPF-style compatibility.

```
int ppf_mul_2exp(
    ppf_t *rop,
    const ppf_t *op,
    unsigned long exponent,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. Reviewed valid initialized-storage paths do not call the project allocator; existing capacity, direct-storage and ownership preconditions apply; GMP allocation is outside this claim.

ppf_nrandom

Generate a normally distributed random value with certified rounding.

```
int ppf_nrandom(
    ppf_t *rop,
    gmp_randstate_t state,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. adaptive random-distribution rounding uses at most 12 precision-escalation attempts. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. Application locking is required for shared mutable use.

ppf_reldiff

Compute a relative difference for MPF-style compatibility.

```
void ppf_reldiff(
    ppf_t *rop,
    const ppf_t *op1,
    const ppf_t *op2,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. No return value.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_urandom

Generate a uniformly distributed floating value in the supported unit interval convention.

```
int ppf_urandom(
    ppf_t *rop,
    gmp_randstate_t state,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. adaptive random-distribution rounding uses at most 12 precision-escalation attempts. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. Application locking is required for shared mutable use.

ppf_urandomb

Generate a uniform random dyadic number from GMP random bits.

```
int ppf_urandomb(ppf_t *rop, gmp_randstate_t state);
```

Result. Returns 0 on success and 1 on failure. The generated significand is exact at the destination precision; the return is not a numerical ternary. Failure indicator: `1`.

Context and rounding. the generated significand is exact at the destination precision.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Concurrency. Application locking is required for shared mutable use.

17.14 Build and ABI information

ppf_abi_contract_sha256

Return the SHA-256 identifier of the normalized public ABI contract.

```
const char *ppf_abi_contract_sha256(void);
```

Result. A borrowed pointer. Do not free the returned storage. Failure indicator: `NULL where documented`.

Ownership. returned string is library-owned static storage.

ppf_abi_epoch

Return the governed public ABI epoch.

```
uint32_t ppf_abi_epoch(void);
```

Result. The value described above, with the declared C return type.

ppf_abi_info_size

Return the size of the current ABI information structure.

```
size_t ppf_abi_info_size(void);
```

Result. The operation-specific count described above, rather than a numerical ternary. Failure indicator: `0 when the operation cannot produce a count`.

ppf_abi_revision

Return the additive revision within the current ABI epoch.

```
uint32_t ppf_abi_revision(void);
```

Result. The value described above, with the declared C return type.

ppf_abi_tag

Return the stable textual tag for the current ABI epoch.

```
const char *ppf_abi_tag(void);
```

Result. A borrowed pointer. Do not free the returned storage. Failure indicator: `NULL where documented`.

Ownership. returned string is library-owned static storage.

ppf_buildopt_thread_cleanup_p

Report whether automatic POSIX thread-exit cleanup is available in this build.

```
int ppf_buildopt_thread_cleanup_p(void);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_get_abi_info

Copy a size-bounded runtime description of the active public ABI into caller storage.

```
int ppf_get_abi_info(ppf_abi_info_t *info, size_t info_size);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

17.15 Arithmetic and product planning

ppf_add

Add two Polyphase values with one final rounding.

```
int ppf_add(
    ppf_t *rop,
    const ppf_t *a,
    const ppf_t *b,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_add_d

Add a Polyphase operand and double with one final rounding.

```
int ppf_add_d(
    ppf_t *rop,
    const ppf_t *op,
    double value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_add_q

Add a Polyphase operand and GMP rational (`mpq_t`) with one final rounding.

```
int ppf_add_q(
    ppf_t *rop,
    const ppf_t *op,
    mpq_srcptr value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_add_si

Add a Polyphase operand and signed long with one final rounding.

```
int ppf_add_si(
    ppf_t *rop,
    const ppf_t *op,
    long value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_add_ui

Add a Polyphase operand and unsigned long with one final rounding.

```
int ppf_add_ui(
    ppf_t *rop,
    const ppf_t *op,
    unsigned long value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_add_z

Add a Polyphase operand and GMP integer (`mpz_t`) with one final rounding.

```
int ppf_add_z(
    ppf_t *rop,
    const ppf_t *op,
    mpz_srcptr value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_d_div

Compute double / Polyphase operand with one final rounding.

```
int ppf_d_div(
    ppf_t *rop,
    double value,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_d_sub

Compute double - Polyphase operand with one final rounding.

```
int ppf_d_sub(
    ppf_t *rop,
    double value,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_div

Divide two Polyphase values with one final rounding.

```
int ppf_div(
    ppf_t *rop,
    const ppf_t *a,
    const ppf_t *b,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_div_d

Compute Polyphase operand divided by double with one final rounding.

```
int ppf_div_d(
    ppf_t *rop,
    const ppf_t *op,
    double value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_div_q

Compute Polyphase operand divided by GMP rational (`mpq_t`) with one final rounding.

```
int ppf_div_q(
    ppf_t *rop,
    const ppf_t *op,
    mpq_srcptr value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_div_si

Compute Polyphase operand divided by signed long with one final rounding.

```
int ppf_div_si(
    ppf_t *rop,
    const ppf_t *op,
    long value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_div_ui

Compute Polyphase operand divided by unsigned long with one final rounding.

```
int ppf_div_ui(
    ppf_t *rop,
    const ppf_t *op,
    unsigned long value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_div_z

Compute Polyphase operand divided by GMP integer (`mpz_t`) with one final rounding.

```
int ppf_div_z(
    ppf_t *rop,
    const ppf_t *op,
    mpz_srcptr value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_mul

Multiply two Polyphase values with one final rounding.

```
int ppf_mul(
    ppf_t *rop,
    const ppf_t *a,
    const ppf_t *b,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_mul_algorithm_threads

Multiply while explicitly forcing FULL, rank-3, rank-5, or window product selection.

```
int ppf_mul_algorithm_threads(
    ppf_t *rop,
    const ppf_t *a,
    const ppf_t *b,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_product_algorithm_t algorithm,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_mul_d

Multiply a Polyphase operand and double with one final rounding.

```
int ppf_mul_d(
    ppf_t *rop,
    const ppf_t *op,
    double value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_mul_q

Multiply a Polyphase operand and GMP rational (`mpq_t`) with one final rounding.

```
int ppf_mul_q(
    ppf_t *rop,
    const ppf_t *op,
    mpq_srcptr value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_mul_si

Multiply a Polyphase operand and signed long with one final rounding.

```
int ppf_mul_si(
    ppf_t *rop,
    const ppf_t *op,
    long value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_mul_threads

Multiply two values using AUTO product planning and the requested worker count.

```
int ppf_mul_threads(
    ppf_t *rop,
    const ppf_t *a,
    const ppf_t *b,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_mul_ui

Multiply a Polyphase operand and unsigned long with one final rounding.

```
int ppf_mul_ui(
    ppf_t *rop,
    const ppf_t *op,
    unsigned long value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_mul_z

Multiply a Polyphase operand and GMP integer (`mpz_t`) with one final rounding.

```
int ppf_mul_z(
    ppf_t *rop,
    const ppf_t *op,
    mpz_srcptr value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_plan_product

Return the resolved algorithm, requested workers, effective workers, and profile-match status.

```
int ppf_plan_product(
    ppf_product_plan_t *plan,
    const ppf_t *a,
    const ppf_t *b,
    ppf_product_operation_t operation,
    unsigned long workers
);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

ppf_plan_product_algorithm

Return the algorithm chosen by the portable or generated AUTO planner.

```
ppf_product_algorithm_t ppf_plan_product_algorithm(
    const ppf_t *a,
    const ppf_t *b,
    int square,
    unsigned long workers
);
```

Result. The value described above, with the declared C return type.

ppf_product_profile_name

Return the compiled product-profile name, such as `portable`.

```
const char *ppf_product_profile_name(void);
```

Result. A borrowed pointer. Do not free the returned storage. Failure indicator: `NULL where documented`.

Ownership. returned string is library-owned static storage.

ppf_product_profile_sha256

Return the evidence hash of the compiled product profile.

```
const char *ppf_product_profile_sha256(void);
```

Result. A borrowed pointer. Do not free the returned storage. Failure indicator: `NULL where documented`.

Ownership. returned string is library-owned static storage.

ppf_si_div

Compute signed long / Polyphase operand with one final rounding.

```
int ppf_si_div(
    ppf_t *rop,
    long value,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_si_sub

Compute signed long - Polyphase operand with one final rounding.

```
int ppf_si_sub(
    ppf_t *rop,
    long value,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_sqr

Square one Polyphase value with one final rounding.

```
int ppf_sqr(ppf_t *rop, const ppf_t *a, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_sqr_algorithm_threads

Square while explicitly forcing a selected product algorithm.

```
int ppf_sqr_algorithm_threads(
    ppf_t *rop,
    const ppf_t *a,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_product_algorithm_t algorithm,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_sqr_threads

Square a value using AUTO product planning and the requested worker count.

```
int ppf_sqr_threads(
    ppf_t *rop,
    const ppf_t *a,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_sub

Subtract two Polyphase values with one final rounding.

```
int ppf_sub(
    ppf_t *rop,
    const ppf_t *a,
    const ppf_t *b,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_sub_d

Compute Polyphase operand minus double with one final rounding.

```
int ppf_sub_d(
    ppf_t *rop,
    const ppf_t *op,
    double value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_sub_q

Compute Polyphase operand minus GMP rational (`mpq_t`) with one final rounding.

```
int ppf_sub_q(
    ppf_t *rop,
    const ppf_t *op,
    mpq_srcptr value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_sub_si

Compute Polyphase operand minus signed long with one final rounding.

```
int ppf_sub_si(
    ppf_t *rop,
    const ppf_t *op,
    long value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_sub_ui

Compute Polyphase operand minus unsigned long with one final rounding.

```
int ppf_sub_ui(
    ppf_t *rop,
    const ppf_t *op,
    unsigned long value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_sub_z

Compute Polyphase operand minus GMP integer (`mpz_t`) with one final rounding.

```
int ppf_sub_z(
    ppf_t *rop,
    const ppf_t *op,
    mpz_srcptr value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_ui_div

Compute unsigned long / Polyphase operand with one final rounding.

```
int ppf_ui_div(
    ppf_t *rop,
    unsigned long value,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_ui_sub

Compute unsigned long - Polyphase operand with one final rounding.

```
int ppf_ui_sub(
    ppf_t *rop,
    unsigned long value,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_z_sub

Compute GMP integer (`mpz_t`) - Polyphase operand with one final rounding.

```
int ppf_z_sub(
    ppf_t *rop,
    mpz_srcptr value,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

17.16 Roots and hypotenuse

ppf_cbrt

Compute the real cube root.

```
int ppf_cbrt(ppf_t *rop, const ppf_t *a, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Input contract. defined for finite real inputs.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. nontrivial exact-input root interval construction uses `PPF_ROOT_ATTEMPTS` refinement attempts (default 10); exact and exceptional-value cases may return earlier. Consult the numerical-limits appendix for the complete cap inventory.

ppf_hypot

Compute `sqrt(a*a + b*b)` with one final rounding.

```
int ppf_hypot(
    ppf_t *rop,
    const ppf_t *a,
    const ppf_t *b,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. the GMP integer fallback uses root interval construction with `PPF_ROOT_ATTEMPTS` refinement attempts (default 10); the direct-limb path calls the square-root operation with its separate 24-attempt limit; these are path-specific limits, not one global attempt budget. Consult the numerical-limits appendix for the complete cap inventory.

ppf_rec_sqrt

Compute the reciprocal square root.

```
int ppf_rec_sqrt(
    ppf_t *rop,
    const ppf_t *a,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. finite regular inputs require $x > 0$.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. the Newton interval branch and the root-plus-quotient fallback each have a `PPF_ROOT_ATTEMPTS` refinement limit (default 10 per branch), not a shared global total of 10 attempts; exact and exceptional-value cases may return earlier. Consult the numerical-limits appendix for the complete cap inventory.

ppf_rec_sqrt_algorithm_threads

Compute reciprocal square root while forcing a selected product algorithm.

```
int ppf_rec_sqrt_algorithm_threads(
    ppf_t *rop,
    const ppf_t *a,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_product_algorithm_t algorithm,
    ppf_context_t *ctx
);
```

Input contract. finite regular inputs require $x > 0$.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. the Newton interval branch and the root-plus-quotient fallback each have a `PPF_ROOT_ATTEMPTS` refinement limit (default 10 per branch), not a shared global total of 10 attempts; exact and exceptional-value cases may return earlier. Consult the numerical-limits appendix for the complete cap inventory.

ppf_rec_sqrt_threads

Compute the reciprocal square root using an internal worker request.

```
int ppf_rec_sqrt_threads(
    ppf_t *rop,
    const ppf_t *a,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_context_t *ctx
);
```

Input contract. finite regular inputs require $x > 0$.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. the Newton interval branch and the root-plus-quotient fallback each have a `PPF_ROOT_ATTEMPTS` refinement limit (default 10 per branch), not a shared global total of 10 attempts; exact and exceptional-value cases may return earlier. Consult the numerical-limits appendix for the complete cap inventory.

ppf_root

Legacy unsigned-degree root interface with its documented signed-zero convention.

```
int ppf_root(
    ppf_t *rop,
    const ppf_t *a,
    unsigned long degree,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. degree must be nonzero; even degrees require nonnegative finite input.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. nontrivial exact-input root interval construction uses `PPF_ROOT_ATTEMPTS` refinement attempts (default 10); exact and exceptional-value cases may return earlier. Consult the numerical-limits appendix for the complete cap inventory.

ppf_rootn_si

Compute an integer-degree root; a negative degree requests a reciprocal root.

```
int ppf_rootn_si(
    ppf_t *rop,
    const ppf_t *a,
    long degree,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. degree must be nonzero; even degrees require nonnegative finite input; negative degrees additionally exclude zero.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. nontrivial exact-input root interval construction uses `PPF_ROOT_ATTEMPTS` refinement attempts (default 10); exact and exceptional-value cases may return earlier. Consult the numerical-limits appendix for the complete cap inventory.

ppf_rootn_ui

Compute an integer-degree root; unsigned degree.

```
int ppf_rootn_ui(
    ppf_t *rop,
    const ppf_t *a,
    unsigned long degree,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. degree must be nonzero; even degrees require nonnegative finite input.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. nontrivial exact-input root interval construction uses `PPF_ROOT_ATTEMPTS` refinement attempts (default 10); exact and exceptional-value cases may return earlier. Consult the numerical-limits appendix for the complete cap inventory.

ppf_sqrt

Compute the square root.

```
int ppf_sqrt(ppf_t *rop, const ppf_t *a, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Input contract. finite regular inputs require x >= 0.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. square-root interval refinement is limited to 24 attempts; exact and exceptional-value cases may return earlier; ppf_sqrt_ui delegates to the same square-root operation. Consult the numerical-limits appendix for the complete cap inventory.

ppf_sqrt_ui

Compute the square root of an unsigned long.

```
int ppf_sqrt_ui(
    ppf_t *rop,
    unsigned long value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. defined for every unsigned input.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. square-root interval refinement is limited to 24 attempts; exact and exceptional-value cases may return earlier; ppf_sqrt_ui delegates to the same square-root operation. Consult the numerical-limits appendix for the complete cap inventory.

17.17 Reusable division, fused operations, powers of two, sums, and dot products

ppf_div_2si

Divide by `2^shift` with a signed shift.

```
int ppf_div_2si(
    ppf_t *rop,
    const ppf_t *a,
    long shift,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_div_2ui

Divide by `2^shift` with an unsigned shift.

```
int ppf_div_2ui(
    ppf_t *rop,
    const ppf_t *a,
    unsigned long shift,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_div_with_plan

Divide one numerator by a planned denominator.

```
int ppf_div_with_plan(
    ppf_t *rop,
    const ppf_t *numerator,
    const ppf_divisor_plan_t *plan,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. plan must be complete and numerator valid.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Concurrency. Share the plan only under its documented immutable-plan and lifetime rules.

ppf_div_with_plan_many

Divide many independent numerators by one immutable plan, optionally in parallel.

```
int ppf_div_with_plan_many(
    ppf_t *outputs[],
    const ppf_t *const numerators[],
    size_t length,
    const ppf_divisor_plan_t *plan,
    ppf_rnd_t rnd,
    unsigned long workers,
    int ternaries[],
    ppf_context_t *ctx
);
```

Input contract. plan and every output/numerator entry must be valid.

Result. Returns 0 on success and -1 on failure. When supplied, the ternaries array receives the individual numerical ternary results; the function return is not a numerical ternary. Failure indicator: `-1`.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. outputsi may equal numeratorsi for each index; Forbidden: duplicate outputs and cross-index output/input overlap.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Concurrency. Share the plan only under its documented immutable-plan and lifetime rules.

ppf_divisor_plan_applications

Return how many successful applications the plan has recorded.

```
unsigned long ppf_divisor_plan_applications(
    const ppf_divisor_plan_t *plan
);
```

Result. The value described above, with the declared C return type.

Concurrency. Share the plan only under its documented immutable-plan and lifetime rules.

ppf_divisor_plan_create

Create an immutable plan for repeatedly dividing into one destination precision.

```
int ppf_divisor_plan_create(
    ppf_divisor_plan_t **out,
    const ppf_t *denominator,
    ppf_prec_t destination_precision,
    ppf_context_t *ctx
);
```

Input contract. denominator must be finite and nonzero; destination precision must be valid.

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Context and rounding. A null `ctx` selects the calling thread's default context.

Ownership. successful creation returns a plan released by ppf_divisor_plan_destroy.

Concurrency. Application locking is required for shared mutable use.

ppf_divisor_plan_create_ex

Create a reusable divisor plan with an expected application count for planning.

```
int ppf_divisor_plan_create_ex(
    ppf_divisor_plan_t **out,
    const ppf_t *denominator,
    ppf_prec_t destination_precision,
    unsigned long expected_applications,
    ppf_context_t *ctx
);
```

Input contract. denominator must be finite and nonzero; destination precision must be valid.

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Context and rounding. A null `ctx` selects the calling thread's default context.

Ownership. successful creation returns a plan released by ppf_divisor_plan_destroy.

Allocation and failure. After initial argument validation, *out is set to NULL; project-allocation failure releases the incomplete plan and leaves* out NULL and the denominator unchanged; this does not preserve an existing pointer passed through out.

Concurrency. Application locking is required for shared mutable use.

ppf_divisor_plan_destroy

Destroy a reusable divisor plan.

```
void ppf_divisor_plan_destroy(ppf_divisor_plan_t *plan);
```

Result. No return value.

Ownership. releases a completed or partially constructed divisor plan.

Concurrency. Application locking is required for shared mutable use.

ppf_divisor_plan_precision

Return the destination precision stored in a divisor plan.

```
ppf_prec_t ppf_divisor_plan_precision( const ppf_divisor_plan_t *plan);
```

Result. The value described above, with the declared C return type.

Concurrency. Share the plan only under its documented immutable-plan and lifetime rules.

ppf_dot

Compute a dot product with exact products and one final rounding.

```
int ppf_dot(
    ppf_t *rop,
    const ppf_t *const a[],
    const ppf_t *const b[],
    size_t length,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_dot_threads

Compute a dot product with parallel product leaves and one final rounding.

```
int ppf_dot_threads(
    ppf_t *rop,
    const ppf_t *const a[],
    const ppf_t *const b[],
    size_t length,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_fac_ui

Compute an unsigned integer factorial.

```
int ppf_fac_ui(
    ppf_t *rop,
    unsigned long value,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_fma

Compute `a*b + c` with one final rounding.

```
int ppf_fma(
    ppf_t *rop,
    const ppf_t *a,
    const ppf_t *b,
    const ppf_t *c,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_fma_algorithm_threads

Threaded FMA while forcing a selected exact product graph.

```
int ppf_fma_algorithm_threads(
    ppf_t *rop,
    const ppf_t *a,
    const ppf_t *b,
    const ppf_t *c,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_product_algorithm_t algorithm,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_fma_threads

Threaded FMA using AUTO product selection.

```
int ppf_fma_threads(
    ppf_t *rop,
    const ppf_t *a,
    const ppf_t *b,
    const ppf_t *c,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_fmma

Compute `a*b + c*d` with one final rounding.

```
int ppf_fmma(
    ppf_t *rop,
    const ppf_t *a,
    const ppf_t *b,
    const ppf_t *c,
    const ppf_t *d,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_fmms

Compute `a*b - c*d` with one final rounding.

```
int ppf_fmms(
    ppf_t *rop,
    const ppf_t *a,
    const ppf_t *b,
    const ppf_t *c,
    const ppf_t *d,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_fms

Compute `a*b - c` with one final rounding.

```
int ppf_fms(
    ppf_t *rop,
    const ppf_t *a,
    const ppf_t *b,
    const ppf_t *c,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_fms_algorithm_threads

Threaded fused multiply-subtract while forcing a product graph.

```
int ppf_fms_algorithm_threads(
    ppf_t *rop,
    const ppf_t *a,
    const ppf_t *b,
    const ppf_t *c,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_product_algorithm_t algorithm,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_fms_threads

Threaded fused multiply-subtract using AUTO product selection.

```
int ppf_fms_threads(
    ppf_t *rop,
    const ppf_t *a,
    const ppf_t *b,
    const ppf_t *c,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_mul_2si

Multiply by `2^shift` with a signed shift.

```
int ppf_mul_2si(
    ppf_t *rop,
    const ppf_t *a,
    long shift,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_mul_2ui

Multiply by `2^shift` with an unsigned shift.

```
int ppf_mul_2ui(
    ppf_t *rop,
    const ppf_t *a,
    unsigned long shift,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

ppf_sum

Sum an array exactly before one final rounding.

```
int ppf_sum(
    ppf_t *rop,
    const ppf_t *const values[],
    size_t length,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

17.18 Logarithms, exponentials, and powers

ppf_compound_si

Compute the compound-interest form `(1 + op)^exponent` with signed integer exponent.

```
int ppf_compound_si(
    ppf_t *rop,
    const ppf_t *op,
    long exponent,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_exp

Compute the natural exponential with certified final rounding.

```
int ppf_exp(ppf_t *rop, const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_exp10

Compute the base-10 exponential with certified final rounding.

```
int ppf_exp10(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_exp10m1

Compute the 10 ^ x-1 with certified final rounding.

```
int ppf_exp10m1(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_exp2

Compute the base-2 exponential with certified final rounding.

```
int ppf_exp2(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_exp2m1

Compute the 2 ^ x-1 with certified final rounding.

```
int ppf_exp2m1(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_expm1

Compute the exp(x)-1 with certified final rounding.

```
int ppf_expm1(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_log

Compute the natural logarithm with certified final rounding.

```
int ppf_log(ppf_t *rop, const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Input contract. finite regular inputs require x > 0; zero and negative inputs follow documented exceptional-value semantics.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_log10

Compute the base-10 logarithm with certified final rounding.

```
int ppf_log10(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. finite regular inputs require x > 0.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_log10p1

Compute the base-10 logarithm of 1+x with certified final rounding.

```
int ppf_log10p1(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. finite regular inputs require x > -1.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_log1p

Compute the natural logarithm of 1+x with certified final rounding.

```
int ppf_log1p(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. finite regular inputs require x > -1.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_log2

Compute the base-2 logarithm with certified final rounding.

```
int ppf_log2(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. finite regular inputs require x > 0.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_log2p1

Compute the base-2 logarithm of 1+x with certified final rounding.

```
int ppf_log2p1(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. finite regular inputs require x > -1.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_log_ui

Compute the natural logarithm of an unsigned integer with certified final rounding.

```
int ppf_log_ui(
    ppf_t *rop,
    unsigned long op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_pow

Compute the general real power with certified final rounding.

```
int ppf_pow(
    ppf_t *rop,
    const ppf_t *base,
    const ppf_t *exponent,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_pow_si

Compute the power with a signed long exponent with certified final rounding.

```
int ppf_pow_si(
    ppf_t *rop,
    const ppf_t *base,
    long exponent,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_pow_sj

Compute the power with an intmax_t exponent with certified final rounding.

```
int ppf_pow_sj(
    ppf_t *rop,
    const ppf_t *base,
    intmax_t exponent,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_pow_ui

Compute the power with an unsigned long exponent with certified final rounding.

```
int ppf_pow_ui(
    ppf_t *rop,
    const ppf_t *base,
    unsigned long exponent,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_pow_uj

Compute the power with a uintmax_t exponent with certified final rounding.

```
int ppf_pow_uj(
    ppf_t *rop,
    const ppf_t *base,
    uintmax_t exponent,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_pow_z

Compute the power with a GMP integer exponent with certified final rounding.

```
int ppf_pow_z(
    ppf_t *rop,
    const ppf_t *base,
    mpz_srcptr exponent,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_pown

Compute the power with a signed integer exponent using the `pown` entry with certified final rounding.

```
int ppf_pown(
    ppf_t *rop,
    const ppf_t *base,
    intmax_t exponent,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_powr

Compute the real power with the restricted nonnegative-base `powr` domain with certified final rounding.

```
int ppf_powr(
    ppf_t *rop,
    const ppf_t *base,
    const ppf_t *exponent,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. real-valued power function requiring a nonnegative base, with zero cases handled separately.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_ui_pow

Compute the unsigned-long base to Polyphase exponent with certified final rounding.

```
int ppf_ui_pow(
    ppf_t *rop,
    unsigned long base,
    const ppf_t *exponent,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

ppf_ui_pow_ui

Compute the unsigned-long base to unsigned-long exponent with certified final rounding.

```
int ppf_ui_pow_ui(
    ppf_t *rop,
    unsigned long base,
    unsigned long exponent,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. certification uses bounded escalation and at most 10,000,000 series or reduction steps in the general backend; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits by default, also bounding the endpoint-to-one exponent gap before exponential-minus-one subtraction; integer-power shortcuts use a precision-dependent budget; unusable certificates retry or reject. Consult the numerical-limits appendix for the complete cap inventory.

17.19 Trigonometric and hyperbolic functions

ppf_acos

Compute inverse cosine in radians with certified final rounding.

```
int ppf_acos(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. finite regular inputs require -1 <= x <= 1.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_acosh

Compute inverse hyperbolic cosine with certified final rounding.

```
int ppf_acosh(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. finite real inputs require x >= 1.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_acospi

Compute inverse cosine divided by pi with certified final rounding.

```
int ppf_acospi(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. finite regular inputs require -1 <= x <= 1.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_acosu

Compute the inverse cos scaled into units of `u/(2*pi)`.

```
int ppf_acosu(
    ppf_t *rop,
    const ppf_t *op,
    unsigned long u,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. finite regular inputs require -1 <= x <= 1 and u != 0.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_asin

Compute inverse sine in radians with certified final rounding.

```
int ppf_asin(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. finite regular inputs require -1 <= x <= 1.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_asinh

Compute inverse hyperbolic sine with certified final rounding.

```
int ppf_asinh(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_asinpi

Compute inverse sine divided by pi with certified final rounding.

```
int ppf_asinpi(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. finite regular inputs require -1 <= x <= 1.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_asinu

Compute the inverse sin scaled into units of `u/(2*pi)`.

```
int ppf_asinu(
    ppf_t *rop,
    const ppf_t *op,
    unsigned long u,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. finite regular inputs require -1 <= x <= 1 and u != 0.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_atan

Compute inverse tangent in radians with certified final rounding.

```
int ppf_atan(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_atan2

Compute quadrant-aware inverse tangent in radians with certified final rounding.

```
int ppf_atan2(
    ppf_t *rop,
    const ppf_t *y,
    const ppf_t *x,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_atan2pi

Compute quadrant-aware inverse tangent divided by pi with certified final rounding.

```
int ppf_atan2pi(
    ppf_t *rop,
    const ppf_t *y,
    const ppf_t *x,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_atan2u

Compute quadrant-aware atan2 scaled into units of `u/(2*pi)`.

```
int ppf_atan2u(
    ppf_t *rop,
    const ppf_t *y,
    const ppf_t *x,
    unsigned long u,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_atanh

Compute inverse hyperbolic tangent with certified final rounding.

```
int ppf_atanh(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. finite real result requires $-1 < x < 1$; endpoints and exterior values follow exceptional semantics.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_atanpi

Compute inverse tangent divided by pi with certified final rounding.

```
int ppf_atanpi(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_atanu

Compute the inverse tan scaled into units of `u/(2*pi)`.

```
int ppf_atanu(
    ppf_t *rop,
    const ppf_t *op,
    unsigned long u,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. u must be nonzero.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_cos

Compute cosine in radians with certified final rounding.

```
int ppf_cos(ppf_t *rop, const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_cosh

Compute hyperbolic cosine with certified final rounding.

```
int ppf_cosh(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_cospi

Compute cos(pi*x) with certified final rounding.

```
int ppf_cospi(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_cosu

Compute cos(2*pi*x/u), where `u` is an unsigned turn-unit denominator.

```
int ppf_cosu(
    ppf_t *rop,
    const ppf_t *op,
    unsigned long u,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_cot

Compute cotangent with certified final rounding.

```
int ppf_cot(ppf_t *rop, const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_coth

Compute hyperbolic cotangent with certified final rounding.

```
int ppf_coth(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_csc

Compute cosecant with certified final rounding.

```
int ppf_csc(ppf_t *rop, const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_csch

Compute hyperbolic cosecant with certified final rounding.

```
int ppf_csch(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_get_trig_threads

Read the current trigonometric worker request.

```
unsigned long ppf_get_trig_threads(void);
```

Result. The value described above, with the declared C return type.

ppf_sec

Compute secant with certified final rounding.

```
int ppf_sec(ppf_t *rop, const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_sech

Compute hyperbolic secant with certified final rounding.

```
int ppf_sech(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_set_trig_threads

Set the default internal worker request for trigonometric calculations.

```
void ppf_set_trig_threads(unsigned long workers);
```

Result. No return value.

ppf_sin

Compute sine in radians with certified final rounding.

```
int ppf_sin(ppf_t *rop, const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_sin_cos

Compute sine and cosine into two outputs.

```
int ppf_sin_cos(
    ppf_t *sop,
    ppf_t *cop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. Test for `PPF_TERNARY_ERROR` before decoding. Otherwise the return is code(first) + 4 * code(second), where 0 means exact, 1 means above the exact result, and 2 means below it: the sine output first and the cosine output second. The sign of the packed return is not a numerical ternary. Failure indicator: `PPF_TERNARY_ERROR`.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. either floating output may alias the floating input individually; Forbidden: the two floating outputs must not be the same object.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_sinh

Compute hyperbolic sine with certified final rounding.

```
int ppf_sinh(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_sinh_cosh

Compute hyperbolic sine and cosine into two outputs.

```
int ppf_sinh_cosh(
    ppf_t *sop,
    ppf_t *cop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. Test for `PPF_TERNARY_ERROR` before decoding. Otherwise the return is code(first) + 4 * code(second), where 0 means exact, 1 means above the exact result, and 2 means below it: the hyperbolic sine output first and the hyperbolic cosine output second. The sign of the packed return is not a numerical ternary. Failure indicator: `PPF_TERNARY_ERROR`.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. either floating output may alias the floating input individually; Forbidden: the two floating outputs must not be the same object.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_sinpi

Compute sin(pi*x) with certified final rounding.

```
int ppf_sinpi(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_sinu

Compute sin(2*pix*/u), where `u` is an unsigned turn-unit denominator.

```
int ppf_sinu(
    ppf_t *rop,
    const ppf_t *op,
    unsigned long u,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_tan

Compute tangent in radians with certified final rounding.

```
int ppf_tan(ppf_t *rop, const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_tanh

Compute hyperbolic tangent with certified final rounding.

```
int ppf_tanh(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_tanpi

Compute tan(pi*x) with certified final rounding.

```
int ppf_tanpi(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

ppf_tanu

Compute tan(2*pi*x/u), where `u` is an unsigned turn-unit denominator.

```
int ppf_tanu(
    ppf_t *rop,
    const ppf_t *op,
    unsigned long u,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. general trigonometric certification uses bounded escalation, bounded reduction growth, and at most 10,000,000 steps. Consult the numerical-limits appendix for the complete cap inventory.

17.20 Special functions

ppf_agm

Compute the arithmetic-geometric mean with certified final rounding.

```
int ppf_agm(
    ppf_t *rop,
    const ppf_t *op1,
    const ppf_t *op2,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

ppf_ai

Compute the Airy Ai function with certified final rounding.

```
int ppf_ai(ppf_t *rop, const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Input contract. implemented real-domain path is restricted to $|x| <= 500$; infinities are handled separately.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates; the current Airy Ai path additionally rejects finite $|x| > 500$. Consult the numerical-limits appendix for the complete cap inventory.

ppf_beta

Compute the Beta function with certified final rounding.

```
int ppf_beta(
    ppf_t *rop,
    const ppf_t *op1,
    const ppf_t *op2,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. real arguments where the Gamma quotient is finite; singular combinations follow exceptional semantics.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

ppf_digamma

Compute the digamma function with certified final rounding.

```
int ppf_digamma(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. real input excluding nonpositive integer poles.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

ppf_eint

Compute the real exponential integral Ei with certified final rounding.

```
int ppf_eint(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

ppf_erf

Compute the error function with certified final rounding.

```
int ppf_erf(ppf_t *rop, const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

ppf_erfc

Compute the complementary error function with certified final rounding.

```
int ppf_erfc(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

ppf_gamma

Compute the Gamma function with certified final rounding.

```
int ppf_gamma(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. real input excluding nonpositive integer poles.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

ppf_gamma_inc

Compute the upper incomplete gamma function for two real arguments.

```
int ppf_gamma_inc(
    ppf_t *rop,
    const ppf_t *op,
    const ppf_t *op2,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

ppf_j0

Compute the Bessel J of order 0 with certified final rounding.

```
int ppf_j0(ppf_t *rop, const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction

permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

ppf_j1

Compute the Bessel J of order 1 with certified final rounding.

```
int ppf_j1(ppf_t *rop, const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

ppf_jn

Compute the Bessel J of integer order n with certified final rounding.

```
int ppf_jn(
    ppf_t *rop,
    long n,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

ppf_lgamma

Compute `log(abs(Gamma(x)))` and return the sign of Gamma separately.

```
int ppf_lgamma(
    ppf_t *rop,
    int *signp,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. real input; sign output records the sign of Gamma away from poles.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

ppf_li2

Compute the real dilogarithm Li2 with certified final rounding.

```
int ppf_li2(ppf_t *rop, const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

ppf_lngamma

Compute the real logarithm of Gamma where Gamma is positive.

```
int ppf_lngamma(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. real input excluding nonpositive integer poles when a finite real logarithm is required.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

ppf_y0

Compute the Bessel Y of order 0 with certified final rounding.

```
int ppf_y0(ppf_t *rop, const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Input contract. finite real series path requires $x > 0$.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

ppf_y1

Compute the Bessel Y of order 1 with certified final rounding.

```
int ppf_y1(ppf_t *rop, const ppf_t *op, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Input contract. finite real series path requires $x > 0$.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

ppf_yn

Compute the Bessel Y of integer order n with certified final rounding.

```
int ppf_yn(
    ppf_t *rop,
    long n,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. finite real series path requires $x > 0$.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

ppf_zeta

Compute the Riemann zeta function with certified final rounding.

```
int ppf_zeta(
    ppf_t *rop,
    const ppf_t *op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Input contract. real input with a pole at $x = 1$.

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Overlap. the floating destination may alias each floating source position exercised by the aliasing matrix.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

ppf_zeta_ui

Compute the Riemann zeta at an unsigned integer with certified final rounding.

```
int ppf_zeta_ui(
    ppf_t *rop,
    unsigned long op,
    ppf_rnd_t rnd,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. special-function certification uses bounded precision escalation; several series and recurrences are capped at 1,000,000 iterations; dyadic interval construction permits at most 1,000,000 extra endpoint-alignment bits and retries or rejects unusable certificates. Consult the numerical-limits appendix for the complete cap inventory.

17.21 Certified constants and constant caches

ppf_clear_constant_cache

Clear the cache for one selected constant.

```
void ppf_clear_constant_cache(ppf_constant_id_t id);
```

Result. No return value.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_const_catalan

Compute Catalan's constant through the cached default-worker certified constant path.

```
int ppf_const_catalan(ppf_t *rop, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. constant certification uses at most 16 precision-escalation attempts per request. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_const_catalan_threads

Compute Catalan's constant through the cached explicit-worker certified constant path.

```
int ppf_const_catalan_threads(
    ppf_t *rop,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. constant certification uses at most 16 precision-escalation attempts per request. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_const_catalan_uncached

Compute Catalan's constant through the uncached explicit-worker certified constant path.

```
int ppf_const_catalan_uncached(
    ppf_t *rop,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. constant certification uses at most 16 precision-escalation attempts per request. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_const_euler

Compute Euler's constant through the cached default-worker certified constant path.

```
int ppf_const_euler(ppf_t *rop, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. constant certification uses at most 16 precision-escalation attempts per request. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_const_euler_threads

Compute Euler's constant through the cached explicit-worker certified constant path.

```
int ppf_const_euler_threads(
    ppf_t *rop,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. constant certification uses at most 16 precision-escalation attempts per request. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_const_euler_uncached

Compute Euler's constant through the uncached explicit-worker certified constant path.

```
int ppf_const_euler_uncached(
    ppf_t *rop,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. constant certification uses at most 16 precision-escalation attempts per request. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_const_log2

Compute log(2) through the cached default-worker certified constant path.

```
int ppf_const_log2(ppf_t *rop, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. constant certification uses at most 16 precision-escalation attempts per request. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_const_log2_threads

Compute log(2) through the cached explicit-worker certified constant path.

```
int ppf_const_log2_threads(
    ppf_t *rop,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. constant certification uses at most 16 precision-escalation attempts per request. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_const_log2_uncached

Compute log(2) through the uncached explicit-worker certified constant path.

```
int ppf_const_log2_uncached(
    ppf_t *rop,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. constant certification uses at most 16 precision-escalation attempts per request. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_const_pi

Compute pi through the cached default-worker certified constant path.

```
int ppf_const_pi(ppf_t *rop, ppf_rnd_t rnd, ppf_context_t *ctx);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. constant certification uses at most 16 precision-escalation attempts per request. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_const_pi_threads

Compute pi through the cached explicit-worker certified constant path.

```
int ppf_const_pi_threads(
    ppf_t *rop,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. constant certification uses at most 16 precision-escalation attempts per request. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_const_pi_uncached

Compute pi through the uncached explicit-worker certified constant path.

```
int ppf_const_pi_uncached(
    ppf_t *rop,
    ppf_rnd_t rnd,
    unsigned long workers,
    ppf_context_t *ctx
);
```

Result. A numerical ternary result. Test for `PPF_TERNARY_ERROR` before interpreting its sign.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

Resource policy. constant certification uses at most 16 precision-escalation attempts per request. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_constant_cache_precision

Return the precision currently retained for a constant cache.

```
ppf_prec_t ppf_constant_cache_precision(ppf_constant_id_t id);
```

Result. The value described above, with the declared C return type.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_free_constant_caches

Release all certified constant caches.

```
void ppf_free_constant_caches(void);
```

Result. No return value.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_get_constant_interval

Export a certified dyadic interval around a supported constant.

```
int ppf_get_constant_interval(
    ppf_constant_id_t id,
    mpz_ptr lower,
    mpz_ptr upper,
    ppf_exp_t *exponent,
    ppf_prec_t precision,
    unsigned long workers
);
```

Result. Returns 0 on success and -1 on failure. The certified dyadic interval is written to lower, upper, and exponent. Failure indicator: `-1`.

Overlap. lower and upper may refer to distinct caller-owned integers; Forbidden: lower and upper must not be the same mpz object.

Allocation and failure. Unrecovered project-allocation failure before publication preserves lower, upper, and exponent and releases temporary project-owned storage; GMP allocation while copying successful outputs is outside this recoverable allocation contract.

Resource policy. constant certification uses at most 16 precision-escalation attempts per request. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_get_constant_threads

Read the default worker request for constant construction.

```
unsigned long ppf_get_constant_threads(void);
```

Result. The value described above, with the declared C return type.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_get_last_constant_stats

Read statistics from the most recent constant computation in the calling thread.

```
int ppf_get_last_constant_stats(ppf_constant_stats_t *out);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_precompute_catalan

Precompute and cache catalan to a requested precision.

```
int ppf_precompute_catalan(ppf_prec_t precision, unsigned long workers);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Allocation and failure. Unrecovered project-allocation failure releases the incomplete interval without publishing it to the constant cache; an existing cached interval remains valid; GMP allocation is outside this recoverable allocation contract.

Resource policy. constant certification uses at most 16 precision-escalation attempts per request. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_precompute_constant

Precompute and cache constant to a requested precision.

```
int ppf_precompute_constant(
    ppf_constant_id_t id,
    ppf_prec_t precision,
    unsigned long workers
);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Allocation and failure. Unrecovered project-allocation failure releases the incomplete interval without publishing it to the constant cache; an existing cached interval remains valid; GMP allocation is outside this recoverable allocation contract.

Resource policy. constant certification uses at most 16 precision-escalation attempts per request. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_precompute_euler

Precompute and cache euler to a requested precision.

```
int ppf_precompute_euler(ppf_prec_t precision, unsigned long workers);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Allocation and failure. Unrecovered project-allocation failure releases the incomplete interval without publishing it to the constant cache; an existing cached interval remains valid; GMP allocation is outside this recoverable allocation contract.

Resource policy. constant certification uses at most 16 precision-escalation attempts per request. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_precompute_log2

Precompute and cache log(2) to a requested precision.

```
int ppf_precompute_log2(ppf_prec_t precision, unsigned long workers);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Allocation and failure. Unrecovered project-allocation failure releases the incomplete interval without publishing it to the constant cache; an existing cached interval remains valid; GMP allocation is outside this recoverable allocation contract.

Resource policy. constant certification uses at most 16 precision-escalation attempts per request. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_precompute_pi

Precompute and cache pi to a requested precision.

```
int ppf_precompute_pi(ppf_prec_t precision, unsigned long workers);
```

Result. A status code. Check the operation's success convention; this is not a numerical ternary. Failure indicator: `function-specific nonzero failure status`.

Allocation and failure. Unrecovered project-allocation failure releases the incomplete interval without publishing it to the constant cache; an existing cached interval remains valid; GMP allocation is outside this recoverable allocation contract.

Resource policy. constant certification uses at most 16 precision-escalation attempts per request. Consult the numerical-limits appendix for the complete cap inventory.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

ppf_set_constant_threads

Set the default worker request for constant construction.

```
void ppf_set_constant_threads(unsigned long workers);
```

Result. No return value.

Concurrency. The library synchronizes its global state. Caller-owned mutable objects still require exclusive access.

17.22 Certified interval assignment

ppf_set_dyadic_interval

Round a certified dyadic interval into a destination, reporting decided, undecided, or error.

```
ppf_interval_status_t ppf_set_dyadic_interval(
    ppf_t *rop,
    mpz_srcptr lower,
    mpz_srcptr upper,
    ppf_exp_t exponent,
    ppf_rnd_t rnd,
    int *ternary,
    ppf_context_t *ctx
);
```

Input contract. lower and upper are exact integers with lower <= upper and share the supplied binary exponent.

Result. `PPF_INTERVAL_DECIDED`, `PPF_INTERVAL_UNDECIDED`, or `PPF_INTERVAL_ERROR`. The numerical ternary is a separate output.

Context and rounding. A null `ctx` selects the calling thread's default context. PPF_RND_DEFAULT resolves through the supplied context or the calling thread default context.

Allocation and failure. An unrecovered project-allocation failure preserves the prior numerical output. Sticky flags may change, and an exact fallback may succeed while retaining the allocation flag. Numerical rejection is a separate case and may commit NaN.

17.23 Core comparison, validation, and diagnostics

ppf_cmp

Compare two Polyphase values.

```
int ppf_cmp(const ppf_t *a, const ppf_t *b, ppf_context_t *ctx);
```

Result. An ordering result. NaNs and unordered comparisons require the operation's documented handling.

Context and rounding. A null `ctx` selects the calling thread's default context.

ppf_dump

Write an internal diagnostic representation to standard output.

```
void ppf_dump(const ppf_t *x);
```

Result. No return value.

ppf_dump_file

Write an internal diagnostic representation to a selected stream.

```
void ppf_dump_file(FILE *stream, const ppf_t *x);
```

Result. No return value.

ppf_equal_p

Test exact equality of represented values.

```
int ppf_equal_p(const ppf_t *a, const ppf_t *b);
```

Result. A predicate result: zero means false and a nonzero value means true.

ppf_kind_name

Return a readable name for a value kind.

```
const char *ppf_kind_name(ppf_kind_t kind);
```

Result. A borrowed pointer. Do not free the returned storage. Failure indicator: `NULL where documented`.

Ownership. returned string is library-owned static storage.

ppf_rounding_name

Return a readable name for a rounding mode.

```
const char *ppf_rounding_name(ppf_rnd_t rnd);
```

Result. Returns a static name for the supplied enumeration value, including `PPF_RND_DEFAULT`. An invalid enumeration value returns the static string `PPF_RND_INVALID`; this function does not return NULL.

Context and rounding. the enumeration is inspected without selecting or resolving a rounding policy; PPF_RND_DEFAULT has its own literal name.

Ownership. returned string is library-owned static storage.

ppf_validate

Check internal representation invariants.

```
int ppf_validate(const ppf_t *x);
```

Result. A predicate result: zero means false and a nonzero value means true.

Chapter 18

Alphabetical function index

Every entry below links to its core API reference.

18.1 A

Function	Function
ppf_abi_contract_sha256	ppf_allocflag_p
ppf_abi_epoch	ppf_arena_alloc
ppf_abi_info_size	ppf_arena_mark
ppf_abi_revision	ppf_arena_release
ppf_abi_tag	ppf_arena_reset
ppf_abs	ppf_asin
ppf_acos	ppf_asinh
ppf_acosh	ppf_asinpi
ppf_acospi	ppf_asinu
ppf_acosu	ppf_asprintf
ppf_add	ppf_atan
ppf_add_d	ppf_atan2
ppf_add_q	ppf_atan2pi
ppf_add_si	ppf_atan2u
ppf_add_ui	ppf_atanh
ppf_add_z	ppf_atanpi
ppf_agm	ppf_atanu
ppf_ai	

18.2 B

Function	Function
ppf_beta	ppf_buildopt_sharedcache_p
ppf_buildopt_decimal_p	ppf_buildopt_thread_cleanup_p
ppf_buildopt_float128_p	ppf_buildopt_tls_p
ppf_buildopt_gmpinternals_p	ppf_buildopt_tune_case

18.3 C

Function	Function
ppf_can_round	ppf_const_log2
ppf_cbrt	ppf_const_log2_threads
ppf_ceil	ppf_const_log2_uncached
ppf_check_range	ppf_const_pi
ppf_clear	ppf_const_pi_threads
ppf_clear_allocflag	ppf_const_pi_uncached
ppf_clear_constant_cache	ppf_constant_cache_precision
ppf_clear_divby0	ppf_context_clear
ppf_clear_erangeflag	ppf_context_clone
ppf_clear_flags	ppf_context_flags
ppf_clear_inexflag	ppf_context_flags_clear
ppf_clear_nanflag	ppf_context_flags_restore
ppf_clear_overflow	ppf_context_flags_save
ppf_clear_underflow	ppf_context_flags_set
ppf_clears	ppf_context_init
ppf_cmp	ppf_context_init_custom
ppf_cmp_d	ppf_context_set_default_precision
ppf_cmp_f	ppf_context_set_range
ppf_cmp_ld	ppf_context_set_rounding
ppf_cmp_q	ppf_copysign
ppf_cmp_si	ppf_cos
ppf_cmp_si_2exp	ppf_cosh
ppf_cmp_ui	ppf_cospi
ppf_cmp_ui_2exp	ppf_cosu
ppf_cmp_z	ppf_cot
ppf_cmpabs	ppf_coth
ppf_cmpabs_ui	ppf_csc
ppf_commit_regular	ppf_csch
ppf_compound_si	ppf_custom_get_exp

Function	Function
ppf_const_catalan	ppf_custom_get_kind
ppf_const_catalan_threads	ppf_custom_get_significand
ppf_const_catalan_uncached	ppf_custom_get_size
ppf_const_euler	ppf_custom_init
ppf_const_euler_threads	ppf_custom_init_set
ppf_const_euler_uncached	ppf_custom_move

18.4 D

Function	Function
ppf_d_div	ppf_div_ui
ppf_d_sub	ppf_div_with_plan
ppf_default_allocator	ppf_div_with_plan_many
ppf_default_context	ppf_div_z
ppf_default_context_cleanup	ppf_divby0_p
ppf_digamma	ppf_divisor_plan_applications
ppf_dim	ppf_divisor_plan_create
ppf_div	ppf_divisor_plan_create_ex
ppf_div_2exp	ppf_divisor_plan_destroy
ppf_div_2si	ppf_divisor_plan_precision
ppf_div_2ui	ppf_dot
ppf_div_d	ppf_dot_threads
ppf_div_q	ppf_dump
ppf_div_si	ppf_dump_file

18.5 E

Function	Function
ppf_eint	ppf_exp
ppf_eq	ppf_exp10
ppf_equal_p	ppf_exp10m1
ppf_erandom	ppf_exp2
ppf_erangeflag_p	ppf_exp2m1
ppf_erf	ppf_expm1
ppf_erfc	

18.6 F

Function	Function
ppf_fac_ui	ppf_fmma
ppf_fits_intmax_p	ppf_fmms
ppf_fits_sint_p	ppf_fmod
ppf_fits_slong_p	ppf_fmod_ui
ppf_fits_sshort_p	ppf_fmodquo
ppf_fits_uint_p	ppf_fms
ppf_fits_uintmax_p	ppf_fms_algorithm_threads
ppf_fits_ulong_p	ppf_fms_threads
ppf_fits_ushort_p	ppf_fpif_export
ppf_flags_clear	ppf_fpif_import
ppf_flags_restore	ppf_fprintf
ppf_flags_save	ppf_frac
ppf_flags_set	ppf_free_cache
ppf_flags_test	ppf_free_cache2
ppf_floor	ppf_free_constant_caches
ppf_fma	ppf_free_pool
ppf_fma_algorithm_threads	ppf_free_str
ppf_fma_threads	ppf_frexp

18.7 G

Function	Function
ppf_gamma	ppf_get_flt
ppf_gamma_inc	ppf_get_last_constant_stats
ppf_get_abi_info	ppf_get_ld
ppf_get_constant_interval	ppf_get_ld_2exp
ppf_get_constant_threads	ppf_get_patches
ppf_get_d	ppf_get_prec
ppf_get_d_2exp	ppf_get_q
ppf_get_decimal128	ppf_get_si
ppf_get_decimal64	ppf_get_sj
ppf_get_default_prec	ppf_get_str
ppf_get_default_rounding_mode	ppf_get_str_ndigits
ppf_get_emax	ppf_get_trig_threads
ppf_get_emax_max	ppf_get_ui
ppf_get_emax_min	ppf_get_uj
ppf_get_emin	ppf_get_version
ppf_get_emin_max	ppf_get_z
ppf_get_emin_min	ppf_get_z_2exp
ppf_get_exp	ppf_grandom
ppf_get_f	ppf_greater_p
ppf_get_float128	ppf_greaterequal_p

18.8 H

Function	Function
ppf_hypot	

18.9 I

Function	Function
ppf_inexflag_p	ppf_init_set_si
ppf_inf_p	ppf_init_set_sj
ppf_init	ppf_init_set_str
ppf_init2	ppf_init_set_ui
ppf_init2_ctx	ppf_init_set_uj
ppf_init_set	ppf_init_set_z
ppf_init_set_d	ppf_inits
ppf_init_set_f	ppf_inits2
ppf_init_set_flt	ppf_inp_str
ppf_init_set_ld	ppf_integer_p
ppf_init_set_q	

18.10 J

Function	Function
ppf_j0	ppf_jn
ppf_j1	

18.11 K

Function	Function
ppf_kind_name	

18.12 L

Function	Function
ppf_less_p	ppf_lngamma
ppf_lessequal_p	ppf_log
ppf_lessgreater_p	ppf_log10
ppf_lgamma	ppf_log10p1
ppf_li2	ppf_log1p
ppf_limb_count	ppf_log2
ppf_limbs_read	ppf_log2p1
ppf_limbs_write	ppf_log_ui

18.13 M

Function	Function
ppf_max	ppf_mul_2ui
ppf_min	ppf_mul_algorithm_threads
ppf_min_prec	ppf_mul_d
ppf_modf	ppf_mul_q
ppf_mp_memory_cleanup	ppf_mul_si
ppf_mul	ppf_mul_threads
ppf_mul_2exp	ppf_mul_ui
ppf_mul_2si	ppf_mul_z

18.14 N

Function	Function
ppf_nan_p	ppf_nextbelow
ppf_nanflag_p	ppf_nexttoward
ppf_neg	ppf_nrandom
ppf_nextabove	ppf_number_p

18.15 O

Function	Function
ppf_out_str	ppf_overflow_p

18.16 P

Function	Function
ppf_plan_product	ppf_prec_round
ppf_plan_product_algorithm	ppf_precompute_catalan
ppf_pow	ppf_precompute_constant
ppf_pow_si	ppf_precompute_euler
ppf_pow_sj	ppf_precompute_log2
ppf_pow_ui	ppf_precompute_pi
ppf_pow_uj	ppf_print_rnd_mode
ppf_pow_z	ppf_printf
ppf_pown	ppf_product_profile_name
ppf_powr	ppf_product_profile_sha256

18.17 R

Function	Function
ppf_rec_sqrt	ppf_rint_round
ppf_rec_sqrt_algorithm_threads	ppf_rint_roundeven
ppf_rec_sqrt_threads	ppf_rint_trunc
ppf_regular_p	ppf_root
ppf_reldiff	ppf_rootn_si
ppf_remainder	ppf_rootn_ui
ppf_remquo	ppf_round
ppf_reserve	ppf_round_nearest_away_finish
ppf_rint	ppf_roundeven
ppf_rint_ceil	ppf_rounding_name
ppf_rint_floor	

18.18 S

Function	Function
ppf_sec	ppf_set_ui_2exp
ppf_sech	ppf_set_uj
ppf_set	ppf_set_uj_2exp
ppf_set_allocflag	ppf_set_underflow
ppf_set_constant_threads	ppf_set_z
ppf_set_d	ppf_set_z_2exp
ppf_set_decimal128	ppf_set_zero
ppf_set_decimal64	ppf_setsign
ppf_set_default_prec	ppf_sgn
ppf_set_default_rounding_mode	ppf_si_div
ppf_set_divby0	ppf_si_sub
ppf_set_dyadic_interval	ppf_signbit
ppf_set_emax	ppf_sin
ppf_set_emin	ppf_sin_cos
ppf_set_erangeflag	ppf_sinh
ppf_set_exp	ppf_sinh_cosh
ppf_set_f	ppf_sinpi
ppf_set_float128	ppf_sinu
ppf_set_flt	ppf_snprintf
ppf_set_inexflag	ppf_sprintf
ppf_set_inf	ppf_sqr
ppf_set_ld	ppf_sqr_algorithm_threads
ppf_set_nan	ppf_sqr_threads
ppf_set_nanflag	ppf_sqrt
ppf_set_overflow	ppf_sqrt_ui
ppf_set_prec	ppf_strtofr
ppf_set_prec_raw	ppf_sub
ppf_set_q	ppf_sub_d
ppf_set_si	ppf_sub_q
ppf_set_si_2exp	ppf_sub_si
ppf_set_sj	ppf_sub_ui
ppf_set_sj_2exp	ppf_sub_z
ppf_set_str	ppf_subnormalize
ppf_set_trig_threads	ppf_sum
ppf_set_ui	ppf_swap

18.19 T

Function	Function
ppf_tan	ppf_thread_cleanup
ppf_tanh	ppf_total_order_p
ppf_tanpi	ppf_trunc
ppf_tanu	

18.20 U

Function	Function
ppf_ui_div	ppf_underflow_p
ppf_ui_pow	ppf_unordered_p
ppf_ui_pow_ui	ppf_urandom
ppf_ui_sub	ppf_urandomb

18.21 V

Function	Function
ppf_validate	ppf_vprintf
ppf_vasprintf	ppf_vsnprintf
ppf_vfprintf	ppf_vsprintf

18.22 Y

Function	Function
ppf_y0	ppf_yn
ppf_y1	

18.23 Z

Function	Function
`ppf_z_sub`	`ppf_zeta`
`ppf_zero_p`	`ppf_zeta_ui`

Chapter 19

Types and named constants

Include the installed public header to obtain these declarations. The listings below are reference material. The installed header is the authoritative declaration source. These definitions summarize the governed public types. Public layout compatibility does not make direct field mutation an ordinary application interface. Use lifecycle and setter functions unless you are implementing the documented direct-storage protocol.

19.1 Scalar types

```
#if PPF_HAVE_DECIMAL_FLOAT
__extension__ typedef _Decimal128 ppf_decimal128_t;
#endif
#if PPF_HAVE_DECIMAL_FLOAT
__extension__ typedef _Decimal64 ppf_decimal64_t;
#endif
typedef int64_t ppf_exp_t;
typedef uint32_t ppf_flags_t;
#if PPF_HAVE_FLOAT128
__extension__ typedef __float128 ppf_float128_t;
#endif
typedef uint64_t ppf_prec_t;
```

The optional decimal and 128-bit binary types exist only when their installed-header capability conditions are satisfied. They do not make those C extensions mandatory for the independent core.

19.2 Enumerations

PPF_ABI_ENDIAN_UNKNOWN

```
enum {
    PPF_ABI_ENDIAN_UNKNOWN = 0,
    PPF_ABI_ENDIAN_LITTLE = 1,
    PPF_ABI_ENDIAN_BIG = 2
};
```

PPF_ABI_FEATURE_FLOAT128

```
enum {
    PPF_ABI_FEATURE_FLOAT128 = 1u << 0,
    PPF_ABI_FEATURE_DECIMAL_FLOAT = 1u << 1,
    PPF_ABI_FEATURE_TLS = 1u << 2,
    PPF_ABI_FEATURE_THREAD_EXIT_CLEANUP = 1u << 3
};
```

PPF_FLAG_UNDERFLOW

```
enum {
    PPF_FLAG_UNDERFLOW = 1u << 0,
    PPF_FLAG_OVERFLOW = 1u << 1,
    PPF_FLAG_DIVBY0 = 1u << 2,
    PPF_FLAG_NAN = 1u << 3,
    PPF_FLAG_INEXACT = 1u << 4,
    PPF_FLAG_ERANGE = 1u << 5,
    PPF_FLAG_ALLOC = 1u << 6,
    PPF_FLAGS_ALL = (1u << 7) - 1u
};
```

PPF_NAN_KIND

```
enum {
    PPF_NAN_KIND = 0,
    PPF_INF_KIND = 1,
    PPF_ZERO_KIND = 2,
    PPF_REGULAR_KIND = 3
};
```

PPF_PREC_MIN

```
enum {
    PPF_PREC_MIN = 2
};
```

PPF_STORAGE_OWNED

```
enum {
    PPF_STORAGE_OWNED = 0u,
    PPF_STORAGE_EXTERNAL = 1u << 0
};
```

ppf_constant_id_t

```
typedef enum {
    PPF_CONSTANT_PI = 1,
    PPF_CONSTANT_LOG2 = 2,
    PPF_CONSTANT_EULER = 3,
    PPF_CONSTANT_CATALAN = 4
} ppf_constant_id_t;
```

ppf_free_cache_t

```
typedef enum {
    PPF_FREE_LOCAL_CACHE = 1u << 0,
    PPF_FREE_GLOBAL_CACHE = 1u << 1
} ppf_free_cache_t;
```

ppf_interval_status_t

```
typedef enum {
    PPF_INTERVAL_ERROR = -1,
    PPF_INTERVAL_UNDECIDED = 0,
    PPF_INTERVAL_DECIDED = 1
} ppf_interval_status_t;
```

ppf_kind_t

```
typedef enum {
    PPF_KIND_REGULAR = 0,
    PPF_KIND_ZERO = 1,
    PPF_KIND_INF = 2,
    PPF_KIND_NAN = 3
} ppf_kind_t;
```

ppf_product_algorithm_t

```
typedef enum {
    PPF_PRODUCT_AUTO = 0,
    PPF_PRODUCT_FULL = 1,
    PPF_PRODUCT_KARATSUBA3 = 2,
    PPF_PRODUCT_TOOM3 = 3,
    PPF_PRODUCT_WINDOW = 4
} ppf_product_algorithm_t;
```

ppf_product_operation_t

```
typedef enum {
    PPF_PRODUCT_OP_MULTIPLY = 1,
    PPF_PRODUCT_OP_SQUARE = 2,
    PPF_PRODUCT_OP_FMA = 3,
    PPF_PRODUCT_OP_REC_SQRT = 4
} ppf_product_operation_t;
```

ppf_rnd_t

```
typedef enum {
    PPF_RND_DEFAULT = -1,
    PPF_RNDN = 0,
    PPF_RNDZ = 1,
    PPF_RNDU = 2,
    PPF_RNDD = 3,
    PPF_RNDA = 4,
    PPF_RNDF = 5
} ppf_rnd_t;
```

19.3 Precision, range, and ABI constants

Version and digest strings identify a build or generated contract. They are not mathematical parameters and do not attest to the bytes of a loaded binary.

```
#define PPF_ABI_EPOCH 1
#define PPF_ABI_INFO_VERSION 1
#define PPF_ABI_REVISION 0
#define PPF_ABI_TAG "polyphase-float-abi-1"
#define PPF_ALIASING_CONTRACT_VERSION 1
#define PPF_EMAX_DEFAULT ((ppf_exp_t) 1073741823)
#define PPF_EMAX_MAX ((ppf_exp_t)(INT64_MAX / 4))
#define PPF_EMIN_DEFAULT ((ppf_exp_t)-1073741823)
#define PPF_EMIN_MIN ((ppf_exp_t)(INT64_MIN / 4))
#define PPF_PREC_DEFAULT ((ppf_prec_t)53)
#define PPF_SEMANTIC_CONTRACT_VERSION 1
#define PPF_TERNARY_ERROR INT_MIN
```

19.4 Public layout records

The allocator function-pointer types and advanced custom-storage callbacks are declared in `<polyphase_float.h>`. Keep callback signatures exact. The records below describe ABI shape, not a license to replace initialized objects with raw structure copies.

ppf_abi_info_t

```
typedef struct {
    uint32_t struct_size;
    uint32_t info_version;
    uint32_t abi_epoch;
    uint32_t abi_revision;
    uint32_t endian;
    uint32_t pointer_bits;
    uint32_t size_t_bits;
    uint32_t limb_bits;
    uint32_t feature_flags;
    uint32_t reserved32[7];
    uint64_t sizeof_ppf_allocator_t;
    uint64_t alignof_ppf_allocator_t;
    uint64_t sizeof_ppf_arena_mark_t;
    uint64_t alignof_ppf_arena_mark_t;
    uint64_t sizeof_ppf_context_t;
    uint64_t alignof_ppf_context_t;
    uint64_t sizeof_ppf_t;
    uint64_t alignof_ppf_t;
    uint64_t sizeof_ppf_product_plan_t;
    uint64_t alignof_ppf_product_plan_t;
    uint64_t sizeof_ppf_constant_stats_t;
    uint64_t alignof_ppf_constant_stats_t;
    uint64_t sizeof_ppf_abi_info_t;
    uint64_t alignof_ppf_abi_info_t;
    uint64_t offsetof_context_default_precision;
    uint64_t offsetof_context_emin;
    uint64_t offsetof_context_emax;
    uint64_t offsetof_context_default_rounding;
    uint64_t offsetof_context_flags;
    uint64_t offsetof_context_allocator;
    uint64_t offsetof_context_arena_head;
    uint64_t offsetof_context_arena_current;
    uint64_t offsetof_context_arena_block_bytes;
    uint64_t offsetof_number_limbs;
    uint64_t offsetof_number_allocated_limbs;
    uint64_t offsetof_number_used_limbs;
    uint64_t offsetof_number_precision;
    uint64_t offsetof_number_exponent;
    uint64_t offsetof_number_sign;
    uint64_t offsetof_number_kind;
    uint64_t offsetof_number_allocator;
    uint64_t offsetof_number_storage_flags;
    uint64_t reserved64[8];
} ppf_abi_info_t;
```

ppf_allocator_t

```
typedef struct {
    ppf_malloc_fn malloc_fn;
    ppf_realloc_fn realloc_fn;
    ppf_free_fn free_fn;
    void *opaque;
} ppf_allocator_t;
```

ppf_arena_mark_t

```
typedef struct {
    void *block;
    size_t used;
} ppf_arena_mark_t;
```

ppf_constant_stats_t

```
typedef struct {
    ppf_constant_id_t constant_id;
    ppf_prec_t requested_precision;
    ppf_prec_t computed_precision;
    ppf_prec_t cached_precision;
    unsigned long workers;
    unsigned long attempts;
    unsigned long terms;
    unsigned long packets;
    unsigned long guard_bits;
    unsigned long interval_width_bits;
    ppf_exp_t interval_exponent;
    int cache_hit;
    int ternary;
    double compute_seconds;
    double rounding_seconds;
    double total_seconds;
} ppf_constant_stats_t;
```

ppf_context_t

```
typedef struct {
    ppf_prec_t default_precision;
    ppf_exp_t emin;
    ppf_exp_t emax;
    ppf_rnd_t default_rounding;
    ppf_flags_t flags;
    ppf_allocator_t allocator;
    void *arena_head;
    void *arena_current;
    size_t arena_block_bytes;
} ppf_context_t;
```

ppf_product_plan_t

```
typedef struct {
    ppf_product_algorithm_t algorithm;
    unsigned long requested_workers;
    unsigned long effective_workers;
    int profile_rule_matched;
} ppf_product_plan_t;
```

ppf_t

```
typedef struct {
    mp_limb_t *limbs;
    size_t allocated_limbs;
    size_t used_limbs;
    ppf_prec_t precision;
    ppf_exp_t exponent;
    int sign;
    ppf_kind_t kind;
    ppf_allocator_t allocator;
    uint32_t storage_flags;
} ppf_t;
```

Chapter 20

Complete example programs

These are the complete source files supplied with the manual. Build them through the manual example targets or the checked-example runner. They include their own initialization, status checks, output, and cleanup. Each program's expected output is recorded in the example manifest and the example-results guide.

20.1 intro_first.c

```c
/* SPDX-License-Identifier: LGPL-3.0-or-later */
#include "polyphase_float.h"
#include <stdio.h>

int main(void) {
    ppf_context_t ctx;
    ppf_t a = {0}, b = {0}, product = {0};
    int status = 1;
    int ternary;

    if (ppf_context_init(&ctx) != 0) {
        fputs("Cannot initialize the context.\n", stderr);
        return 1;
    }
    if (ppf_init2_ctx(&a, 128, &ctx) != 0 ||
        ppf_init2_ctx(&b, 128, &ctx) != 0 ||
        ppf_init2_ctx(&product, 128, &ctx) != 0)
        goto cleanup;
    if (ppf_set_ui(&a, 2, PPF_RNDN, &ctx) == PPF_TERNARY_ERROR ||
        ppf_set_ui(&b, 3, PPF_RNDN, &ctx) == PPF_TERNARY_ERROR)
        goto cleanup;

    ternary = ppf_mul(&product, &a, &b, PPF_RNDN, &ctx);
    if (ternary == PPF_TERNARY_ERROR) goto cleanup;
    if (ternary != 0 || ppf_cmp_ui(&product, 6, &ctx) != 0)
        goto cleanup;
```

```
    if (fputs("2 * 3 = ", stdout) == EOF ||
        ppf_out_str(stdout, 10, 6, &product, PPF_RNDN, &ctx) == 0 ||
        fputc('\n', stdout) == EOF || fflush(stdout) == EOF)
        goto cleanup;
    status = 0;

cleanup:
    if (status != 0) fputs("The calculation or output failed.\n", stderr);
    ppf_clear(&product);
    ppf_clear(&b);
    ppf_clear(&a);
    ppf_context_clear(&ctx);
    ppf_thread_cleanup();
    return status;
}
```

20.2 intro_flags.c

```
/* SPDX-License-Identifier: LGPL-3.0-or-later */
#include "polyphase_float.h"
#include <stdio.h>

int main(void) {
    ppf_context_t ctx;
    ppf_t one = {0}, three = {0}, zero = {0}, result = {0};
    ppf_flags_t flags;
    int status = 1;
    int ternary;

    if (ppf_context_init(&ctx) != 0) return 1;
    if (ppf_init2_ctx(&one, 8, &ctx) != 0 ||
        ppf_init2_ctx(&three, 8, &ctx) != 0 ||
        ppf_init2_ctx(&zero, 8, &ctx) != 0 ||
        ppf_init2_ctx(&result, 8, &ctx) != 0)
        goto cleanup;
    if (ppf_set_ui(&one, 1, PPF_RNDN, &ctx) == PPF_TERNARY_ERROR ||
        ppf_set_ui(&three, 3, PPF_RNDN, &ctx) == PPF_TERNARY_ERROR)
        goto cleanup;
    ppf_set_zero(&zero, 1);
    ppf_context_flags_clear(&ctx, PPF_FLAGS_ALL);

    ternary = ppf_div(&result, &one, &three, PPF_RNDN, &ctx);
    if (ternary == PPF_TERNARY_ERROR) goto cleanup;
    flags = ppf_context_flags(&ctx);
    if (ternary == 0 || (flags & PPF_FLAG_INEXACT) == 0)
        goto cleanup;
    if (puts("1 / 3: INEXACT is set") == EOF) goto cleanup;

    ternary = ppf_set_ui(&result, 2, PPF_RNDN, &ctx);
    if (ternary == PPF_TERNARY_ERROR) goto cleanup;
    if (ternary != 0 ||
```

```
        (ppf_context_flags(&ctx) & PPF_FLAG_INEXACT) == 0)
        goto cleanup;
    if (puts("exact assignment: INEXACT is still set") == EOF)
        goto cleanup;

    ppf_context_flags_clear(&ctx, PPF_FLAGS_ALL);
    if (ppf_context_flags(&ctx) != 0) goto cleanup;
    if (puts("after clearing: no flags are set") == EOF) goto cleanup;

    ternary = ppf_div(&result, &one, &zero, PPF_RNDN, &ctx);
    if (ternary == PPF_TERNARY_ERROR) goto cleanup;
    flags = ppf_context_flags(&ctx);
    if (!ppf_inf_p(&result) || ppf_signbit(&result) ||
        (flags & PPF_FLAG_DIVBY0) == 0)
        goto cleanup;
    if (puts("1 / 0: positive infinity; DIVBY0 is set") == EOF)
        goto cleanup;

    ppf_context_flags_clear(&ctx, PPF_FLAGS_ALL);
    ternary = ppf_div(&result, &zero, &zero, PPF_RNDN, &ctx);
    if (ternary == PPF_TERNARY_ERROR) goto cleanup;
    flags = ppf_context_flags(&ctx);
    if (!ppf_nan_p(&result) || (flags & PPF_FLAG_NAN) == 0)
        goto cleanup;
    if (puts("0 / 0: NaN; NAN is set") == EOF || fflush(stdout) == EOF)
        goto cleanup;
    status = 0;

cleanup:
    if (status != 0) fputs("The flag example failed.\n", stderr);
    ppf_clear(&result);
    ppf_clear(&zero);
    ppf_clear(&three);
    ppf_clear(&one);
    ppf_context_clear(&ctx);
    ppf_thread_cleanup();
    return status;
}
```

20.3 intro_precision.c

```
/* SPDX-License-Identifier: LGPL-3.0-or-later */
#include "polyphase_float.h"
#include <stdio.h>

static int print_value(const char *label, const ppf_t *value,
                       ppf_context_t *ctx) {
    if (fputs(label, stdout) == EOF ||
        ppf_out_str(stdout, 10, 12, value, PPF_RNDN, ctx) == 0 ||
        fputc('\n', stdout) == EOF)
        return -1;
```

```
    return 0;
}

int main(void) {
    ppf_context_t ctx;
    ppf_t coarse = {0}, widened = {0}, fresh = {0};
    int status = 1;
    int ternary;

    if (ppf_context_init(&ctx) != 0) return 1;
    if (ppf_init2_ctx(&coarse, 8, &ctx) != 0 ||
        ppf_init2_ctx(&widened, 128, &ctx) != 0 ||
        ppf_init2_ctx(&fresh, 128, &ctx) != 0)
        goto cleanup;
    /* set_str returns a status, unlike the ternary-returning set. */
    if (ppf_set_str(&coarse, "0.1", 10, PPF_RNDN, &ctx) != 0 ||
        ppf_set_str(&fresh, "0.1", 10, PPF_RNDN, &ctx) != 0)
        goto cleanup;
    ternary = ppf_set(&widened, &coarse, PPF_RNDN, &ctx);
    if (ternary == PPF_TERNARY_ERROR) goto cleanup;
    if (ternary != 0 || ppf_cmp(&coarse, &widened, &ctx) != 0 ||
        ppf_cmp(&widened, &fresh, &ctx) == 0)
        goto cleanup;
    if (ppf_context_set_default_precision(&ctx, 256) != 0 ||
        ppf_get_prec(&coarse) != 8 || ppf_get_prec(&fresh) != 128)
        goto cleanup;
    if (print_value("parsed at 8 bits     = ", &coarse, &ctx) != 0 ||
        print_value("copied to 128 bits   = ", &widened, &ctx) != 0 ||
        print_value("parsed at 128 bits   = ", &fresh, &ctx) != 0 ||
        puts("changing the default leaves existing precisions unchanged") == EOF ||
        fflush(stdout) == EOF)
        goto cleanup;
    status = 0;

cleanup:
    if (status != 0) fputs("The precision example failed.\n", stderr);
    ppf_clear(&fresh);
    ppf_clear(&widened);
    ppf_clear(&coarse);
    ppf_context_clear(&ctx);
    ppf_thread_cleanup();
    return status;
}
```

20.4 intro_rounding.c

```c
/* SPDX-License-Identifier: LGPL-3.0-or-later */
#include "polyphase_float.h"
#include <stdio.h>

static int print_result(const char *label, const ppf_t *value,
                        int ternary, ppf_context_t *ctx) {
    const char *direction;
    if (ternary == PPF_TERNARY_ERROR) return -1;
    direction = ternary < 0 ? "below" : ternary > 0 ? "above" : "exact";
    if (fputs(label, stdout) == EOF ||
        ppf_out_str(stdout, 10, 9, value, PPF_RNDN, ctx) == 0 ||
        fprintf(stdout, " (%s)\n", direction) < 0)
        return -1;
    return 0;
}

int main(void) {
    ppf_context_t ctx;
    ppf_t one = {0}, three = {0};
    ppf_t lower = {0}, nearest = {0}, upper = {0};
    int down, near, up;
    int status = 1;

    if (ppf_context_init(&ctx) != 0) return 1;
    if (ppf_init2_ctx(&one, 8, &ctx) != 0 ||
        ppf_init2_ctx(&three, 8, &ctx) != 0 ||
        ppf_init2_ctx(&lower, 8, &ctx) != 0 ||
        ppf_init2_ctx(&nearest, 8, &ctx) != 0 ||
        ppf_init2_ctx(&upper, 8, &ctx) != 0)
        goto cleanup;
    if (ppf_set_ui(&one, 1, PPF_RNDN, &ctx) == PPF_TERNARY_ERROR ||
        ppf_set_ui(&three, 3, PPF_RNDN, &ctx) == PPF_TERNARY_ERROR)
        goto cleanup;
    down = ppf_div(&lower, &one, &three, PPF_RNDD, &ctx);
    near = ppf_div(&nearest, &one, &three, PPF_RNDN, &ctx);
    up = ppf_div(&upper, &one, &three, PPF_RNDU, &ctx);
    if (down == PPF_TERNARY_ERROR || near == PPF_TERNARY_ERROR ||
        up == PPF_TERNARY_ERROR)
        goto cleanup;
    if (!(down < 0 && near > 0 && up > 0) ||
        ppf_cmp(&lower, &upper, &ctx) >= 0 ||
        ppf_cmp(&nearest, &upper, &ctx) != 0)
        goto cleanup;
    if (print_result("down    = ", &lower, down, &ctx) != 0 ||
        print_result("nearest = ", &nearest, near, &ctx) != 0 ||
        print_result("up      = ", &upper, up, &ctx) != 0 ||
        fflush(stdout) == EOF)
        goto cleanup;
    status = 0;

cleanup:
```

```
    if (status != 0) fputs("The rounding example failed.\n", stderr);
    ppf_clear(&upper);
    ppf_clear(&nearest);
    ppf_clear(&lower);
    ppf_clear(&three);
    ppf_clear(&one);
    ppf_context_clear(&ctx);
    ppf_thread_cleanup();
    return status;
}
```

20.5 numeric_constants.c

```
/* SPDX-License-Identifier: LGPL-3.0-or-later */
#include "polyphase_float.h"
#include <stdio.h>
#define REQUIRE(c) do { if (!(c)) goto cleanup; } while (0)
int main(void) {
    ppf_context_t ctx;
    ppf_t direct = {0}, certified = {0};
    mpz_t lower, upper;
    ppf_exp_t exponent = 0;
    int status = 1, direct_t, interval_t = 0;
    if (ppf_context_init(&ctx) != 0) return 1;
    mpz_inits(lower, upper, NULL);
    REQUIRE(ppf_init2_ctx(&direct, 80, &ctx) == 0);
    REQUIRE(ppf_init2_ctx(&certified, 80, &ctx) == 0);
    direct_t = ppf_const_pi(&direct, PPF_RNDN, &ctx);
    REQUIRE(direct_t != PPF_TERNARY_ERROR);
    /* Interval export is a status API using the thread-default context. */
    REQUIRE(ppf_get_constant_interval(PPF_CONSTANT_PI, lower, upper,
                                      &exponent, 160, 1) == 0);
    REQUIRE(mpz_cmp(lower, upper) <= 0);
    REQUIRE(ppf_set_dyadic_interval(&certified, lower, upper, exponent,
                PPF_RNDN, &interval_t, &ctx) == PPF_INTERVAL_DECIDED);
    REQUIRE(ppf_equal_p(&direct, &certified));
    REQUIRE((direct_t > 0) == (interval_t > 0));
    REQUIRE((direct_t < 0) == (interval_t < 0));
    puts("Pi: direct evaluation and exported interval agree at 80 bits.");
    puts("The interval proves the value and its rounding direction.");
    status = 0;
cleanup:
    if (status) fputs("Constant example failed.\n", stderr);
    mpz_clears(lower, upper, NULL);
    ppf_clear(&direct); ppf_clear(&certified);
    ppf_context_clear(&ctx); ppf_free_constant_caches(); ppf_thread_cleanup();
    return status;
}
```

20.6 numeric_elementary.c

```c
/* SPDX-License-Identifier: LGPL-3.0-or-later */
#include "polyphase_float.h"
#include <stdio.h>
#define REQUIRE(c) do { if (!(c)) goto cleanup; } while (0)
#define NUMBER_CALL(c) REQUIRE((c) != PPF_TERNARY_ERROR)
int main(void) {
    ppf_context_t ctx;
    ppf_t x = {0}, one = {0}, sum = {0}, result = {0}, pi = {0};
    int status = 1;
    if (ppf_context_init(&ctx) != 0) return 1;
    REQUIRE(ppf_init2_ctx(&x, 53, &ctx) == 0);
    REQUIRE(ppf_init2_ctx(&one, 53, &ctx) == 0);
    REQUIRE(ppf_init2_ctx(&sum, 53, &ctx) == 0);
    REQUIRE(ppf_init2_ctx(&result, 53, &ctx) == 0);
    REQUIRE(ppf_init2_ctx(&pi, 53, &ctx) == 0);
    REQUIRE(ppf_set_ui_2exp(&x, 1, -80, PPF_RNDN, &ctx) == 0);
    REQUIRE(ppf_set_ui(&one, 1, PPF_RNDN, &ctx) == 0);
    NUMBER_CALL(ppf_add(&sum, &one, &x, PPF_RNDN, &ctx));
    REQUIRE(ppf_equal_p(&sum, &one));
    NUMBER_CALL(ppf_log(&result, &sum, PPF_RNDN, &ctx));
    REQUIRE(ppf_zero_p(&result));
    puts("log of the rounded 1 + 2^-80 is zero at 53 bits.");
    NUMBER_CALL(ppf_log1p(&result, &x, PPF_RNDN, &ctx));
    REQUIRE(ppf_regular_p(&result) && !ppf_signbit(&result));
    NUMBER_CALL(ppf_expm1(&result, &x, PPF_RNDN, &ctx));
    REQUIRE(ppf_regular_p(&result) && !ppf_signbit(&result));
    puts("log1p(2^-80) and expm1(2^-80) remain positive.");
    REQUIRE(ppf_sinpi(&result, &one, PPF_RNDN, &ctx) == 0);
    REQUIRE(ppf_zero_p(&result));
    NUMBER_CALL(ppf_const_pi(&pi, PPF_RNDN, &ctx));
    NUMBER_CALL(ppf_sin(&result, &pi, PPF_RNDN, &ctx));
    REQUIRE(ppf_regular_p(&result));
    puts("sinpi(1) is exactly zero; sin of rounded pi is not.");
    status = 0;
cleanup:
    if (status) fputs("Elementary-function example failed.\n", stderr);
    ppf_clear(&x); ppf_clear(&one); ppf_clear(&sum);
    ppf_clear(&result); ppf_clear(&pi);
    ppf_context_clear(&ctx); ppf_free_constant_caches(); ppf_thread_cleanup();
    return status;
}
```

20.7 numeric_import.c

```c
/* SPDX-License-Identifier: LGPL-3.0-or-later */
#include "polyphase_float.h"
#include <float.h>
#include <stdio.h>
#define REQUIRE(c) do { if (!(c)) goto cleanup; } while (0)
int main(void) {
    ppf_context_t ctx;
    ppf_t decimal = {0}, native = {0};
    mpq_t target, qdecimal, qnative, edecimal, enative, widened;
    int status = 1;
    if (FLT_RADIX != 2 || DBL_MANT_DIG != 53) {
        fputs("This example requires a 53-bit binary double.\n", stderr);
        return 1;
    }
    if (ppf_context_init(&ctx) != 0) return 1;
    mpq_inits(target, qdecimal, qnative, edecimal, enative, widened, NULL);
    REQUIRE(ppf_init2_ctx(&decimal, 256, &ctx) == 0);
    REQUIRE(ppf_init2_ctx(&native, 256, &ctx) == 0);
    REQUIRE(ppf_set_str(&decimal, "0.1", 10, PPF_RNDN, &ctx) == 0);
    REQUIRE(ppf_set_d(&native, 0.1, PPF_RNDN, &ctx) == 0);
    mpq_set_ui(target, 1, 10);
    ppf_get_q(qdecimal, &decimal, &ctx);
    ppf_get_q(qnative, &native, &ctx);
    mpq_sub(edecimal, qdecimal, target); mpq_abs(edecimal, edecimal);
    mpq_sub(enative, qnative, target); mpq_abs(enative, enative);
    REQUIRE(mpq_sgn(edecimal) != 0 && mpq_sgn(enative) != 0);
    REQUIRE(mpq_cmp(edecimal, enative) < 0);
    REQUIRE(ppf_prec_round(&native, 512, PPF_RNDN, &ctx) == 0);
    ppf_get_q(widened, &native, &ctx);
    REQUIRE(mpq_equal(qnative, widened));
    puts("Neither binary value is exactly 1/10.");
    puts("The 256-bit decimal parse is closer to 1/10.");
    puts("Widening the native import preserves its old value.");
    status = 0;
cleanup:
    if (status) fputs("Import example failed.\n", stderr);
    mpq_clears(target, qdecimal, qnative, edecimal, enative, widened, NULL);
    ppf_clear(&decimal); ppf_clear(&native);
    ppf_context_clear(&ctx); ppf_thread_cleanup();
    return status;
}
```

20.8 numeric_ranges.c

```c
/* SPDX-License-Identifier: LGPL-3.0-or-later */
#include "polyphase_float.h"
#include <stdio.h>
#define REQUIRE(c) do { if (!(c)) goto cleanup; } while (0)
int main(void) {
    ppf_context_t ctx;
    ppf_t a = {0}, b = {0}, result = {0};
    int status = 1, t;
    if (ppf_context_init(&ctx) != 0) return 1;
    REQUIRE(ppf_init2_ctx(&a, 8, &ctx) == 0);
    REQUIRE(ppf_init2_ctx(&b, 8, &ctx) == 0);
    REQUIRE(ppf_init2_ctx(&result, 2, &ctx) == 0);
    REQUIRE(ppf_context_set_range(&ctx, -9, 9) == 0);
    REQUIRE(ppf_set_ui(&a, 7, PPF_RNDN, &ctx) == 0);
    REQUIRE(ppf_set_ui_2exp(&b, 15, -10, PPF_RNDN, &ctx) == 0);
    ppf_context_flags_clear(&ctx, PPF_FLAGS_ALL);
    t = ppf_div(&result, &a, &b, PPF_RNDN, &ctx);
    REQUIRE(t != PPF_TERNARY_ERROR && t > 0 && ppf_inf_p(&result));
    REQUIRE(ppf_context_flags(&ctx) == (PPF_FLAG_OVERFLOW | PPF_FLAG_INEXACT));
    puts("Nearest quotient: +infinity; overflow and inexact.");
    ppf_context_flags_clear(&ctx, PPF_FLAGS_ALL);
    t = ppf_div(&result, &a, &b, PPF_RNDZ, &ctx);
    REQUIRE(t != PPF_TERNARY_ERROR && t < 0);
    REQUIRE(ppf_cmp_ui(&result, 384, &ctx) == 0);
    REQUIRE(ppf_context_flags(&ctx) == PPF_FLAG_INEXACT);
    puts("Toward-zero quotient: 384; inexact only.");
    REQUIRE(ppf_context_set_range(&ctx, -30, 100) == 0);
    REQUIRE(ppf_set_ui_2exp(&result, 1, -31, PPF_RNDN, &ctx) == 0);
    ppf_context_flags_clear(&ctx, PPF_FLAGS_ALL);
    t = ppf_subnormalize(&result, 0, PPF_RNDN, &ctx);
    REQUIRE(t == 0 && ppf_context_flags(&ctx) == PPF_FLAG_UNDERFLOW);
    puts("Exact subnormal emulation: underflow without inexact.");
    status = 0;
cleanup:
    if (status) fputs("Range example failed.\n", stderr);
    ppf_clear(&a); ppf_clear(&b); ppf_clear(&result);
    ppf_context_clear(&ctx); ppf_thread_cleanup();
    return status;
}
```

20.9 numeric_refinement.c

```c
/* SPDX-License-Identifier: LGPL-3.0-or-later */
#include "polyphase_float.h"
#include <stdio.h>
#define REQUIRE(c) do { if (!(c)) goto cleanup; } while (0)
int main(void) {
    const unsigned scales[] = {8, 16, 32, 64, 128};
    ppf_context_t ctx;
    ppf_t result = {0};
    mpz_t radicand, lower, upper, remainder;
    int status = 1, ternary = 777, decided = 0;
    if (ppf_context_init(&ctx) != 0) return 1;
    mpz_inits(radicand, lower, upper, remainder, NULL);
    REQUIRE(ppf_init2_ctx(&result, 53, &ctx) == 0);
    REQUIRE(ppf_set_ui(&result, 1, PPF_RNDN, &ctx) == 0);
    for (size_t i = 0; i < sizeof(scales) / sizeof(scales[0]); ++i) {
        unsigned k = scales[i];
        mpz_set_ui(radicand, 2);
        mpz_mul_2exp(radicand, radicand, 2u * k);
        mpz_sqrtrem(lower, remainder, radicand);
        REQUIRE(mpz_sgn(remainder) > 0);
        mpz_add_ui(upper, lower, 1);
        ppf_interval_status_t s = ppf_set_dyadic_interval(
            &result, lower, upper, -(ppf_exp_t)k, PPF_RNDN, &ternary, &ctx);
        REQUIRE(s != PPF_INTERVAL_ERROR);
        if (s == PPF_INTERVAL_UNDECIDED) {
            REQUIRE(ternary == 777 && ppf_cmp_ui(&result, 1, &ctx) == 0);
            printf("scale %u: refine\n", k);
        } else {
            REQUIRE(ternary > 0);
            printf("scale %u: certified 53-bit sqrt(2), rounded upward\n", k);
            decided = 1;
            break;
        }
    }
    REQUIRE(decided);
    status = 0;
cleanup:
    if (status) fputs("Refinement example did not obtain its certificate.\n", stderr);
    mpz_clears(radicand, lower, upper, remainder, NULL);
    ppf_clear(&result); ppf_context_clear(&ctx); ppf_thread_cleanup();
    return status;
}
```

20.10 numeric_special_domains.c

```c
/* SPDX-License-Identifier: LGPL-3.0-or-later */
#include "polyphase_float.h"
#include <stdio.h>
#define REQUIRE(c) do { if (!(c)) goto cleanup; } while (0)
int main(void) {
    ppf_context_t ctx;
    ppf_t x = {0}, result = {0};
    int status = 1, t, gamma_sign = 0;
    if (ppf_context_init(&ctx) != 0) return 1;
    REQUIRE(ppf_init2_ctx(&x, 80, &ctx) == 0);
    REQUIRE(ppf_init2_ctx(&result, 80, &ctx) == 0);
    REQUIRE(ppf_set_ui(&x, 5, PPF_RNDN, &ctx) == 0);
    REQUIRE(ppf_gamma(&result, &x, PPF_RNDN, &ctx) == 0);
    REQUIRE(ppf_cmp_ui(&result, 24, &ctx) == 0);
    puts("Gamma(5) = 24 exactly.");
    REQUIRE(ppf_set_si_2exp(&x, -1, -1, PPF_RNDN, &ctx) == 0);
    t = ppf_lgamma(&result, &gamma_sign, &x, PPF_RNDN, &ctx);
    REQUIRE(t != PPF_TERNARY_ERROR && ppf_regular_p(&result));
    REQUIRE(gamma_sign == -1);
    puts("lgamma(-1/2) returns a finite log magnitude and sign -1.");
    ppf_context_flags_clear(&ctx, PPF_FLAGS_ALL);
    t = ppf_lngamma(&result, &x, PPF_RNDN, &ctx);
    REQUIRE(t != PPF_TERNARY_ERROR && ppf_nan_p(&result));
    REQUIRE((ppf_context_flags(&ctx) & PPF_FLAG_NAN) != 0);
    puts("lngamma(-1/2) reports a real-domain NaN.");
    REQUIRE(ppf_set_ui(&x, 501, PPF_RNDN, &ctx) == 0);
    REQUIRE(ppf_cmpabs_ui(&x, 500, &ctx) > 0);
    ppf_context_flags_clear(&ctx, PPF_FLAGS_ALL);
    t = ppf_ai(&result, &x, PPF_RNDN, &ctx);
    REQUIRE(t == PPF_TERNARY_ERROR);
    REQUIRE((ppf_context_flags(&ctx) & PPF_FLAG_ERANGE) != 0);
    REQUIRE((ppf_context_flags(&ctx) & PPF_FLAG_ALLOC) == 0);
    puts("Ai(501) exceeds the implementation domain: ERANGE, not ALLOC.");
    status = 0;
cleanup:
    if (status) fputs("Special-function example failed.\n", stderr);
    ppf_clear(&x); ppf_clear(&result); ppf_context_clear(&ctx);
    ppf_free_constant_caches(); ppf_thread_cleanup();
    return status;
}
```

20.11 systems_allocation.c

```c
/* SPDX-License-Identifier: LGPL-3.0-or-later */
/* Core-only example: one controlled allocation failure and recovery. */
#include <polyphase_float.h>

#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int fail_next;
    size_t live_blocks;
} allocation_state;

static void *tracked_malloc(void *opaque, size_t bytes) {
    allocation_state *state = opaque;
    void *block;
    if (state->fail_next) { state->fail_next = 0; return NULL; }
    block = malloc(bytes);
    if (block) ++state->live_blocks;
    return block;
}

static void *tracked_realloc(void *opaque, void *block,
                             size_t old_bytes, size_t new_bytes) {
    allocation_state *state = opaque;
    int was_null = block == NULL;
    void *replacement;
    (void)old_bytes;
    if (state->fail_next) { state->fail_next = 0; return NULL; }
    replacement = realloc(block, new_bytes);
    if (replacement && was_null) ++state->live_blocks;
    return replacement;
}

static void tracked_free(void *opaque, void *block, size_t bytes) {
    allocation_state *state = opaque;
    (void)bytes;
    if (block) { --state->live_blocks; free(block); }
}

int main(void) {
    allocation_state state = {0, 0};
    ppf_allocator_t allocator = {
        tracked_malloc, tracked_realloc, tracked_free, &state
    };
    ppf_context_t ctx;
    ppf_t value = {0};
    int status = EXIT_FAILURE;

    if (ppf_context_init_custom(&ctx, &allocator) != 0) return status;
    if (ppf_init2_ctx(&value, 80, &ctx) != 0 ||
        ppf_set_ui(&value, 37, PPF_RNDN, &ctx) == PPF_TERNARY_ERROR)
```

```
        goto cleanup;

    ppf_context_flags_clear(&ctx, PPF_FLAGS_ALL);
    state.fail_next = 1;
    if (ppf_set_str(&value, "1.25", 10, PPF_RNDN, &ctx) == 0 ||
        state.fail_next != 0 || ppf_cmp_ui(&value, 37, &ctx) != 0 ||
        !(ppf_context_flags(&ctx) & PPF_FLAG_ALLOC)) goto cleanup;

    ppf_context_flags_clear(&ctx, PPF_FLAGS_ALL);
    if (ppf_set_str(&value, "1.25", 10, PPF_RNDN, &ctx) != 0 ||
        ppf_cmp_d(&value, 1.25, &ctx) != 0 ||
        (ppf_context_flags(&ctx) & PPF_FLAG_ALLOC)) goto cleanup;
    status = EXIT_SUCCESS;

cleanup:
    ppf_clear(&value);
    ppf_context_clear(&ctx);
    ppf_thread_cleanup();
    if (state.live_blocks != 0) status = EXIT_FAILURE;
    if (status == EXIT_SUCCESS) {
        if (puts("allocation failure preserved 37; retry produced 1.25") == EOF)
            status = EXIT_FAILURE;
    } else {
        fputs("allocation example failed\n", stderr);
    }
    return status;
}
```

20.12 systems_io.c

```
/* SPDX-License-Identifier: LGPL-3.0-or-later */
/* Core-only example: text ownership and a PPFIF round trip. */
#include <polyphase_float.h>

#include <inttypes.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    ppf_context_t ctx;
    ppf_t value = {0}, restored = {0};
    ppf_exp_t exponent = 0;
    char *digits = NULL, *formatted = NULL;
    FILE *record = NULL;
    int status = EXIT_FAILURE;

    if (ppf_context_init(&ctx) != 0) return status;
    if (ppf_init2_ctx(&value, 80, &ctx) != 0 ||
        ppf_init2_ctx(&restored, 17, &ctx) != 0) goto cleanup;
    if (ppf_set_str(&value, "1.25", 10, PPF_RNDN, &ctx) != 0)
```

```
        goto cleanup;

    digits = ppf_get_str(NULL, &exponent, 10, 5, &value, PPF_RNDN, &ctx);
    if (!digits || strcmp(digits, "12500") != 0 || exponent != 1)
        goto cleanup;
    if (ppf_asprintf(&formatted, "value=%.2RNf", &value) < 0 ||
        !formatted || strcmp(formatted, "value=1.25") != 0) goto cleanup;

    record = tmpfile();
    if (!record || ppf_fpif_export(record, &value) != 0 ||
        fflush(record) != 0 || fseek(record, 0L, SEEK_SET) != 0)
        goto cleanup;
    if (ppf_fpif_import(&restored, record, &ctx) != 0 ||
        ppf_cmp(&restored, &value, &ctx) != 0 ||
        ppf_get_prec(&restored) != ppf_get_prec(&value)) goto cleanup;
    if (fclose(record) != 0) { record = NULL; goto cleanup; }
    record = NULL;

    if (printf("digits=%s exponent=%" PRId64 "\n", digits, exponent) < 0 ||
        puts(formatted) == EOF ||
        puts("PPFIF round trip: value and precision preserved") == EOF)
        goto cleanup;
    status = EXIT_SUCCESS;

cleanup:
    if (record) (void)fclose(record);
    free(formatted);
    ppf_free_str(digits);
    ppf_clear(&restored);
    ppf_clear(&value);
    ppf_context_clear(&ctx);
    ppf_thread_cleanup();
    if (status != EXIT_SUCCESS) fputs("I/O example failed\n", stderr);
    return status;
}
```

20.13 systems_mpfr_bridge.c

```
/* SPDX-License-Identifier: LGPL-3.0-or-later */
/* Requires the MPFR adapter, official MPFR headers, and the core library. */
#include <polyphase_float_mpfr.h>

#include <stdio.h>
#include <stdlib.h>

int main(void) {
    ppf_context_t ctx;
    ppf_t value = {0};
    mpfr_t input, output;
    int status = EXIT_FAILURE;
```

```
    if (ppf_context_init(&ctx) != 0) return status;
    mpfr_init2(input, 128);
    mpfr_init2(output, 53);
    if (ppf_init2_ctx(&value, 53, &ctx) != 0) goto cleanup;
    if (mpfr_set_ui(input, 5, MPFR_RNDN) != 0 ||
        mpfr_div_ui(input, input, 4, MPFR_RNDN) != 0) goto cleanup;
    if (ppf_import_mpfr_exact(&value, input, &ctx) != 0 ||
        ppf_get_prec(&value) != 128 ||
        ppf_mul_ui(&value, &value, 8, PPF_RNDN, &ctx) != 0 ||
        ppf_get_mpfr(output, &value, MPFR_RNDN) != 0 ||
        mpfr_cmp_ui(output, 10) != 0) goto cleanup;
    status = EXIT_SUCCESS;

cleanup:
    ppf_clear(&value);
    mpfr_clear(output);
    mpfr_clear(input);
    mpfr_free_cache();
    ppf_context_clear(&ctx);
    ppf_thread_cleanup();
    if (status == EXIT_SUCCESS) {
        if (puts("MPFR -> Polyphase -> MPFR: 10") == EOF)
            status = EXIT_FAILURE;
    } else {
        fputs("MPFR bridge example failed\n", stderr);
    }
    return status;
}
```

20.14 systems_threads.c

```
/* SPDX-License-Identifier: LGPL-3.0-or-later */
/* Core + POSIX threads: shared immutable input, private mutable state. */
#include <polyphase_float.h>

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    const ppf_t *input;
    int success;
} worker_arguments;

static void *square_worker(void *opaque) {
    worker_arguments *args = opaque;
    ppf_context_t ctx;
    ppf_t result = {0};
    if (ppf_context_init(&ctx) != 0) return NULL;
    if (ppf_init2_ctx(&result, 80, &ctx) == 0 &&
        ppf_sqr(&result, args->input, PPF_RNDN, &ctx) != PPF_TERNARY_ERROR &&
```

```
        ppf_cmp_ui(&result, 49, &ctx) == 0)
        args->success = 1;
    ppf_clear(&result);
    ppf_context_clear(&ctx);
    ppf_thread_cleanup();
    return NULL;
}

int main(void) {
    ppf_context_t ctx;
    ppf_t input = {0};
    pthread_t threads[2];
    worker_arguments args[2] = {{&input, 0}, {&input, 0}};
    unsigned started = 0;
    int status = EXIT_FAILURE;

    if (ppf_context_init(&ctx) != 0) return status;
    if (ppf_init2_ctx(&input, 80, &ctx) != 0 ||
        ppf_set_ui(&input, 7, PPF_RNDN, &ctx) == PPF_TERNARY_ERROR)
        goto cleanup;
    for (unsigned i = 0; i < 2; ++i) {
        if (pthread_create(&threads[i], NULL, square_worker, &args[i]) != 0)
            break;
        ++started;
    }
    for (unsigned i = 0; i < started; ++i) {
        /* Do not release a shared input unless every reader has joined. */
        if (pthread_join(threads[i], NULL) != 0) {
            fputs("could not join a worker\n", stderr);
            return EXIT_FAILURE;
        }
    }
    if (started == 2 && args[0].success && args[1].success)
        status = EXIT_SUCCESS;

cleanup:
    ppf_clear(&input);
    ppf_context_clear(&ctx);
    ppf_thread_cleanup();
    if (status == EXIT_SUCCESS) {
        if (puts("2 workers read one immutable input; both produced 49") == EOF)
            status = EXIT_FAILURE;
    } else {
        fputs("thread example failed\n", stderr);
    }
    return status;
}
```

Chapter 21

Glossary

ABI. The application binary interface: calling conventions, symbol names, public object layouts, enum values, and related requirements that let an already compiled application use a compatible library.

Aliasing. Two arguments refer to the same object or to overlapping storage. An allowed single-output in-place operation does not imply that two distinct outputs may be collapsed into one object.

Allocator provenance. The identity and state of the allocator that owns an allocation. Return storage through its documented release function rather than choosing a deallocator by the pointer's C type.

Arbitrary precision. The application chooses the finite significand precision. This is not infinite precision, unlimited memory, or a guarantee that every difficult operation will certify at every requested precision.

Binary exponent. The power of two that scales a binary significand. The library's normalized exponent conventions and configured bounds determine range behavior. A decimal output exponent uses its own stated radix.

Borrowed storage. Memory the caller may access subject to lifetime rules but does not own and must not release. Changing the owning object can invalidate an object-storage pointer.

Certificate. An enclosure or other exact mathematical evidence used to justify a rounded result. Certification can remain undecided within the available precision or resource budget.

Context. The configuration and state used by operations: defaults, rounding, exponent bounds, sticky flags, allocator callbacks, and working storage. An explicit mutable context requires exclusive access.

Correct rounding. The stored result equals the result of rounding the exact mathematical value once to the selected representable set and rule. This property is stronger than observing agreement at two working precisions.

Destination precision. The precision attached to the output object. Passing a higher-precision context does not silently enlarge an existing destination. Some named precision-changing operations have explicit rules.

Directed rounding. Rounding toward positive infinity, negative infinity, zero, or away from zero. For a negative value, toward positive infinity and away from zero point in different directions.

Dyadic. An integer or rational scaled by an integer power of two. Every finite binary floating-point value is a dyadic rational.

Exact input. The mathematical value represented by the supplied data. Importing a C `double` exactly preserves that binary `double`, including its earlier rounding. It does not recover an intended decimal value.

Exponent range. The configured interval used by operations to classify and settle their output. Range limits, storage limits, and precision limits are separate constraints.

Faithful rounding. Selection of an allowed neighboring representable value. It is not the same contract as the unique nearest or directed result, and its ternary output must be interpreted under the faithful-mode rules.

Flag. A sticky diagnostic bit in context state. Flags can outlive the operation that raised them. Clear or save them deliberately when attributing a diagnostic to one computation.

Fused operation. An operation that combines its terms before a final rounding, rather than rounding each intermediate expression separately. Its result can differ from a sequence of individually rounded calls.

GMP. The GNU Multiple Precision Arithmetic Library. Polyphase uses its integer and rational machinery. GMP's own allocation behavior is outside the recoverable project-allocator rollback contract.

Guard precision. Extra working precision used inside an algorithm. It does not by itself prove that a final result is correctly rounded.

Inexact. The stored numerical result differs from the exact value in the operation's contract. Conversion and formatting can introduce their own rounding steps.

Interval settlement. Deciding whether every exact value in a supplied enclosure has a common rounded output, compatible ternary direction, and the required common range-flag state. An undecided result asks for better evidence, not a guessed midpoint.

LP64. The supported C data model in which `long` and pointers are 64 bits while `int` is 32 bits. This differs from native 64-bit Windows' usual LLP64 model.

MPFR. A separate multiple-precision floating-point library. Polyphase's optional adapter imports and exports its values; the independent production core does not need it. Dif-

ferential test agreement is finite execution evidence, not a replacement for mathematical arguments.

NaN. Not a Number, a special value used for invalid numerical results and some numerical-rejection paths. Inspect status and flags to understand the particular outcome.

Nearest, ties to even. Select the nearest representable value. At an exact midpoint choose the candidate whose retained least-significant significand bit is even.

Numerical rejection. An operation cannot produce a certified result under its resource or admission policy. It may return `PPF_TERNARY_ERROR`, raise `PPF_FLAG_ERANGE`, and commit NaN. This differs from allocation rollback.

Precision. The number of bits available in the significand, including the leading significant bit. The rough decimal information capacity is `precision * log10(2)`, not a promise that a chosen decimal expression has that many correct digits.

PPFIF. Polyphase's binary interchange format. Its import contract includes admission limits and validation before committing the numerical destination.

Remainder bound. A bound on terms omitted from a finite series or asymptotic expression. A decreasing term or a small last term is not alone a universal remainder proof.

Representable neighbor. The adjacent value in the destination's precision and range model. Spacing changes with binary exponent.

SONAME. The ELF shared-library name recorded for runtime loading. The project's release version and its governed ABI SONAME need not have the same major number.

Status return. A success or failure code, predicate, count, or other non-ternary return. An `int` declaration alone does not identify a numerical ternary. In particular, `ppf_set_str` and `ppf_strtofr` have different return contracts.

Sticky diagnostic. A diagnostic that remains set until explicitly cleared or restored. A successful later operation does not erase it.

Subnormalization. The explicit operation that implements the library's reduced-precision range semantics when requested. An exact reduced-precision result can still raise underflow under that operation's contract, while inexactness depends on the rounding relation.

Ternary. A numerical return whose sign compares the stored result with the exact result. Zero means exact, negative means below, and positive means above, subject to the selected rounding contract. The error sentinel is not a numerical direction.

Thread default. State owned by the calling thread and selected by APIs that permit a null context. It is distinct from an application-owned explicit context and from global synchronized caches.

Transactional allocation failure. An unrecovered project-allocation failure preserves the prior numerical destination under the operation's documented contract. This does not undo file I/O, kernel effects, GMP failure, or arbitrary application callbacks.

ULP. A unit in the last retained place, measured at a specified precision and exponent. It is not a fixed absolute distance over the whole number line.

Working precision. The precision used to construct intermediate evidence or values. It may exceed the destination precision and may increase during bounded certification attempts.